

SpringCamp 2025

10th Anniversary

SpringCamp 2025

Connection, Sharing, Growth, Festival

SpringCamp 2025

통합의 근본! 인가 플랫폼

HR SaaS 는 왜 복잡할까?

이명현



flex : 이명현

HR 문제 해결을 위해 몰입하고 있습니다.

현) 플렉스팀 Engineering Lead

- Platform Division Lead

전) NAVER Tech Lead

- Plasma Project (네이버페이 아키텍처 재구성)
- 오픈소스 Fixture Monkey, Spring JDBC Plus

전) 삼성 SDS



1. All in one HR SaaS : 통합의 가치

2. HR 에서 인가 판단은 왜 어려울까?

3. ReBAC : 통합 인가 플랫폼

4. 통합의 완성을 위한 플랫폼 고도화

5. 단단한 플랫폼 위에 쌓아 올리는 통합의 가치 실현

1. All in one HR SaaS : 통합의 가치

2. HR 에서 인가 판단은 왜 어려울까?

3. ReBAC : 통합 인가 플랫폼

4. 통합의 완성을 위한 플랫폼 고도화

5. 단단한 플랫폼 위에 쌓아 올리는 통합의 가치 실현

1. All in one HR SaaS : 통합의 가치

2. HR 에서 인가 판단은 왜 어려울까?

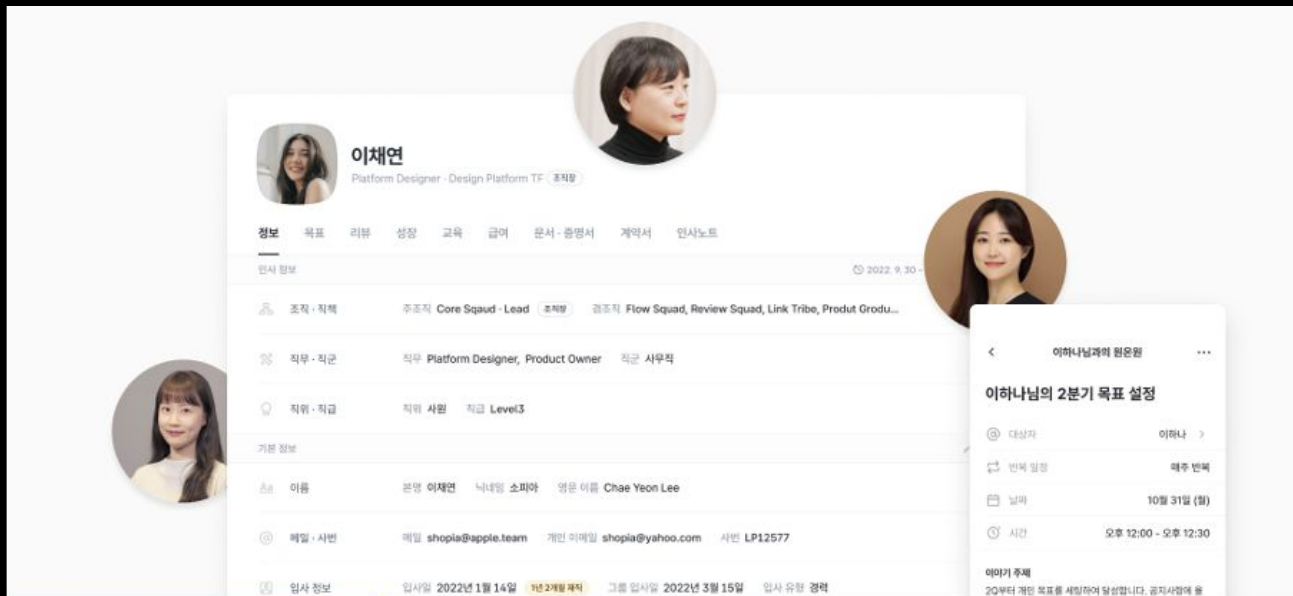
3. ReBAC : 통합 인가 플랫폼

4. 통합의 완성을 위한 플랫폼 고도화

5. 단단한 플랫폼 위에 쌓아 올리는 통합의 가치 실현

1. All in one HR SaaS : 통합의 가치

1. All in one HR SaaS : 통합의 가치



Product

올인원 HR 플랫폼



Partners

진단 - 솔루션 - 운영
모두 지원하는 전문가
서비스



Outcome

기술과 사람이 함께
만드는
Insight-Driven HR

HR

기업의 목표를 달성하기 위해

사람들이 모여 조직화 하고,

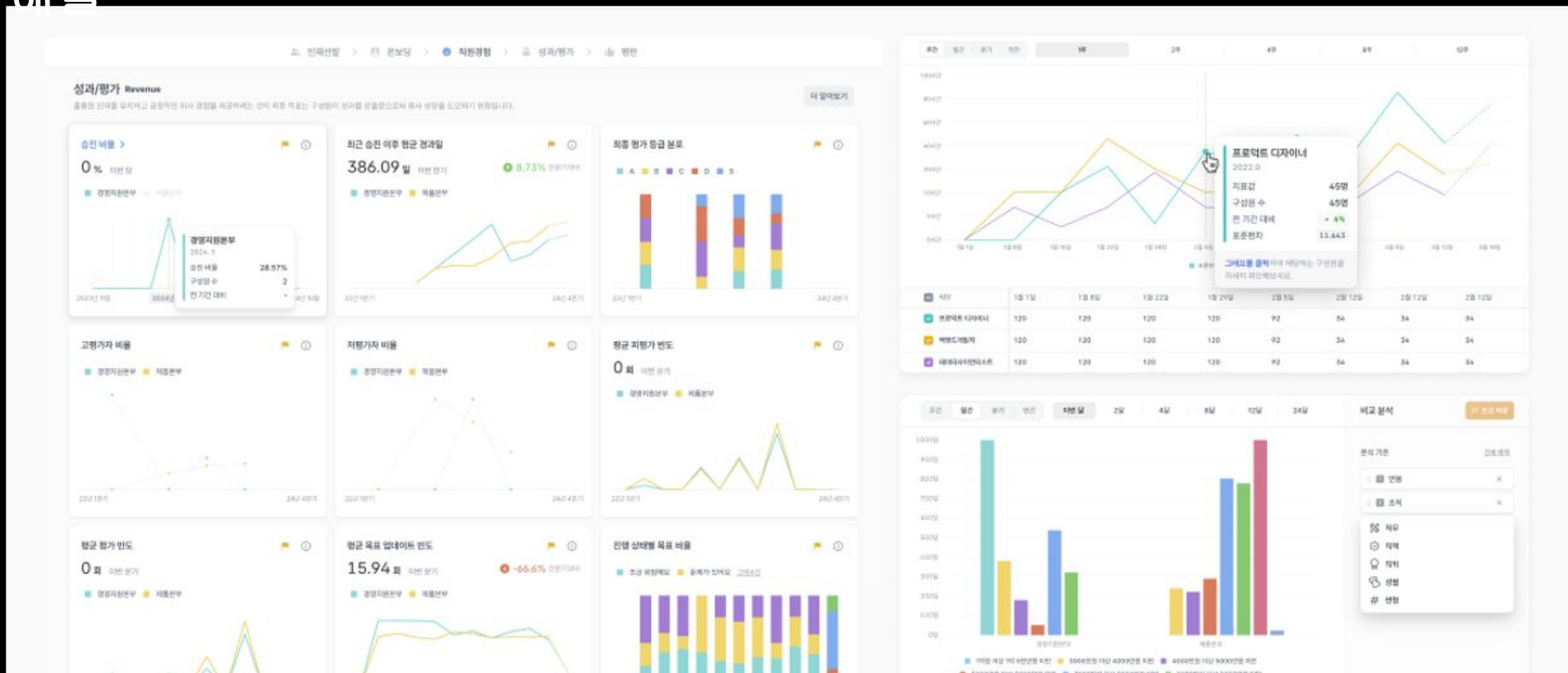
의사결정 및 실행하는 모든 것



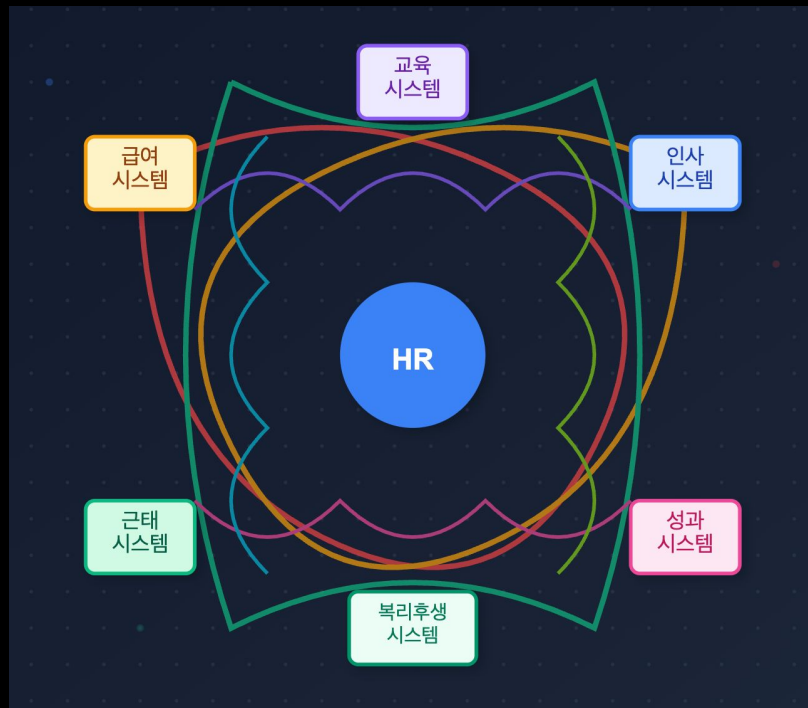
구성원 생애주기에 따른 모든 HR 을 통합 관리 HR Full Cycle Data



통합이 줄 수 있는 가치와 통찰에 집중하여 : 조직과 사람의 문제를 해결



생각보다 어려운 통합 문제



1. 도메인 통합 시나리오

2. 기능적 통합 시나리오

도메인 통합 시나리오

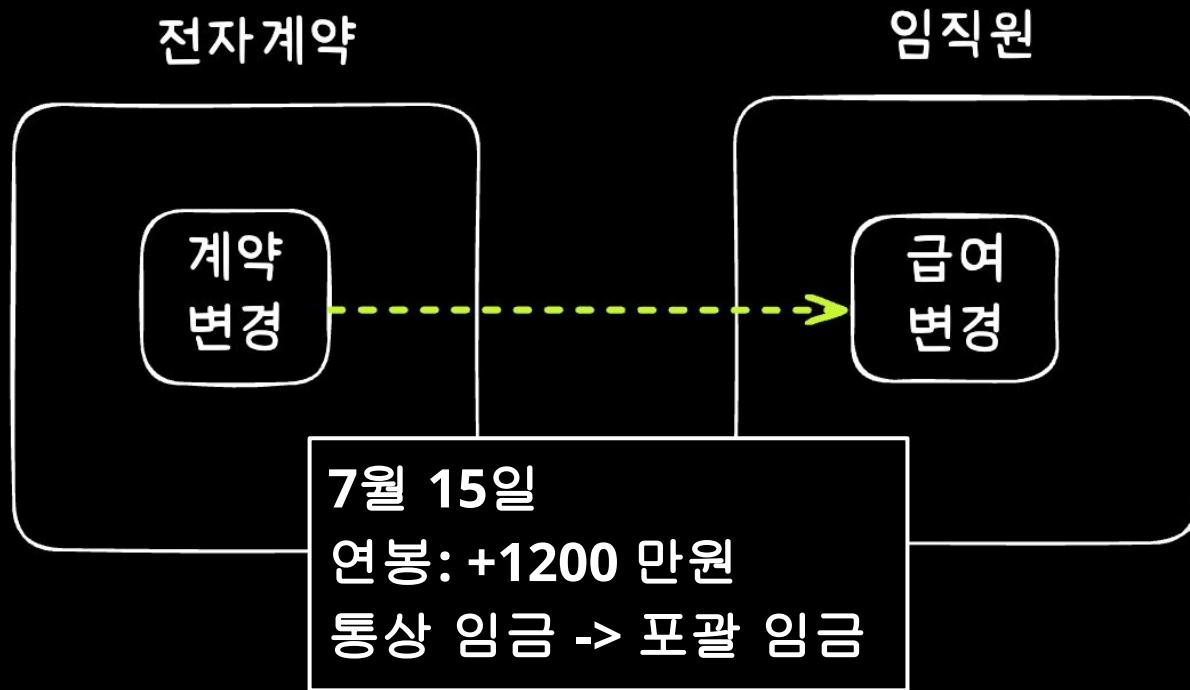
전자계약

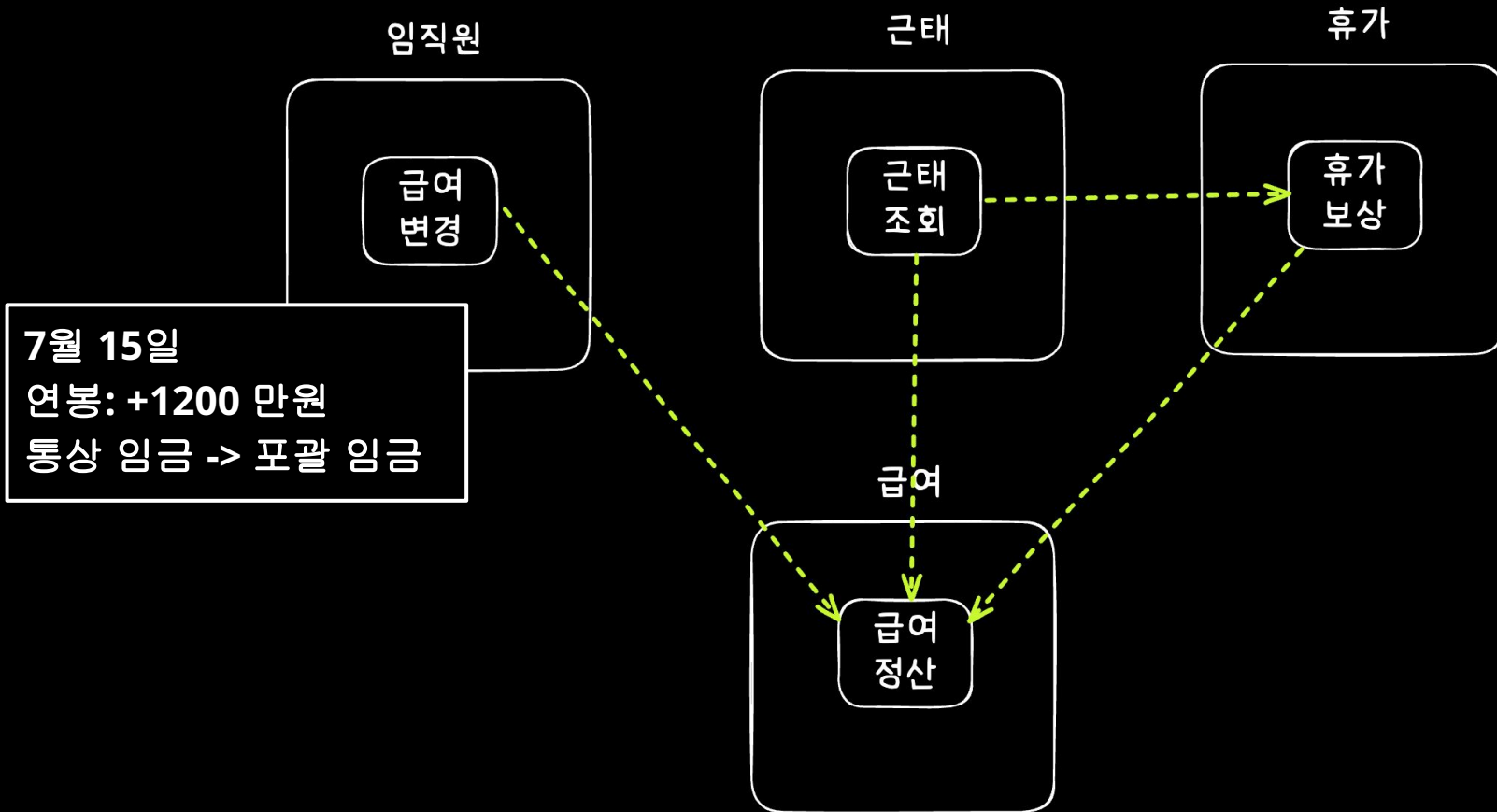
계약
변경

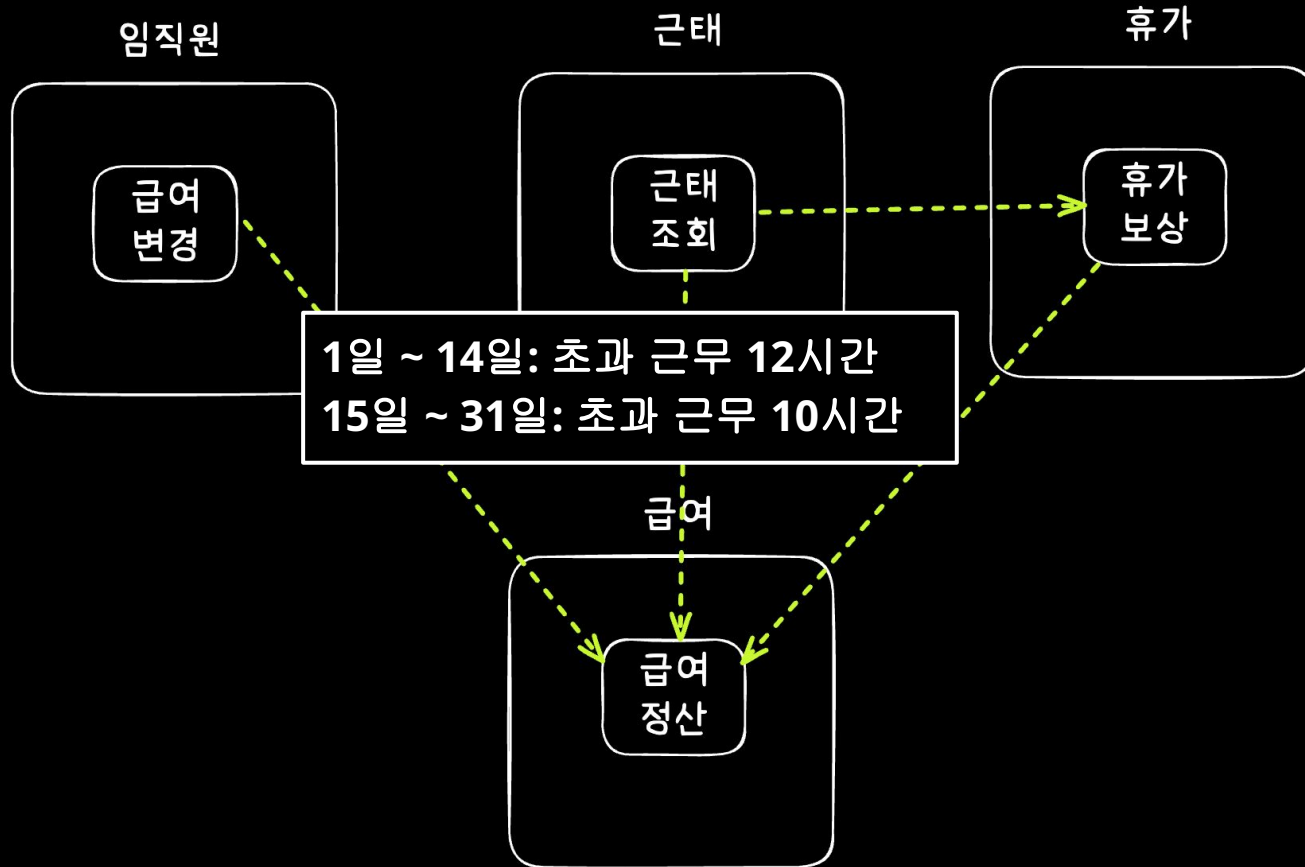
7월 15일

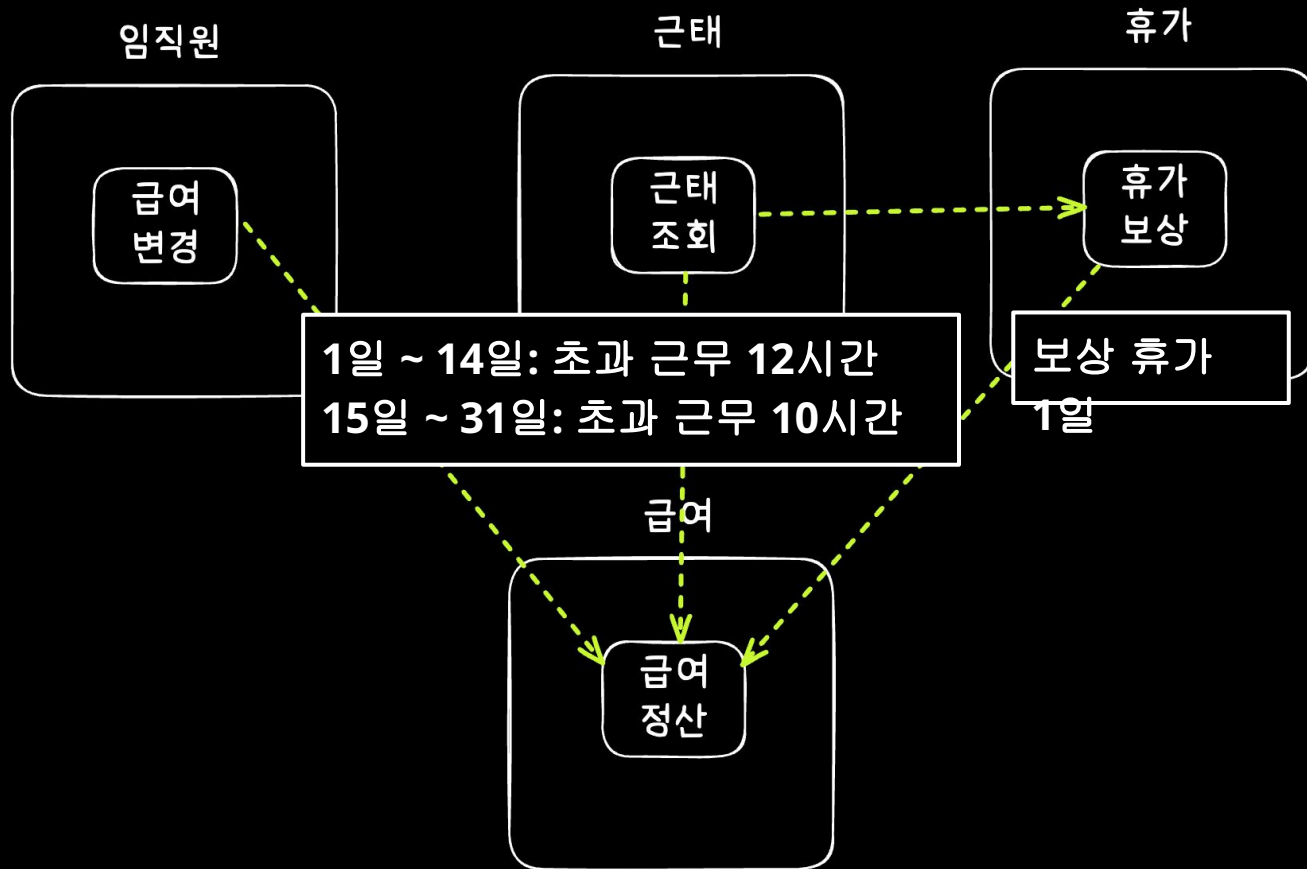
연봉: +1200 만원

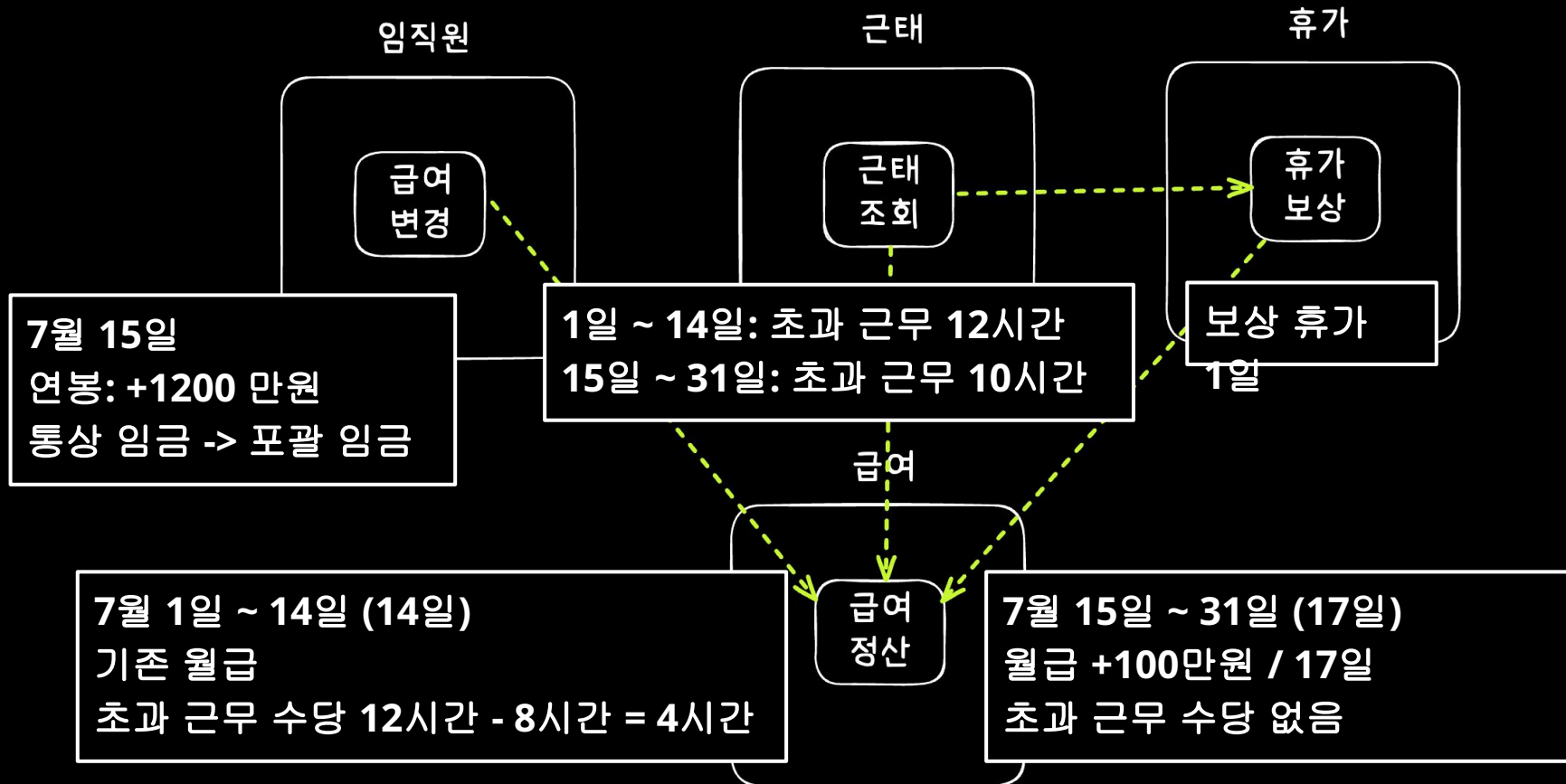
통상 임금 -> 포괄 임금











급여

급여
정산

급여
지급

7월 1일 ~ 14일 (14일)

기존 월급

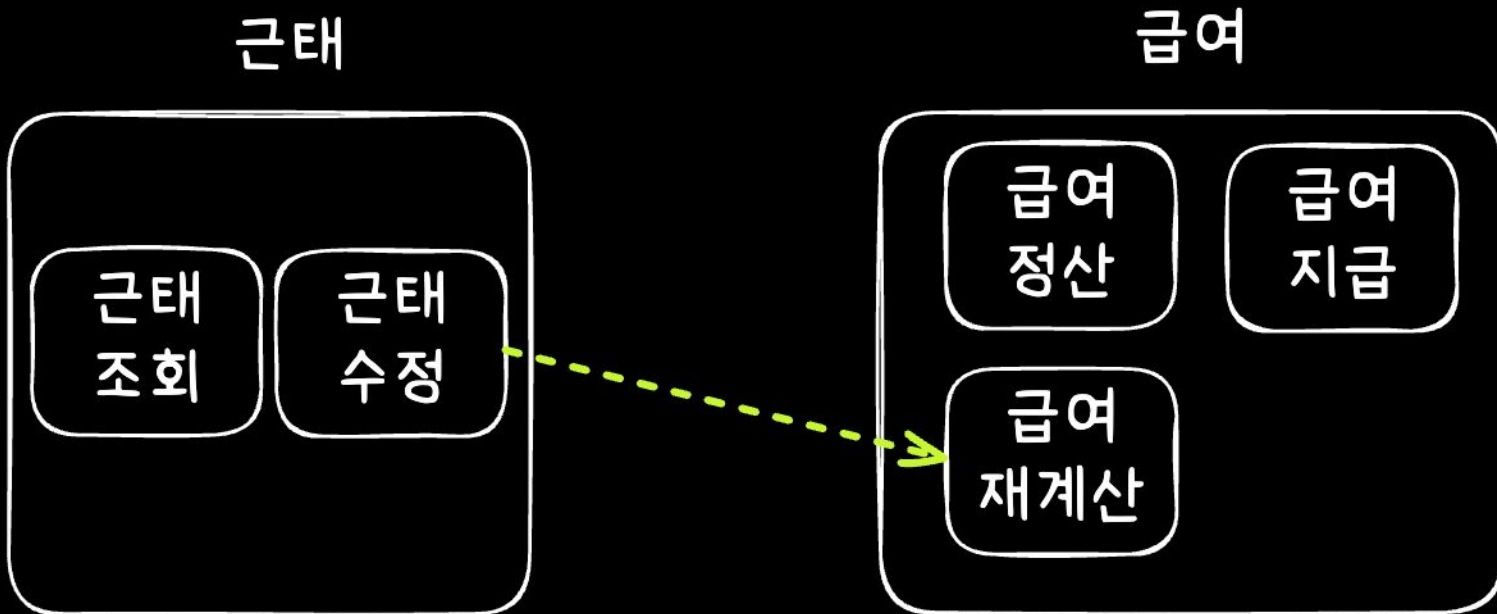
초과 근무 수당 12시간 - 8시간 = 4시간

+

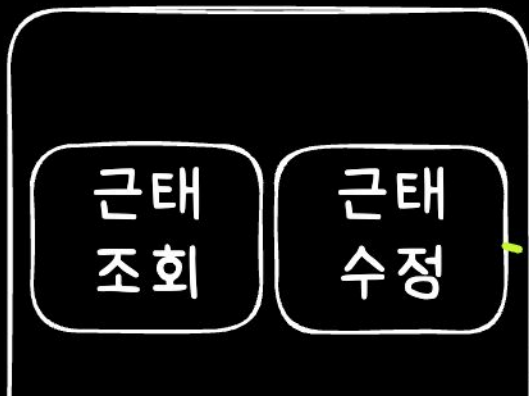
7월 15일 ~ 31일 (17일)

월급 +100만원 / 17일

초과 근무 수당 없음



근태

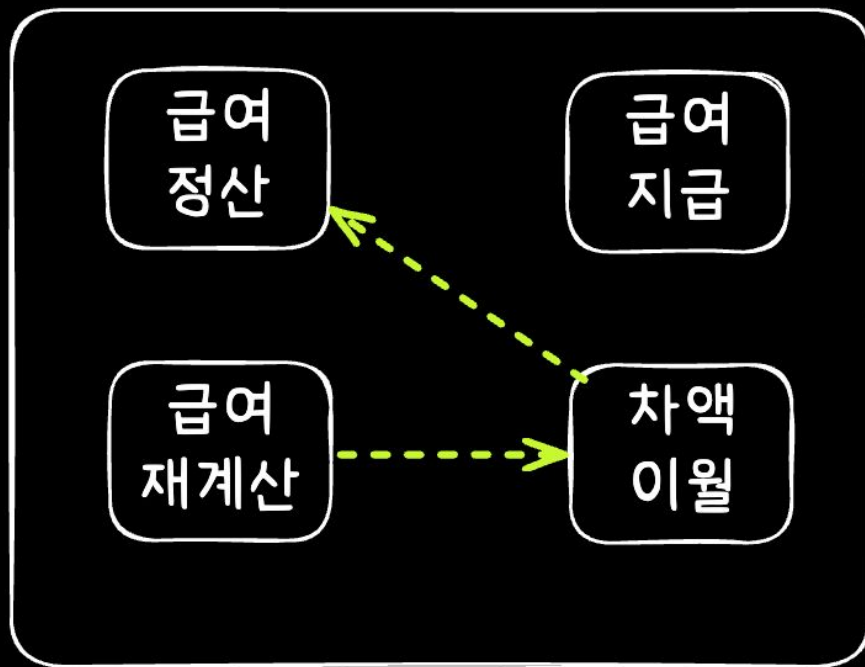


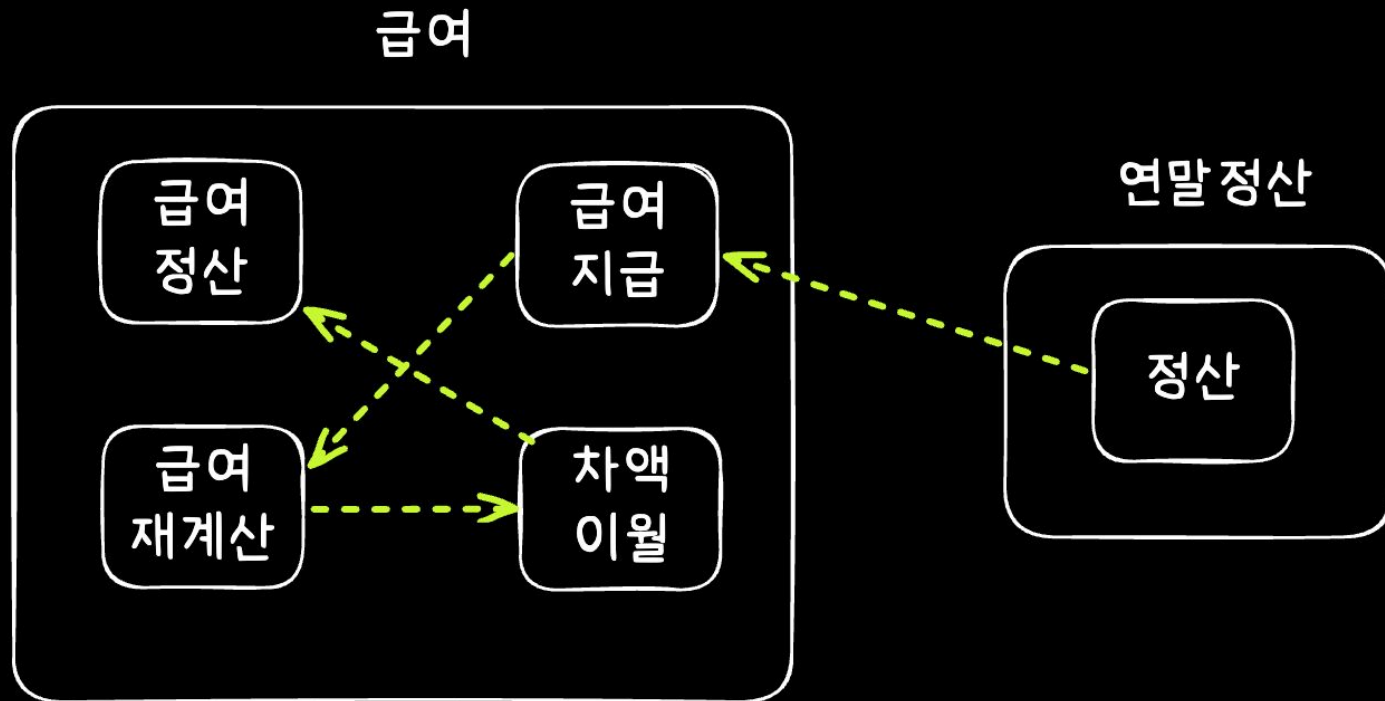
1일 ~ 14일: 초과 근무 **12시간** -> **10시간**
15일 ~ 31일: 초과 근무 10시간

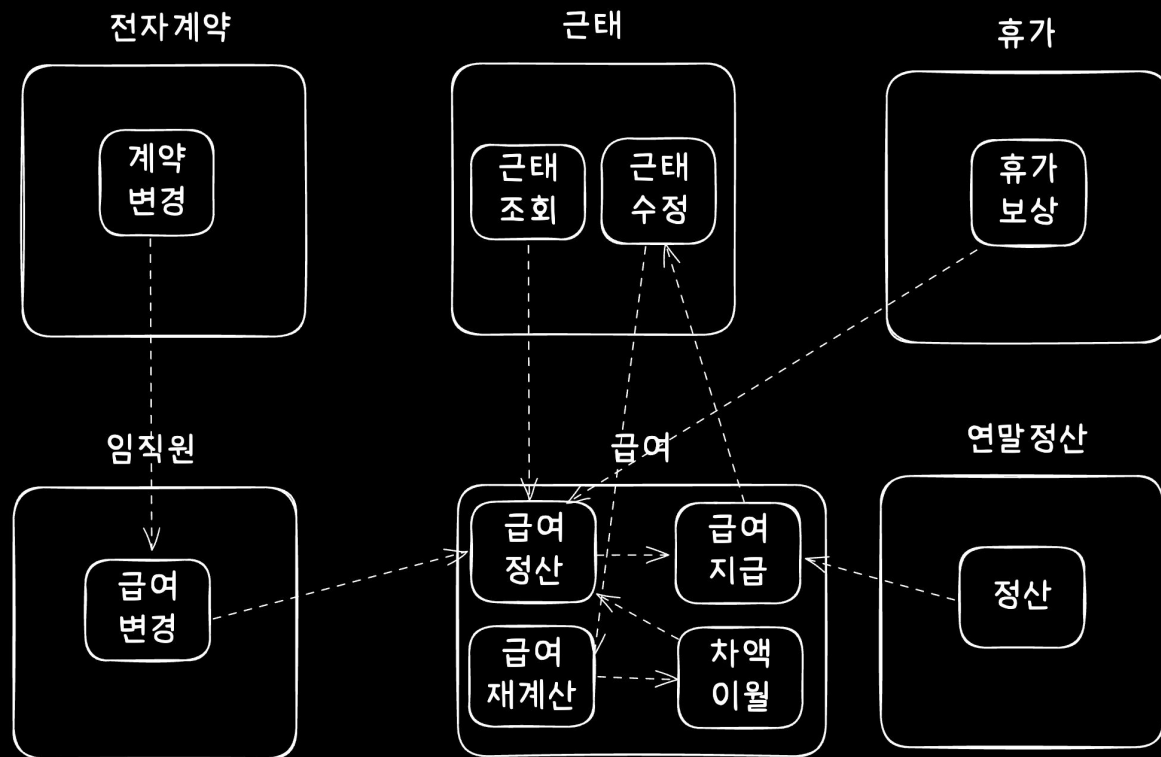
급여



급여







전자계약

계약

근태

근태

근태

휴가

휴가

도메인 논리의 높은 복잡도와 상호 의존성

↓
급여
변경

급여
정산

급여
재계산

급여
지급

차액
이월

정산

기능적 통합 시나리오

☆

📁

⋮

🖨️

⚙️

✕

진행중 1 / 2

지출결의서(세금계산서)

문서번호 2023-178 · 작성 2023년 10월 23일 (월) 오후 5:10

강민성님이 글을 작성했어요. 2023년 10월 23일

강민성 2023년 10월 23일
사유를 작성해주세요

강민성 2023년 10월 23일
여러개 사면 할인이 되나요?

강민성 2023년 12월 1일
확인 후 승인 부탁드립니다

댓글 입력

T 부서

영업팀

T 지출명

모니터

☞ 지출 일자

2023년 10월 17일

💰 입금 금액(세전)

1,000,000 원

○ 계약 유형

월간계약

📄 세금계산서 발행일

2023년 10월 23일

1. 작성 안내

세금계산서에 대한 지출 결의서를 작성하시고,
직인을 받은 계약서, 통장 사본, 신분증 사본을 파일로 첨부해주세요.

승인 · 참조

1 단계 진행 중

진도준 1차 조직원
사업본부 대기중

경영지원팀
참조

2 단계 대기중

김진희 2차 조직원
가능조직 대기중

전자결재

^

v

☆ ㉪ ...   ✕

진행중 1 / 2

지출결의서(세금계산서)

문서번호 2023-178 · 작성 2023년 10월 23일 (월) 오후 5:10

 강민성

강민성님이 글을 작성했어요. 2023년 10월 23일

 강민성

2023년 10월 23일

사유를 작성해주세요

 강민성

2023년 10월 23일

여러개 사면 할인이 되나요?

 강민성

2023년 12월 1일

확인 후 승인 부탁드립니다

댓글 입력

Tt 부서	영업팀
Tt 지출명	모니터
㊦ 지출 일자	2023년 10월 17일
㊦ 입금 금액(세전)	1,000,000 원
㊦ 계약 유형	월간계약
㊦ 세금계산서 발행일	2023년 10월 23일

1. 작성 안내

세금계산서에 대한 지출 결의서를 작성하시고,
직인을 받은 계약서, 통장 사본, 신분증 사본을 파일로 첨부해주세요.

승인 · 참조

1 단계 진행 중

 진도준

1차 조직장

대기중

 경영지원팀

참조

2 단계 대기중

 김진희

2차 조직장

대기중

 기능조직

대기중

댓글, 로그, 첨부파일

지출결의서(세금계산서)

문서번호 2023-178 · 작성 2023년 10월 23일 (월) 오후 5:10

강민성님이 글을 작성했어요. 2023년 10월 23일

강민성 2023년 10월 23일
사유를 작성해주세요

강민성 2023년 10월 23일
여러개 사면 할인이 되나요?

강민성 2023년 12월 1일
확인 후 승인 부탁드립니다

댓글 입력

T 부서	영업팀
T 지출명	모니터
日 지출 일자	2023년 10월 17일
₩ 입금 금액(세전)	1,000,000 원
○ 계약 유형	월간계약
日 세금계산서 발행일	2023년 10월 23일

1. 작성 안내

세금계산서에 대한 지출 결의서를 작성하시고,
직인을 받은 계약서, 통장 사본, 신분증 사본을 파일로 첨부해주세요.

승인

^

v

☆ | 📁 | ... | 🗨️ | ⌵ | ✕

진행중 1 / 2

지출결의서(세금계산서)

문서번호 2023-178 · 작성 2023년 10월 23일 (월) 오후 5:10

 강민성님이 글을 작성했어요. 2023년 10월 23일

 강민성 2023년 10월 23일

사유를 작성해주세요

 강민성 2023년 10월 23일

여러개 사면 할인이 되나요?

 강민성 2023년 12월 1일

확인 후 승인 부탁드립니다

댓글 입력

Tf 부서

영업팀

Tf 지출명

모니터

📅 지출 일자

2023년 10월 17일

💰 입금 금액(세전)

1,000,000 원

🕒 계약 유형

월간계약

📅 세금계산서 발행일

2023년 10월 23일

1. 작성 안내

세금계산서에 대한 지출 결의서를 작성하시고,
직인을 받은 계약서, 통장 사본, 신분증 사본을 파일로 첨부해주세요.

승인 · 참조

1 단계 진행 중

 진도준 1차 조직장

사업본부

대기중

 경영지원팀

추진

참조

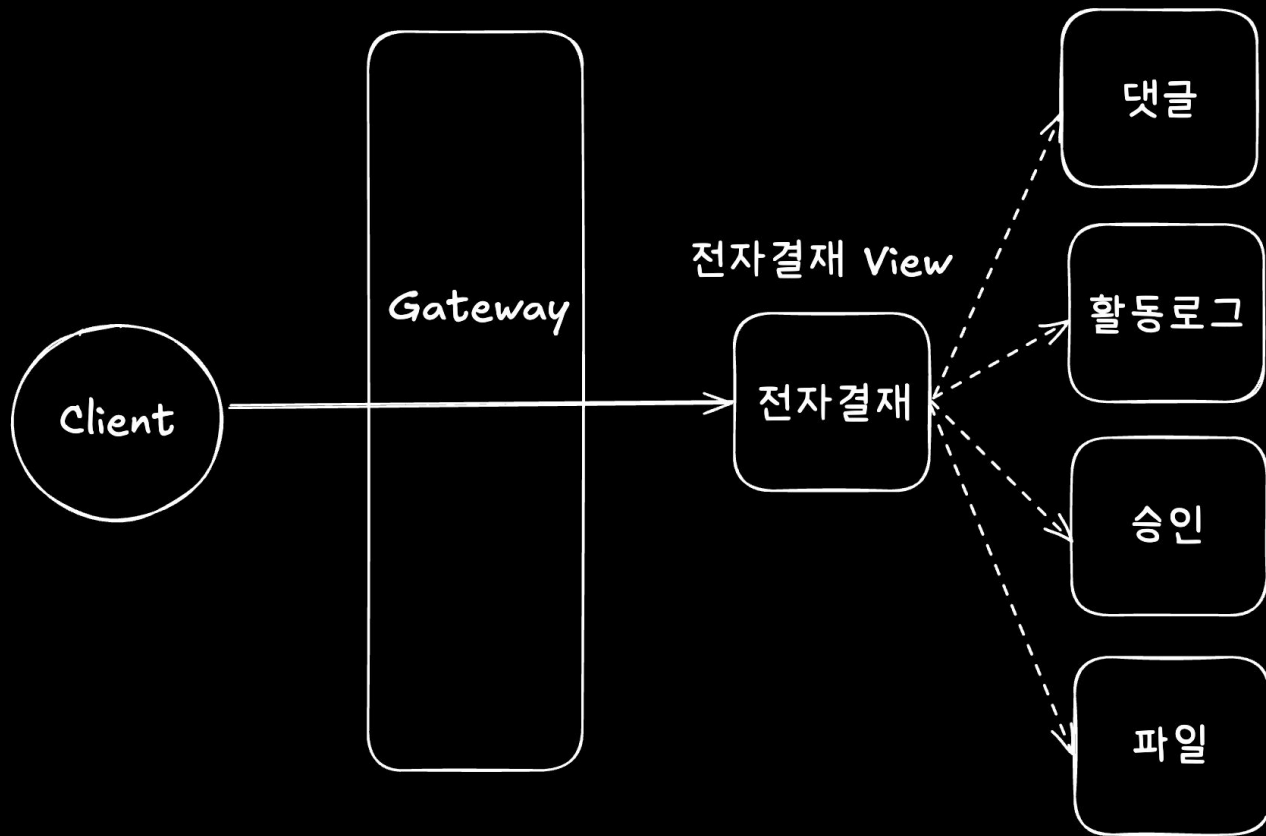
2 단계 대기중

 김진희 2차 조직장

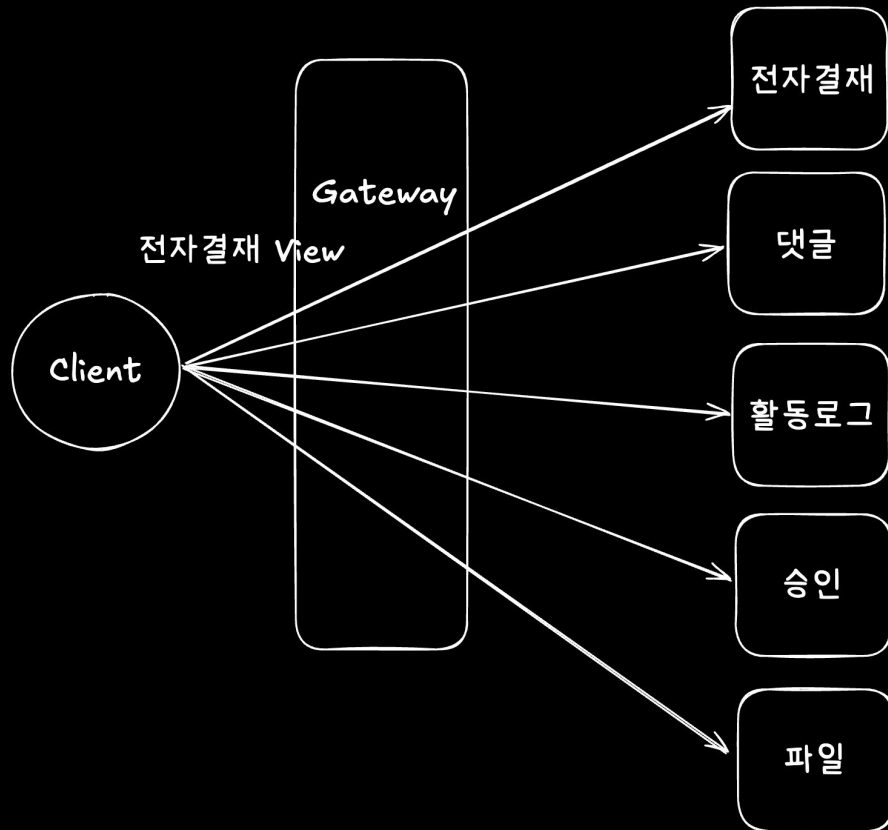
기능조직

대기중

👤

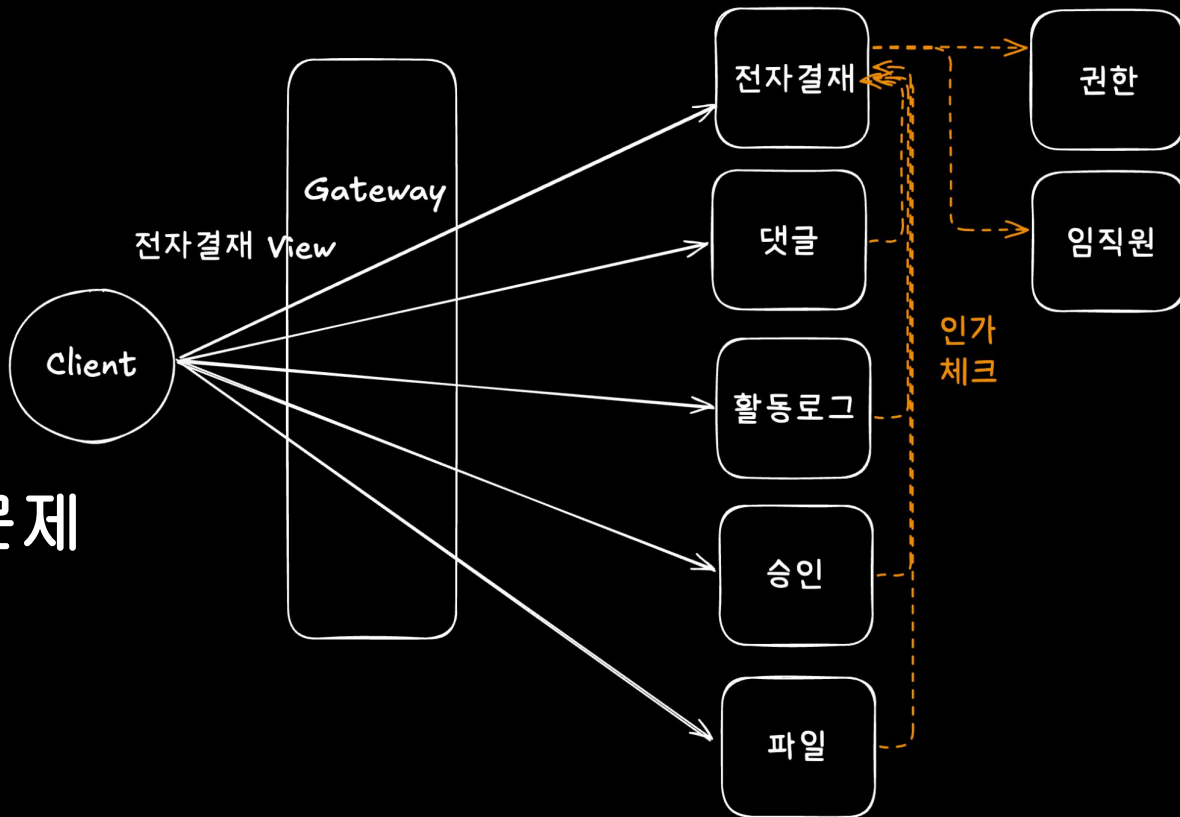


각 기능을 직접 호출하여
조립하면 안될까?



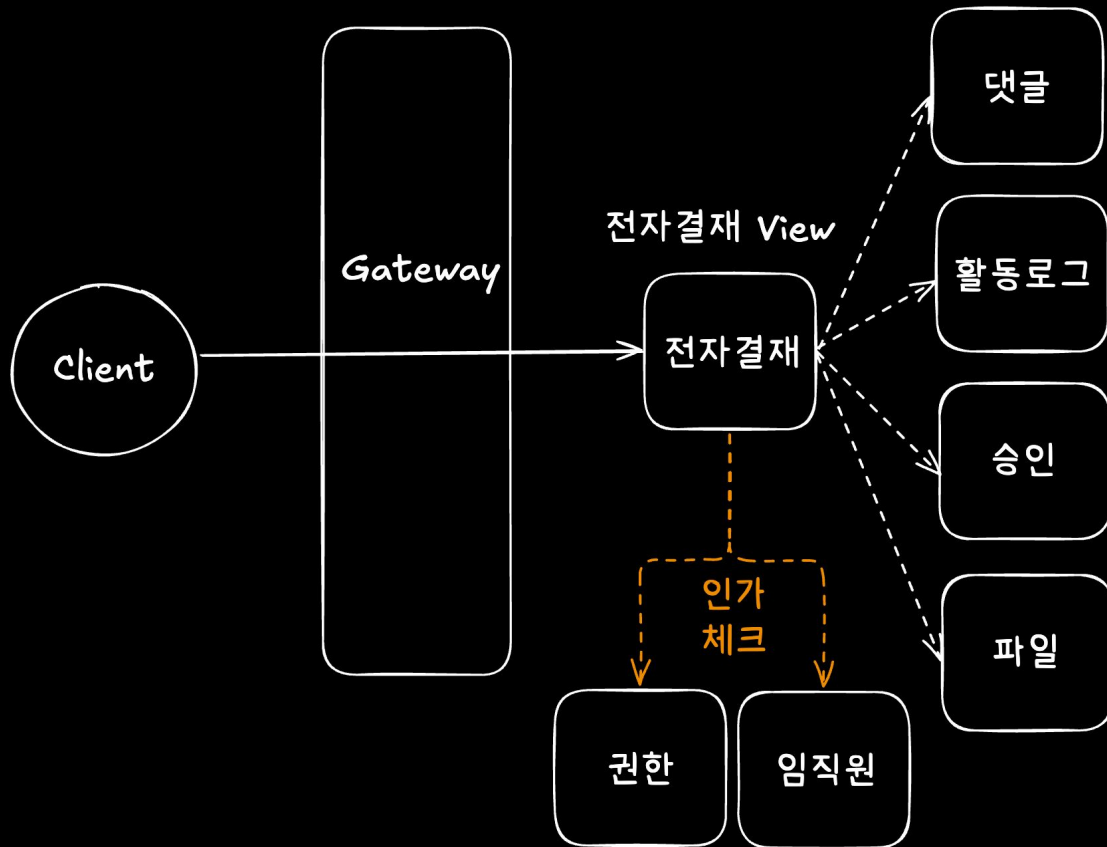
권한 / 인가 체크

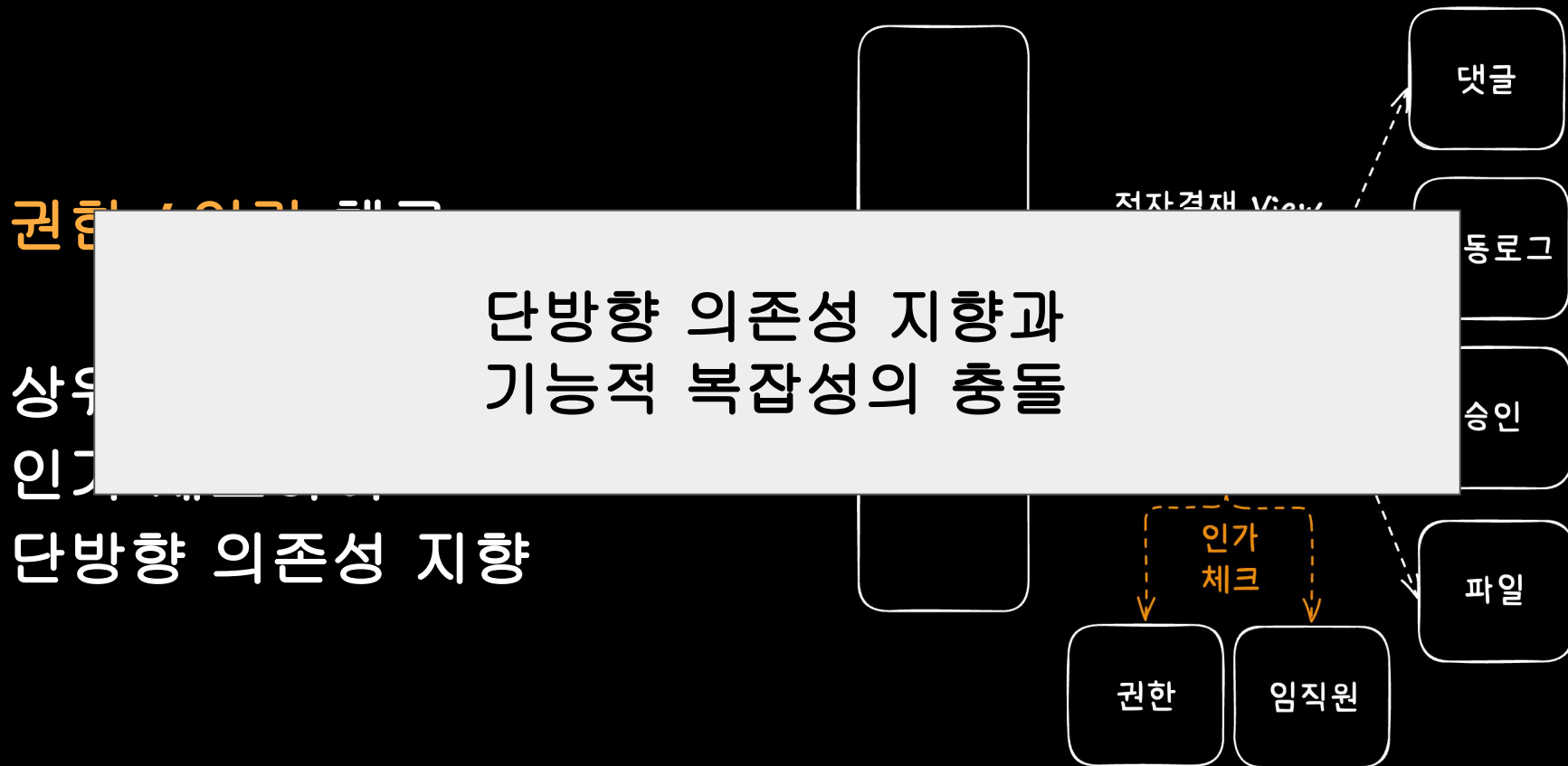
하위 리소스에서
상위 리소스 참조 문제



권한 / 인가 체크

상위 리소스에서
인가 체크하여
단방향 의존성 지향





1. 도메인 통합 시나리오

2. 기능적 통합 시나리오

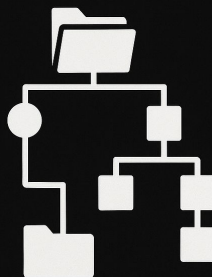
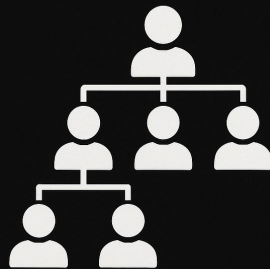
권한, 인가 판단의 복잡성과 해결 이야기

권한과 인가 판단

B2B SaaS 의 엔터프라이즈 통합 요구에서 어느순간 극악의 난이도로 만나게 되는 문제

이 문제를 풀지 않고, 통합을 위한 Scale 은 불가능하고 다음 단계로 나아갈 수 없다

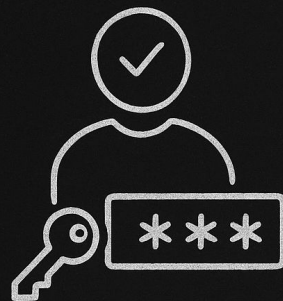
PERMISSION



AUTHORIZATION

2. HR 에서 인가 판단은 왜 어려울까?

AUTHENTICATION



PERMISSION



AUTHORIZATION



인증 (Authentication)

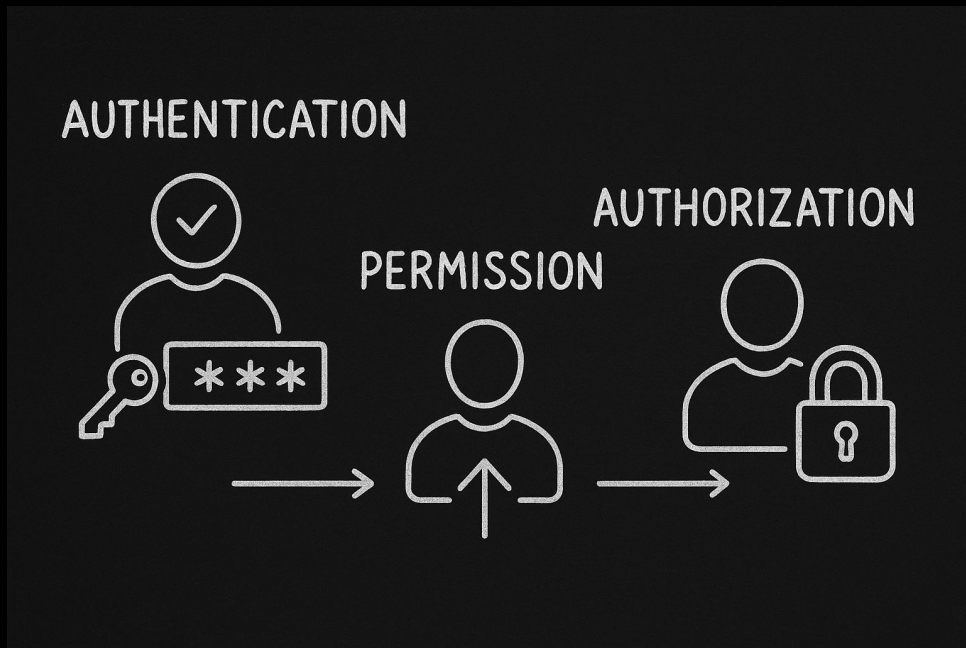
: 사용자 식별

권한 (Permission)

: 사용자 또는 그룹에 부여된
역할(Role)로 할 수 있는 액션

인가 (Authorization)

: 사용자가 리소스에 액션 **허가**를 판단



인증 (Authentication)

: 사용자 식별

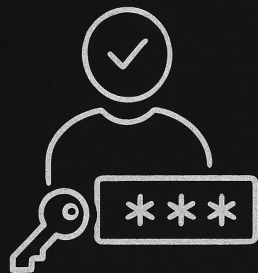
권한 (Permission)

: 사용자 또는 그룹에 부여된
역할(Role)로 할 수 있는 액션

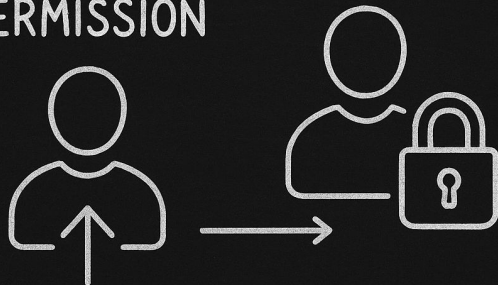
인가 (Authorization)

: 사용자가 리소스에 액션 **허가**를 판단

AUTHENTICATION



PERMISSION



AUTHORIZATION



인증 (Authentication)

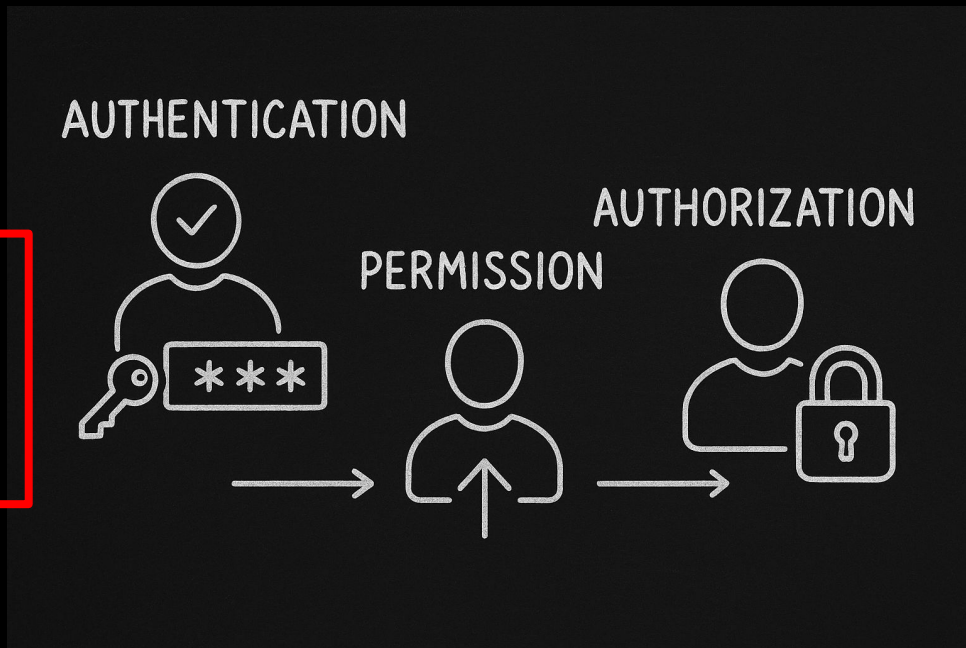
: 사용자 식별

권한 (Permission)

: 사용자 또는 그룹에 부여된
역할(Role)로 할 수 있는 액션

인가 (Authorization)

: 사용자가 리소스에 액션 **허가**를 판단



인증 (Authentication)

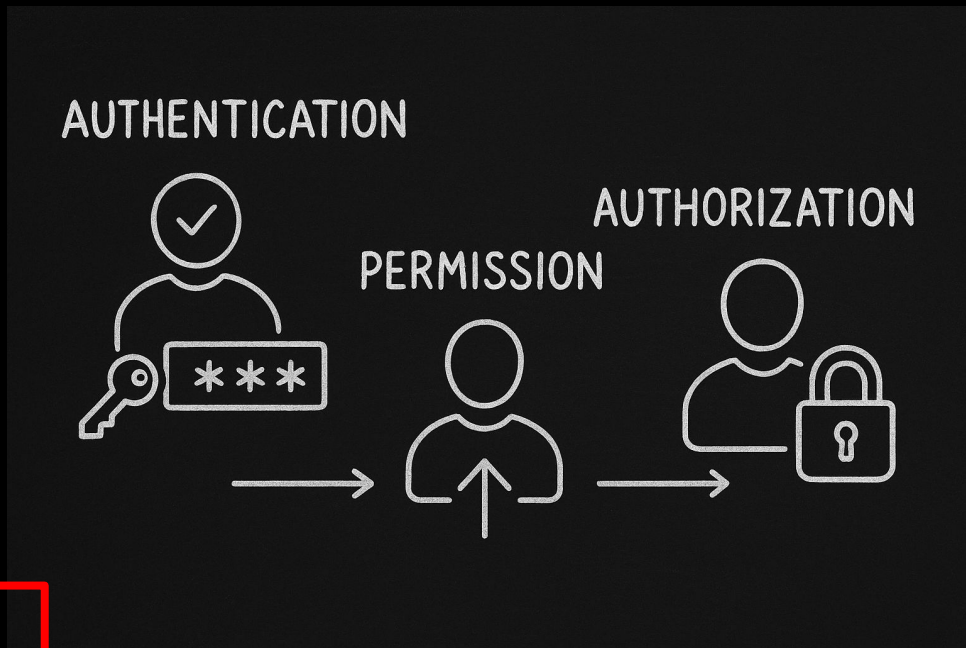
: 사용자 식별

권한 (Permission)

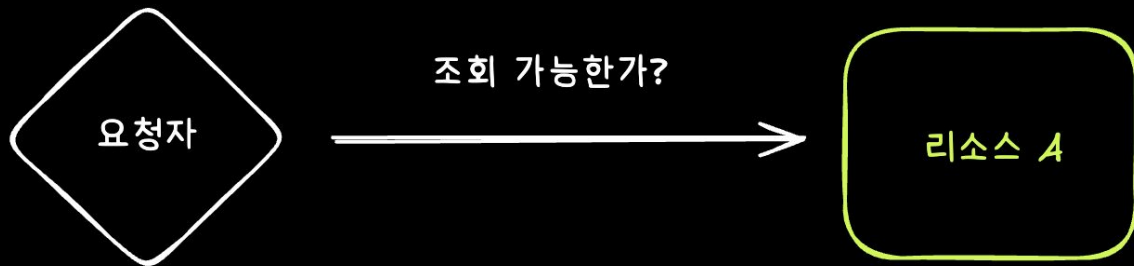
: 사용자 또는 그룹에 부여된
역할(Role)로 할 수 있는 액션

인가 (Authorization)

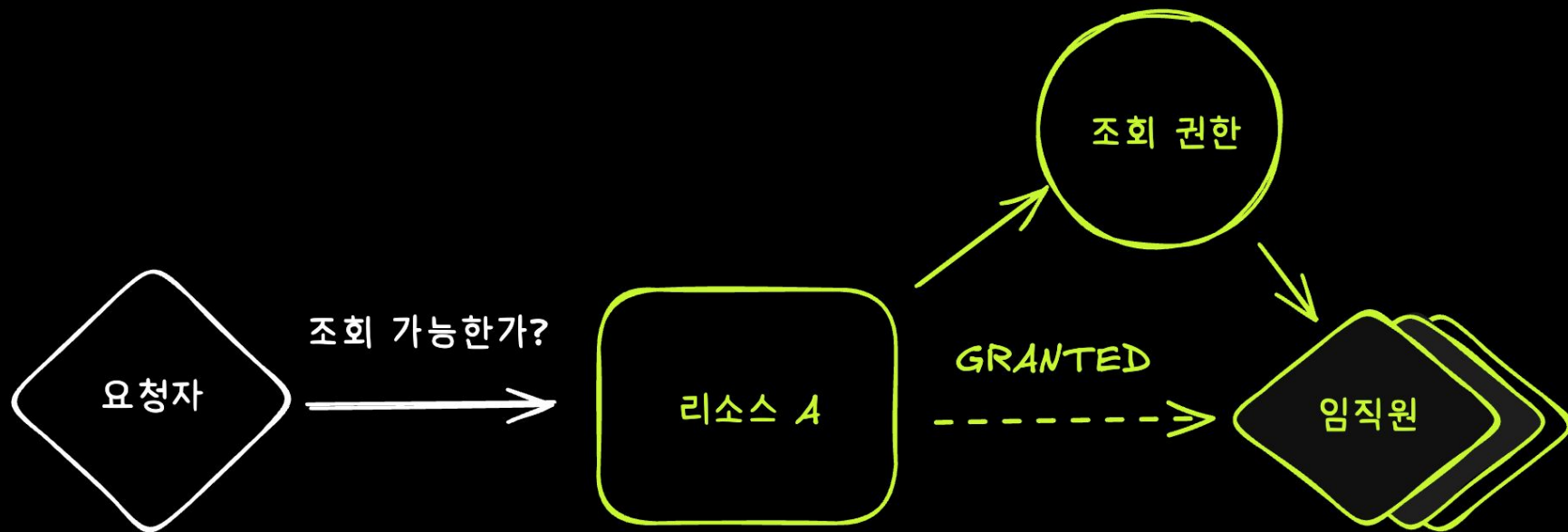
: 사용자가 리소스에 액션 **허가**를 판단



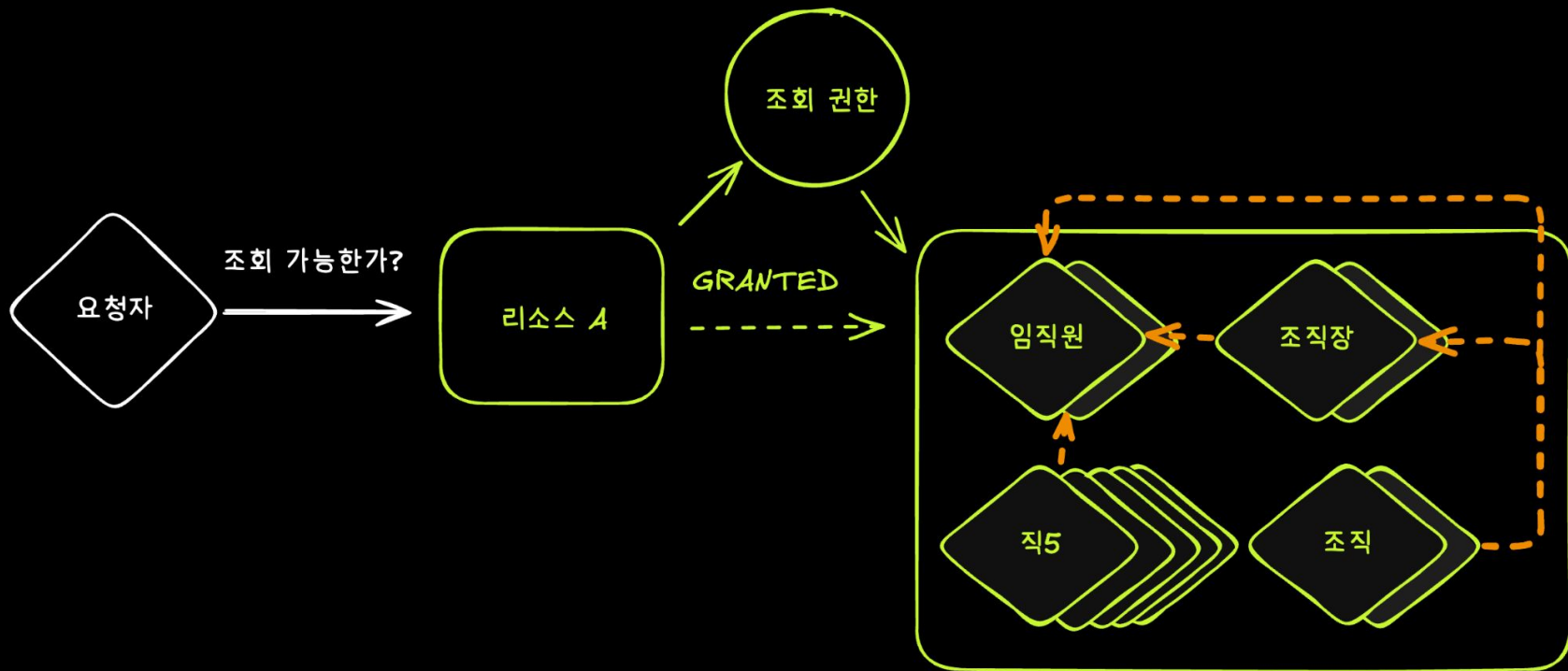
요청자가 전자결재 문서 A 를 조회할 수 있는가?



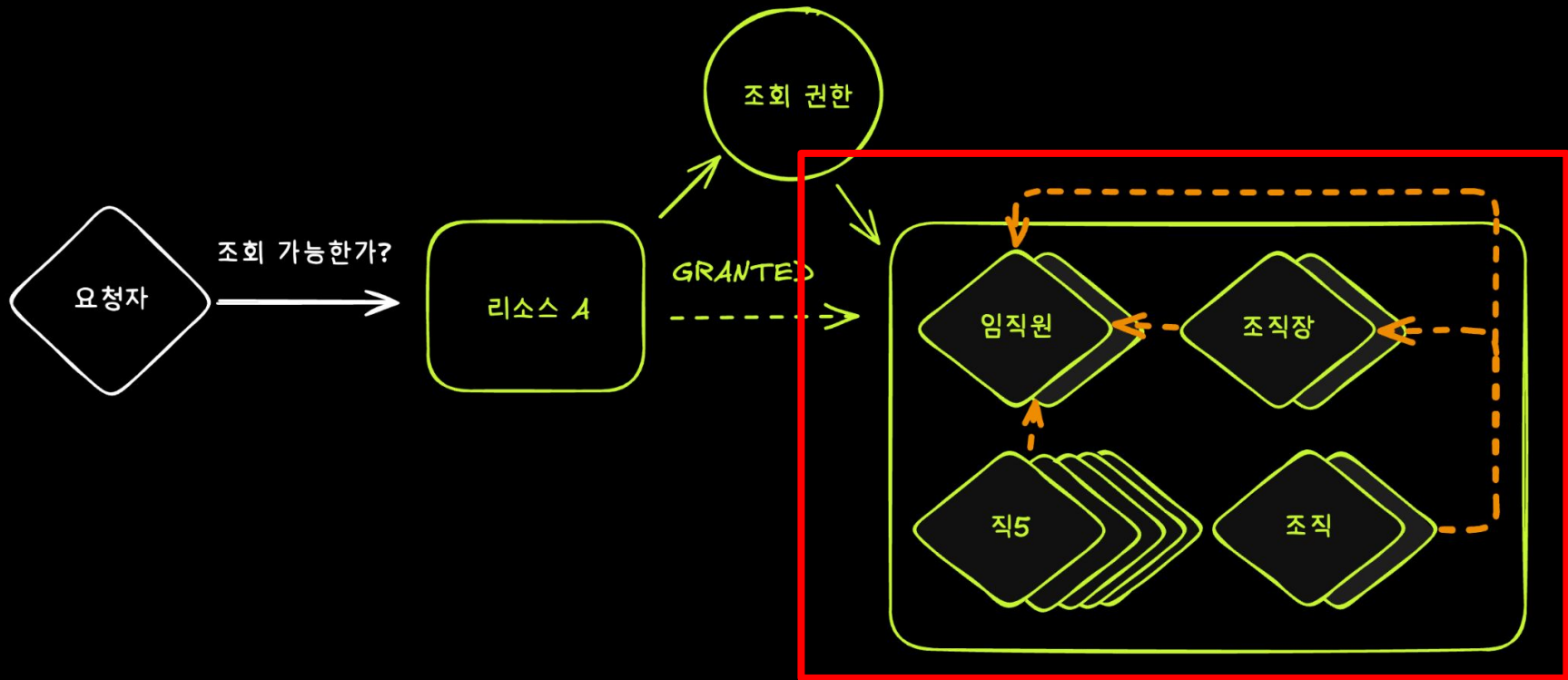
요청자는 권한을 부여 받은
임직원인가?



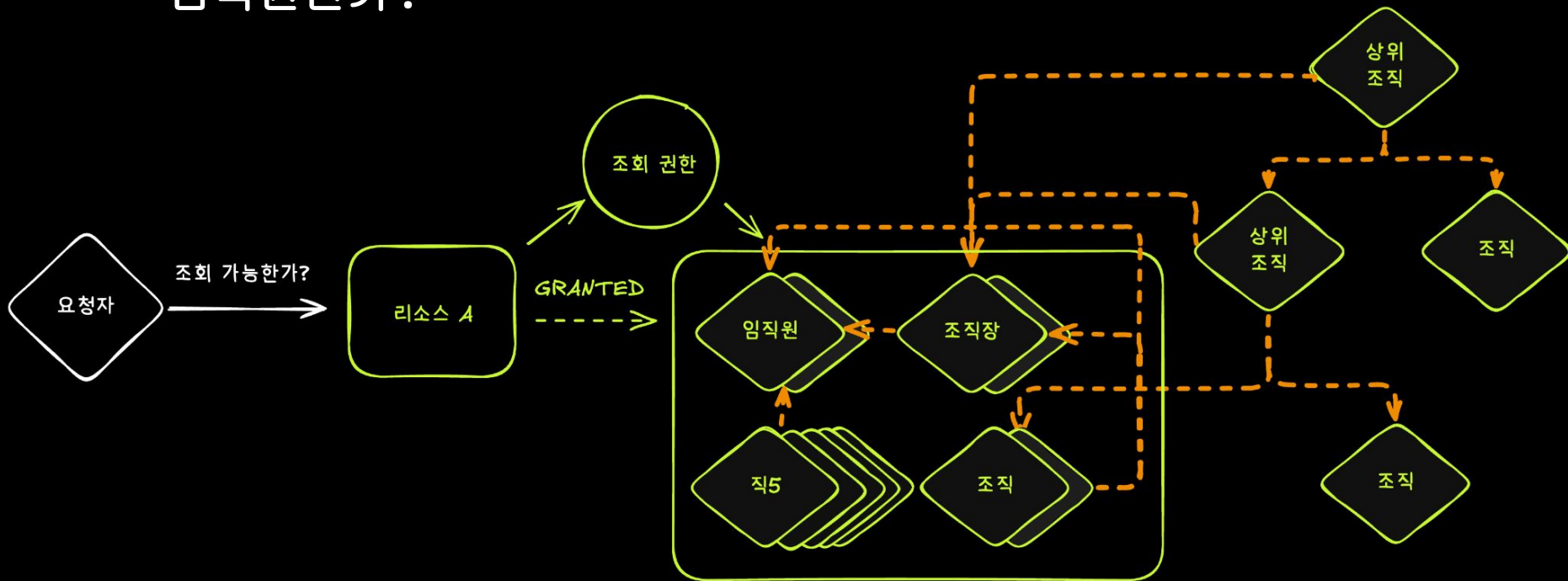
요청자는 권한을 부여 받은 대상과 연결된
이지의이기??



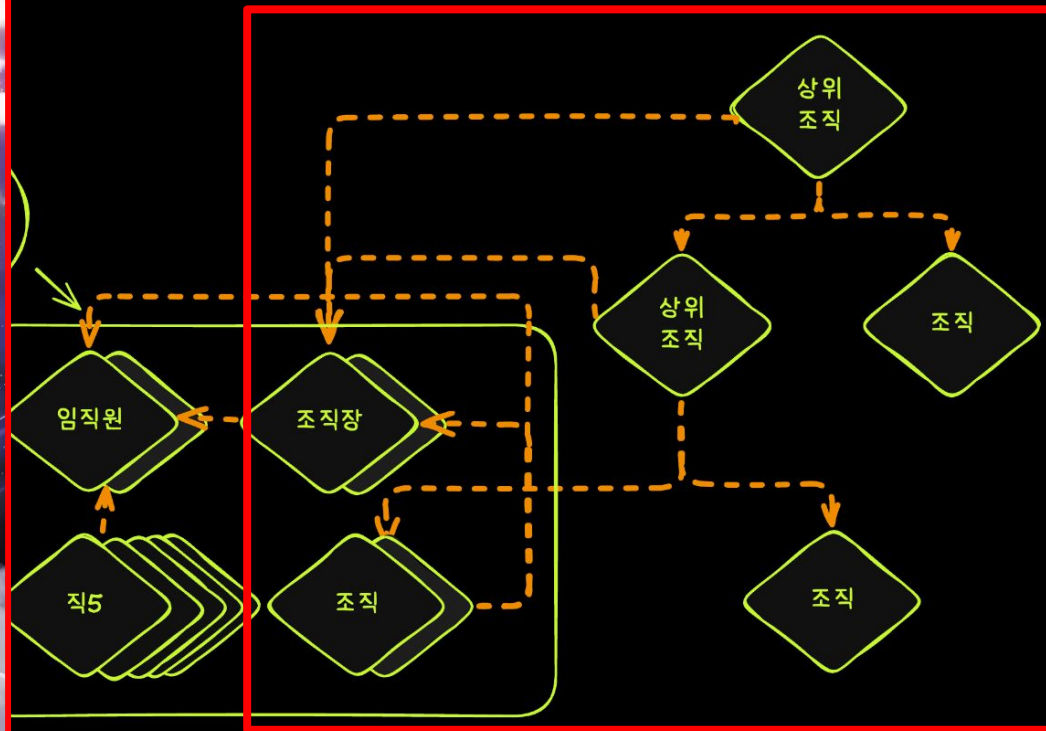
요청자는 권한을 부여 받은 대상과 연결된 이지의이기??



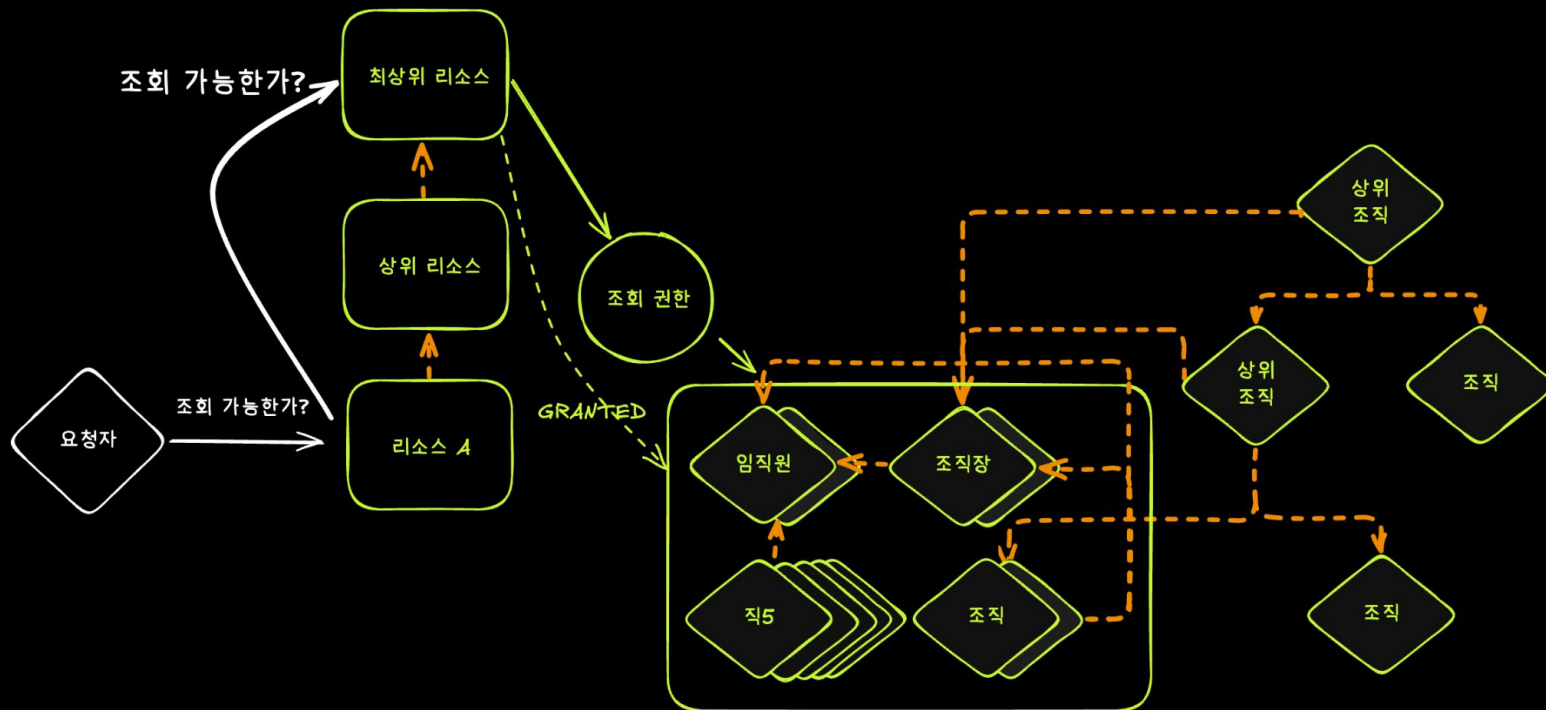
요청자는 권한을 부여 받은 조직의 상위 조직장으로 연결된
임직원인가?



조직의 상위 조직장으로 연결된

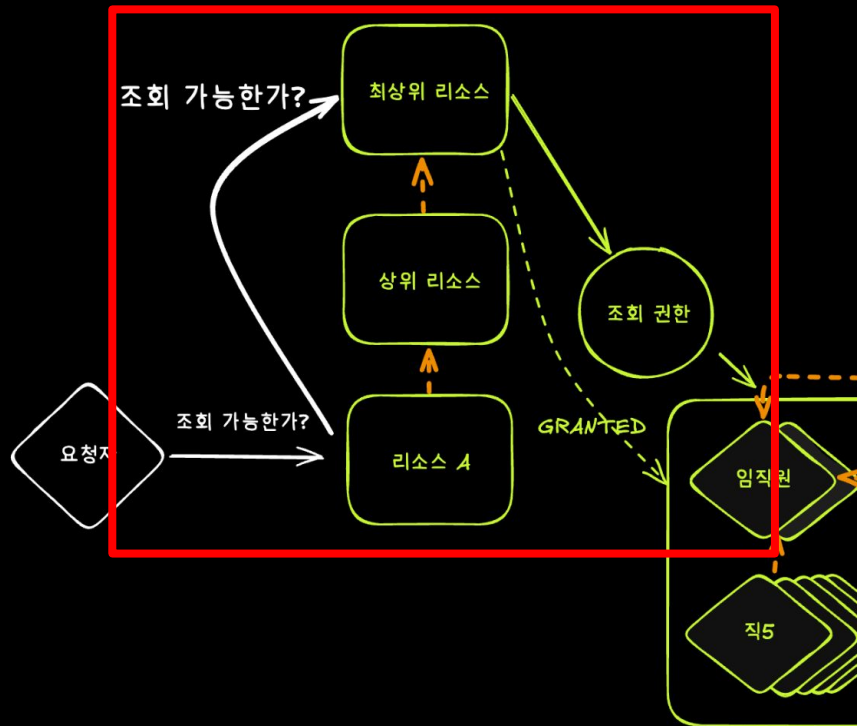


요청자는 최상위 리소스에 접근 가능한가?

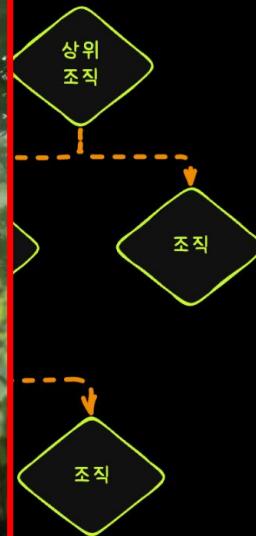


2. HR 에서 인가 판단은 왜 어려울까?

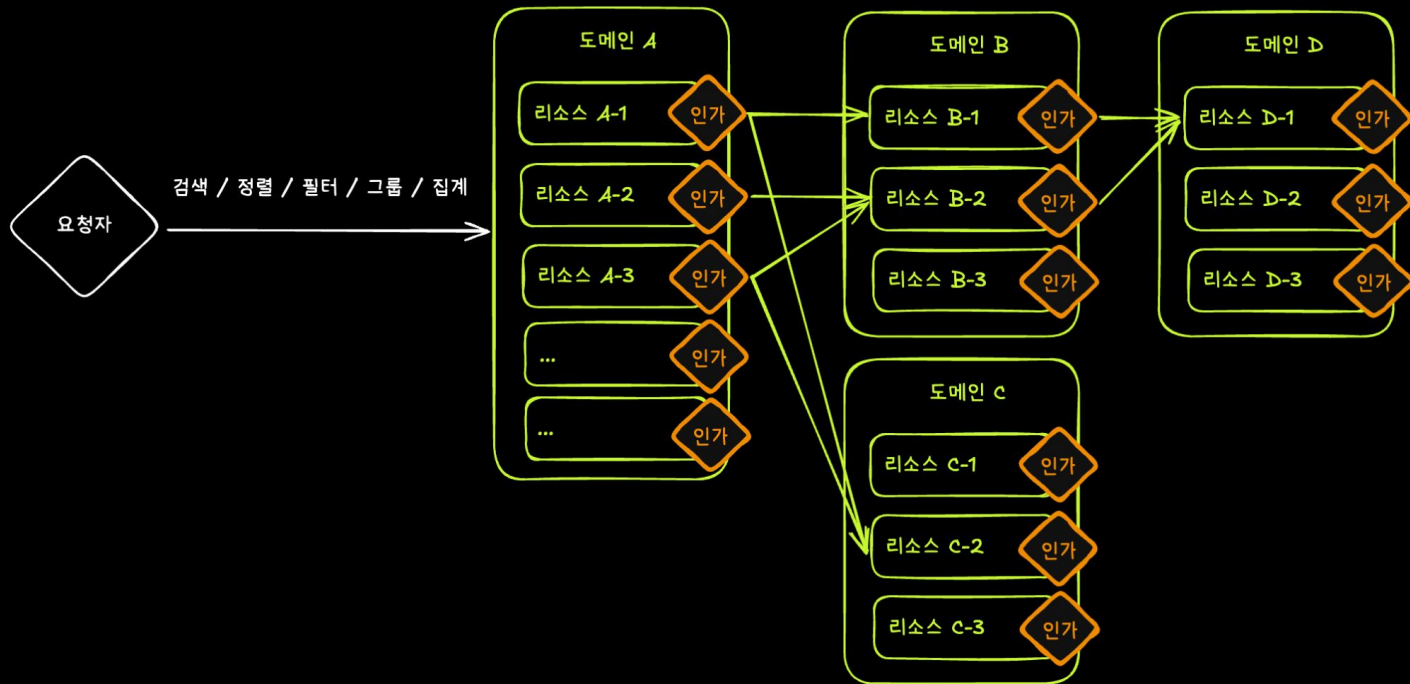
요청자는 최상위 리소스



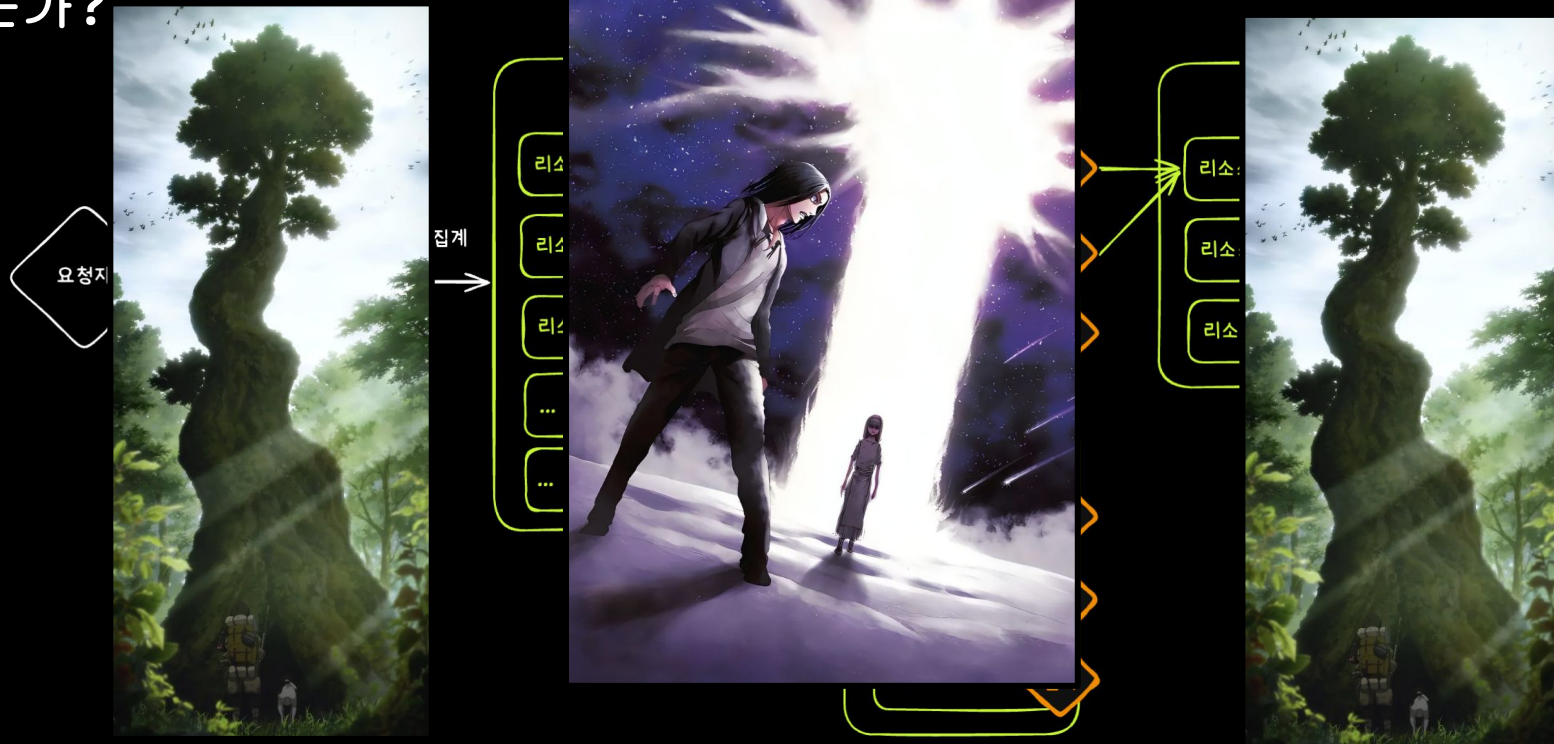
?



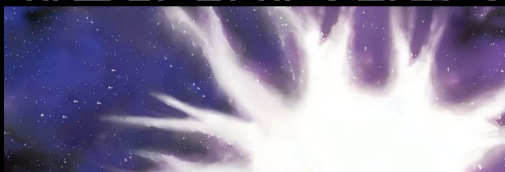
N 개의 접근 가능한 데이터 세트의 검색 / 정렬 / 필터 / 그룹 / 집계 할 수 있는가?



N 개의 접근 가능한 데이터 필터 / 그룹 / 집계 할 수 있는가?



N 개의 접근 가능한 데이터 / 필터 / 그룹 / 집계 할 수 있는가?



HR 에서 인가 판단은 왜 어려울까?

HR 은 회사 / 조직 / 임직원 / 담당자 등
다양하고 복잡한 **관계(Relation)**를
기반으로한 정보의 접근 제어를 관리한다

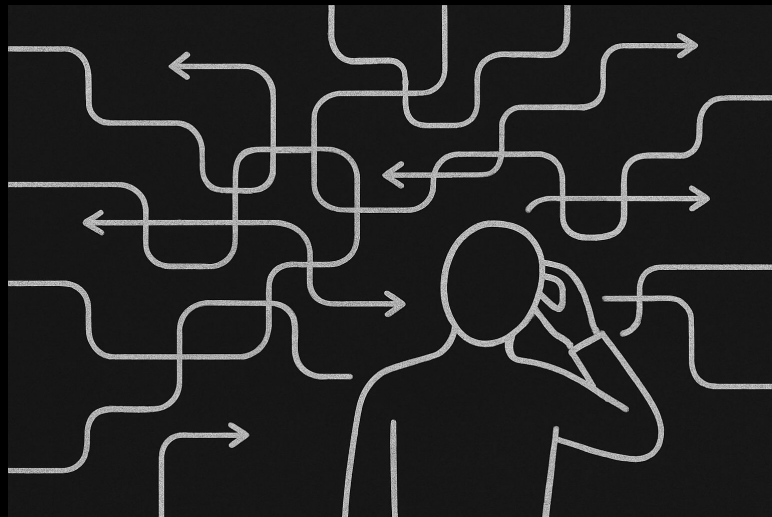


HR 에서 인가 판단은 왜 어려울까?

다양한 관계와 다중 역할

연결된 정보들의 복잡도 증가

접근 제어를 판단하는 복잡성과 경우의 수
급증



3. ReBAC : 통합 인가 플랫폼

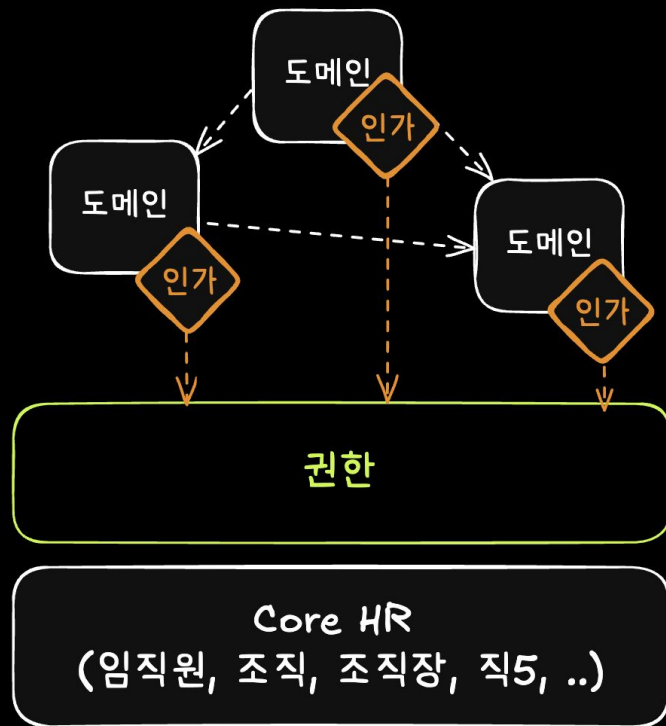
점점 높아지는 통합의 가치 실현 비용

MSA 환경에서 도메인별 복잡한 인가 판단

도메인간 연결된 정보의 인가 판단 비용

통합이 고도화 될수록

복잡도와 비용의 폭발적 증가



RBAC

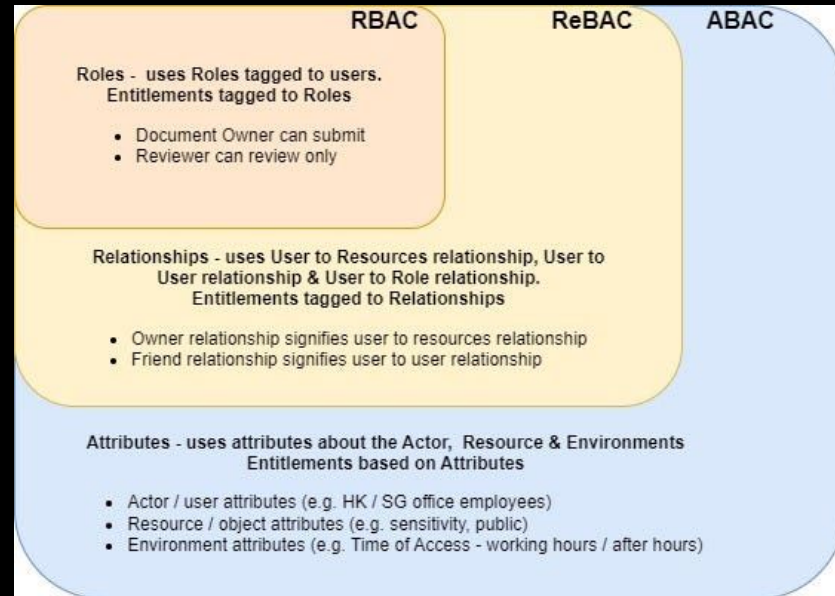
: Role Based Access Control

ReBAC

: Relation Based Access Control

ABAC

: Attribute Based Access Control



RBAC

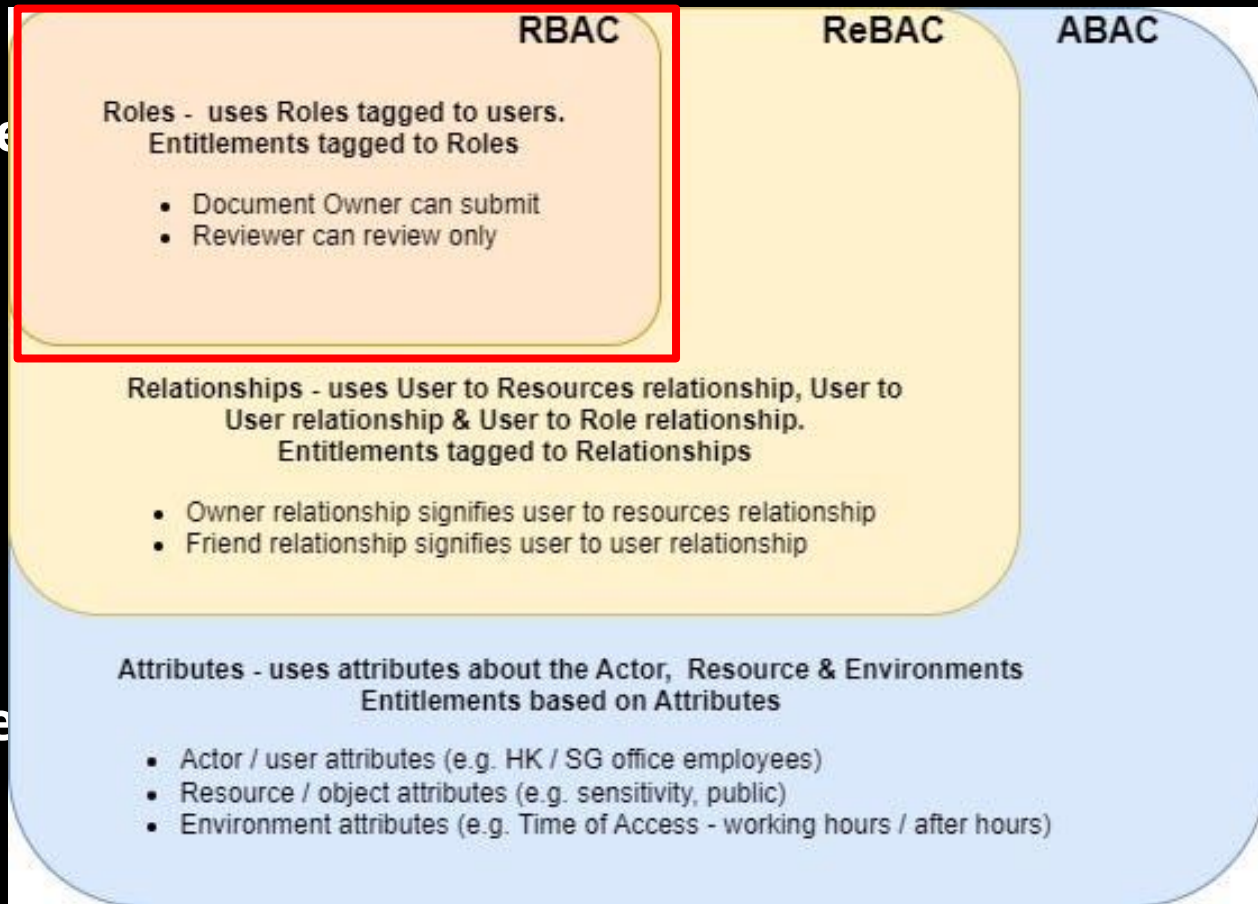
: Role Based

ReBAC

: Relation

ABAC

: Attribute



RBAC

RBAC

ReBAC

ABAC

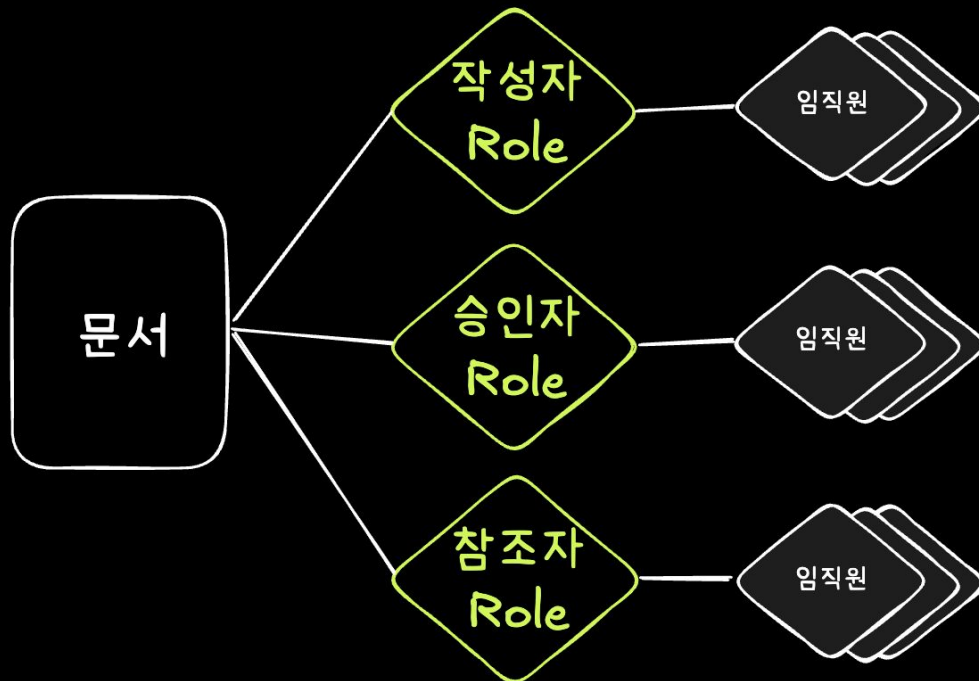
: Role

ReBA

: Rel

ABAC

: Att



RBAC

RBAC

ReBAC

ABAC

: Role

Coarse-Grained Authorization w/ RBAC

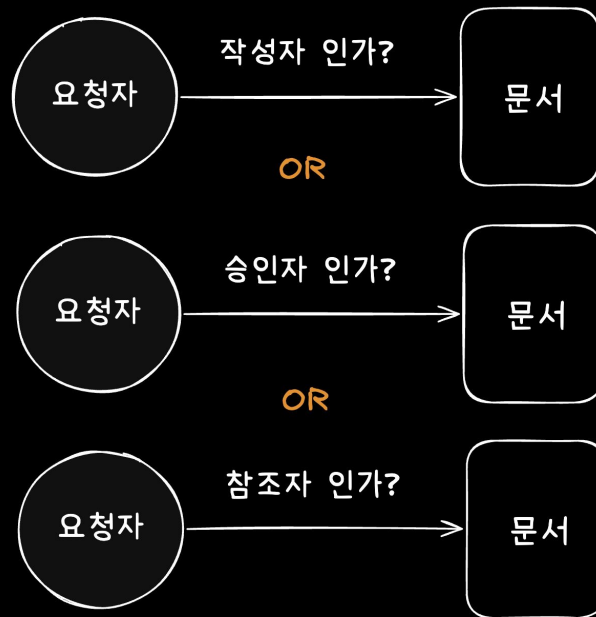
ReBA

: Rel

ABAC

: Att

조회 가능한가?



RBAC
: Role

ReBA
: Rel

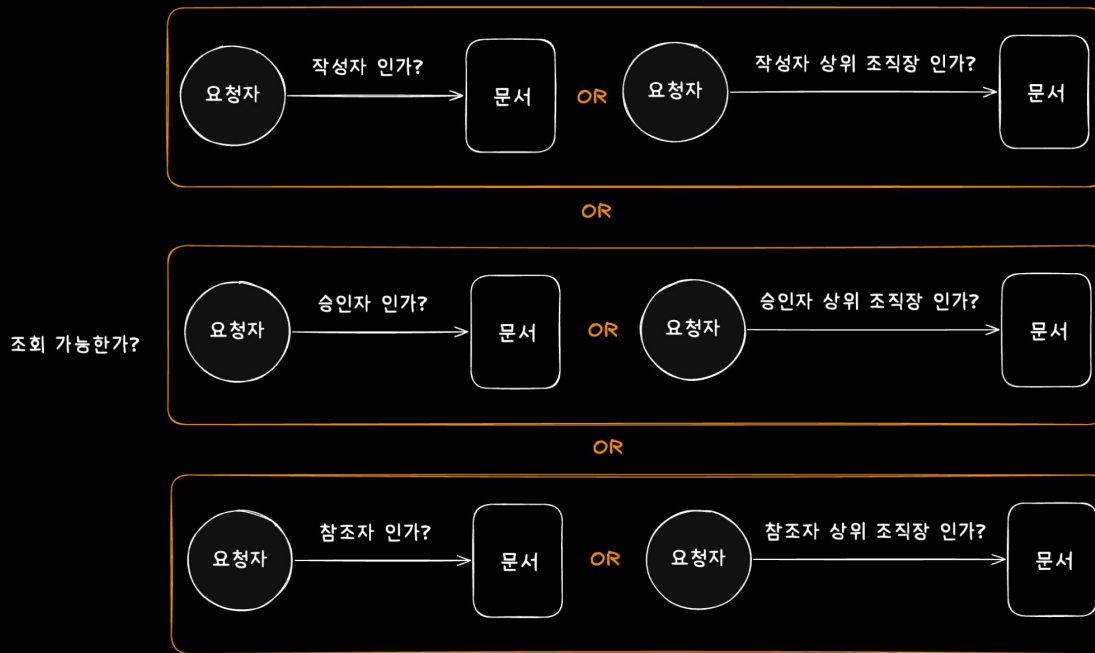
ABAC
: Att

RBAC

ReBAC

ABAC

Coarse-Grained Authorization w/ RBAC



RBAC

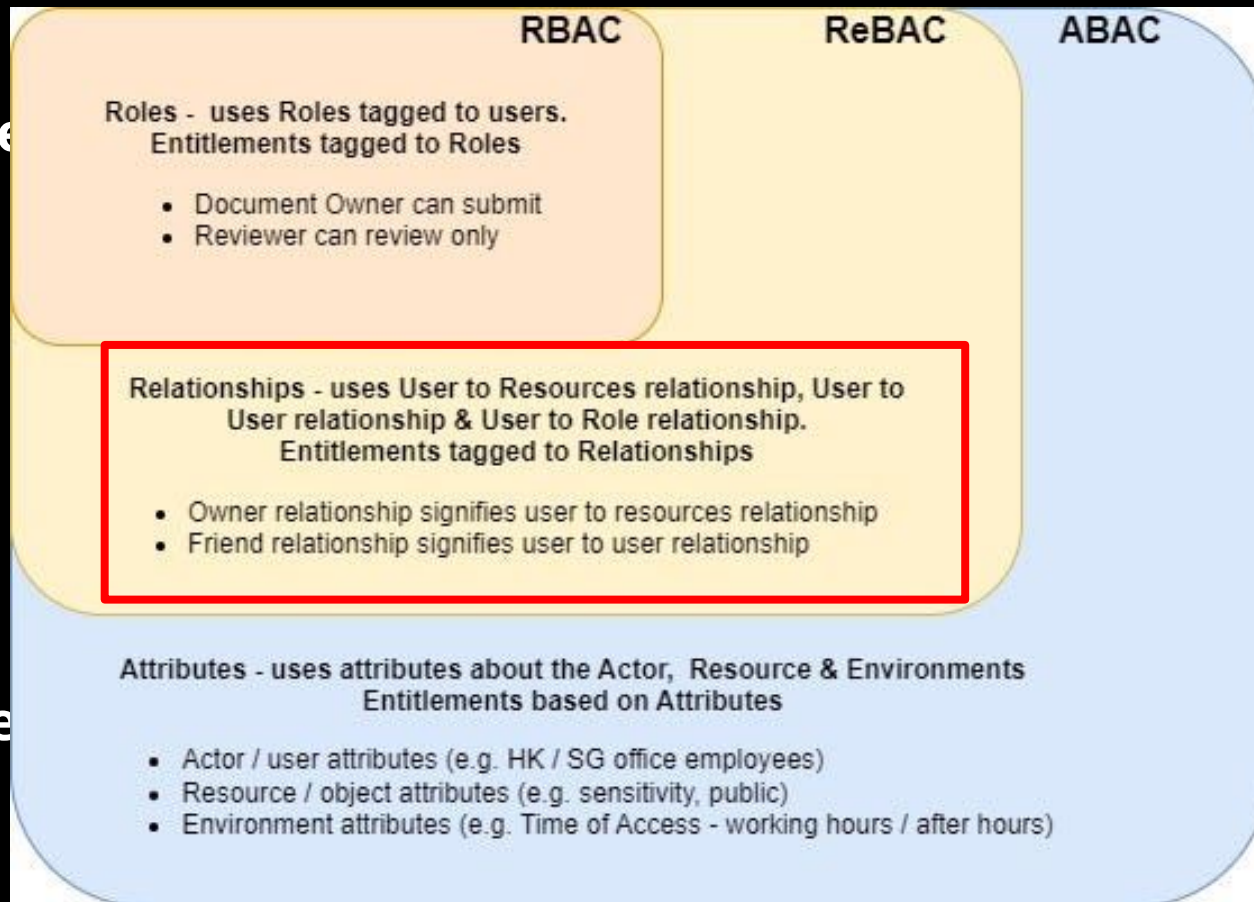
: Role Base

ReBAC

: Relation

ABAC

: Attribute



RBAC

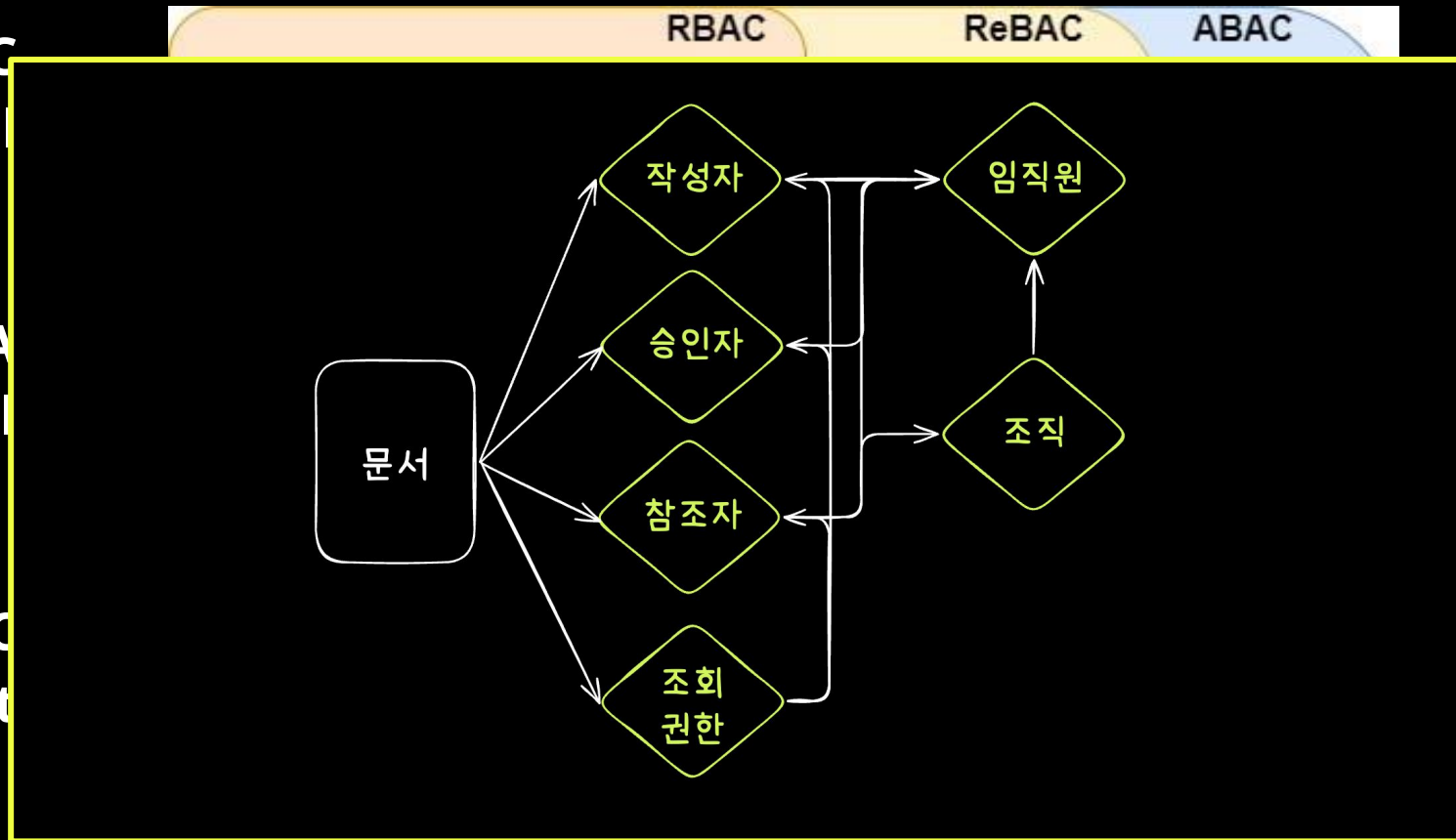
: Role

ReBA

: Rel

ABAC

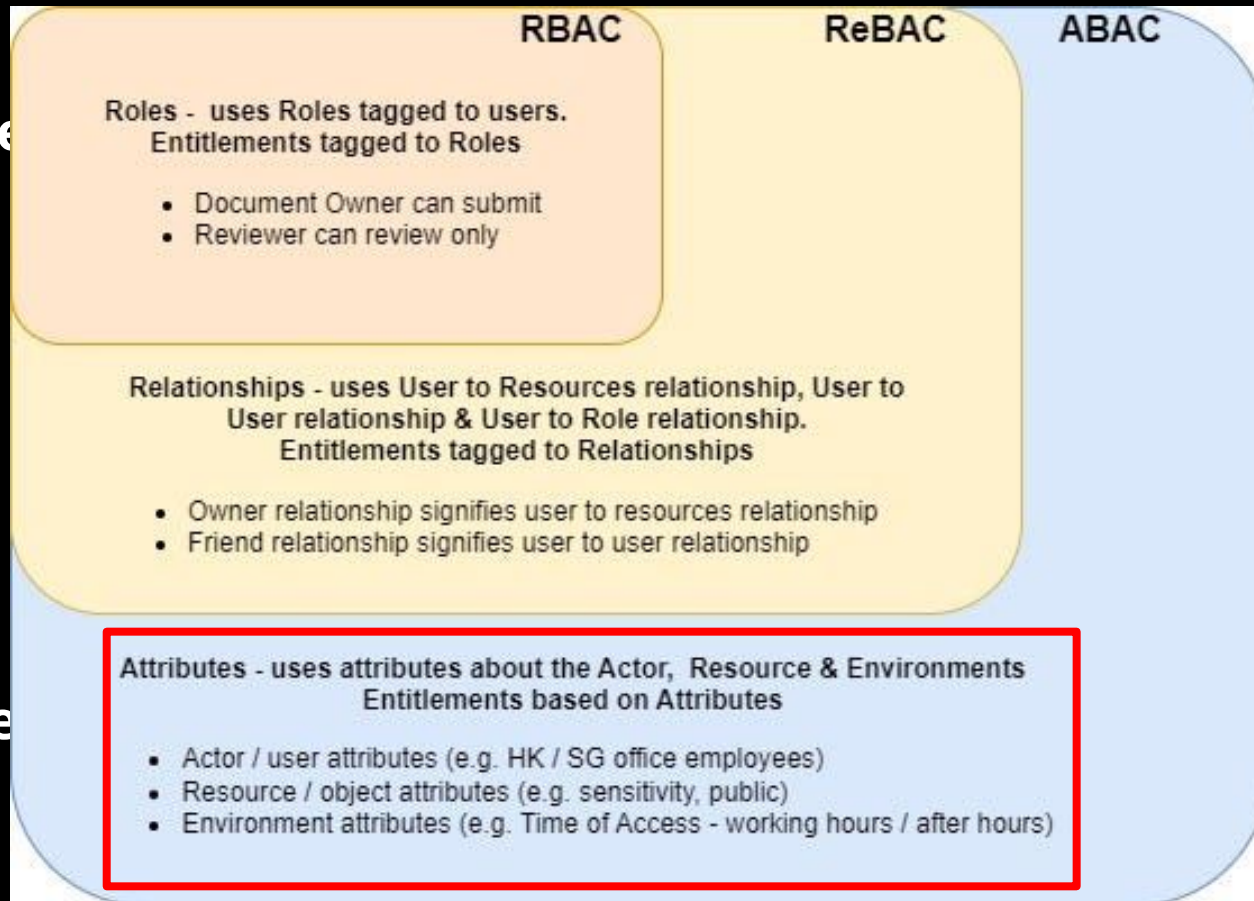
: Att



RBAC
: Role Based

ReBAC
: Relation

ABAC
: Attribute



RBAC

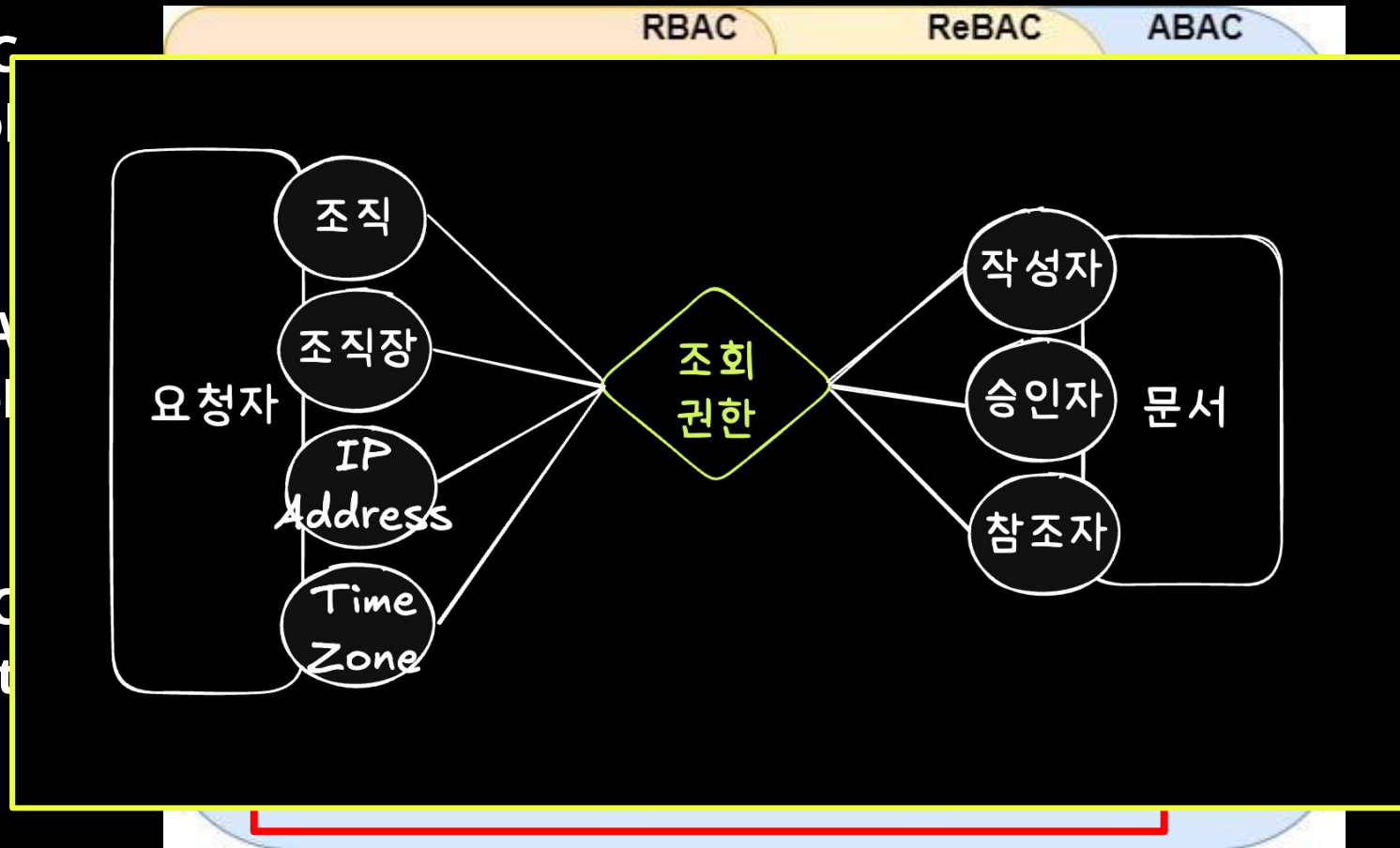
: Role

ReBA

: Rel

ABAC

: Att



RBAC

RBAC

ReBAC

ABAC

: Ro

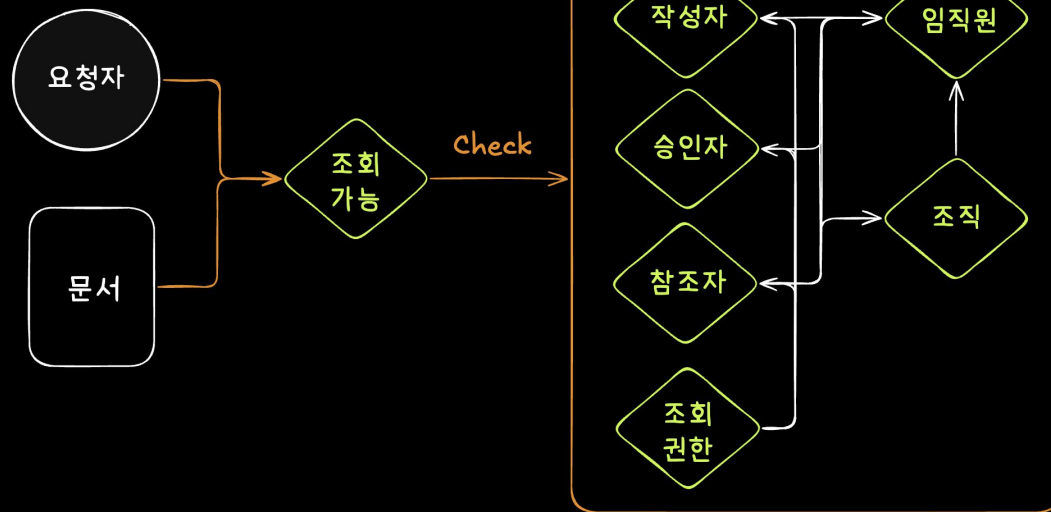
Fine-Grained Authorization w/ ReBAC, ABAC

ReBA

: Re

ABAC

: Att



	RBAC	ReBAC	ABAC
AuthZ	Coarse-Grained	Fine-Grained	
정책 관리	단순 명확	관계 복잡도	속성 복잡도
유연성	낮음	높음	매우 높음
확장성	낮음	중간	중간
정책 표현력	낮음	높음	높음 + 복잡
실시간 제어	어려움	적합	적합

	RBAC	ReBAC	ABAC
--	------	-------	------

AuthZ			
-------	--	--	--

Coarse-Grained			
----------------	--	--	--

Fine-Grained			
--------------	--	--	--

“Does user **U** have **relation R** with object **O**?”

확장성			
-----	--	--	--

낮음			
----	--	--	--

높음			
----	--	--	--

속성 복잡도			
--------	--	--	--

정책 표현력			
--------	--	--	--

낮음			
----	--	--	--

높음			
----	--	--	--

높음 + 복잡			
---------	--	--	--

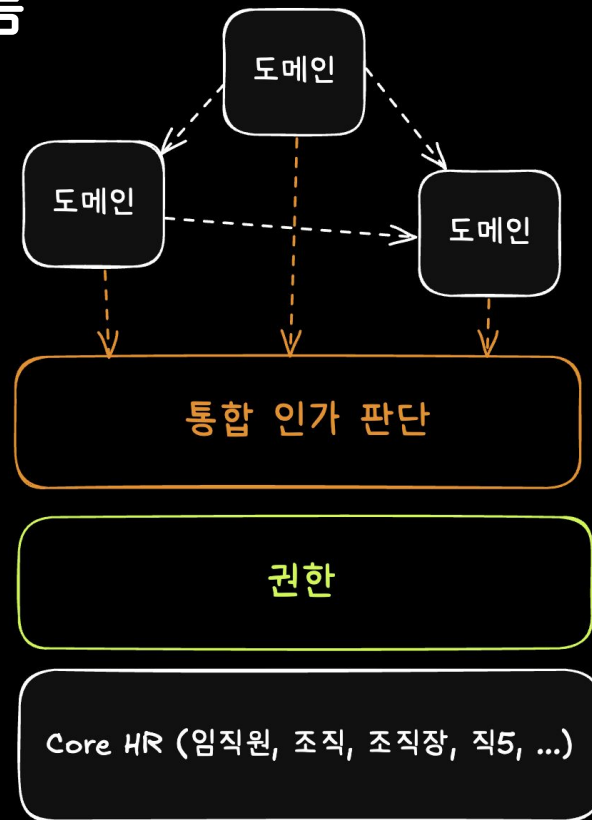
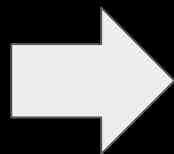
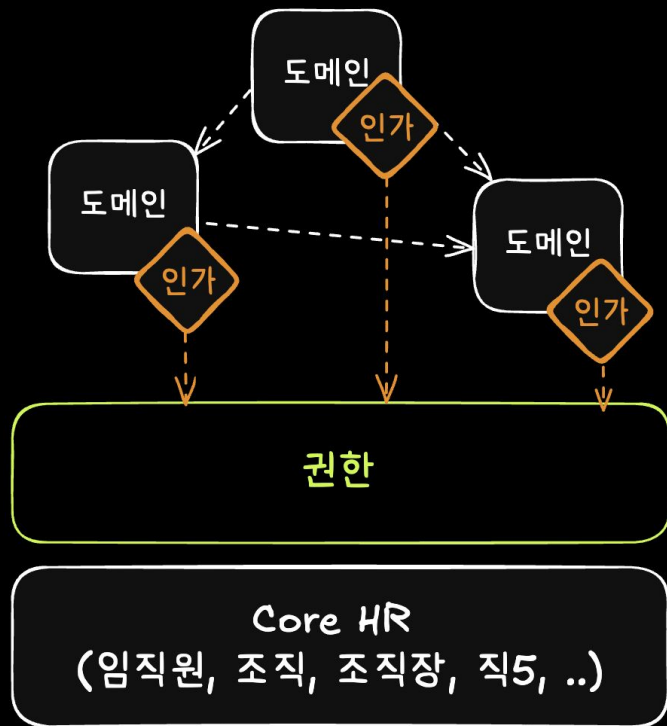
실시간 제어			
--------	--	--	--

어려움			
-----	--	--	--

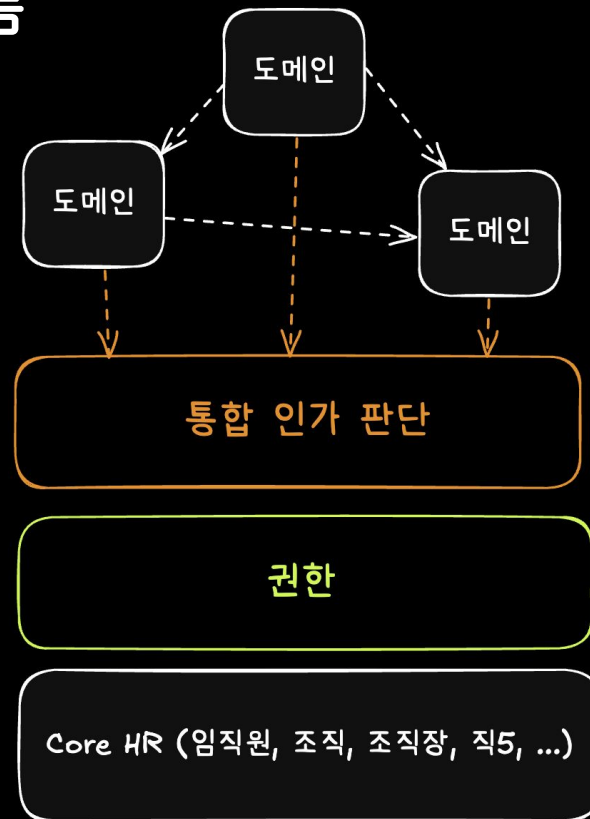
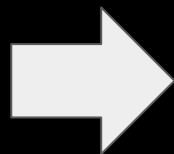
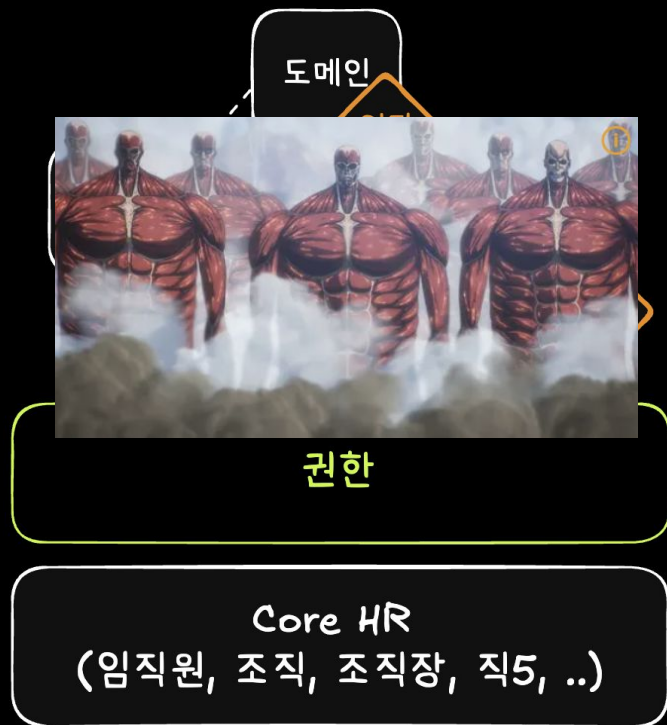
적합			
----	--	--	--

적합			
----	--	--	--

통합 인가 플랫폼



통합 인가 플랫폼

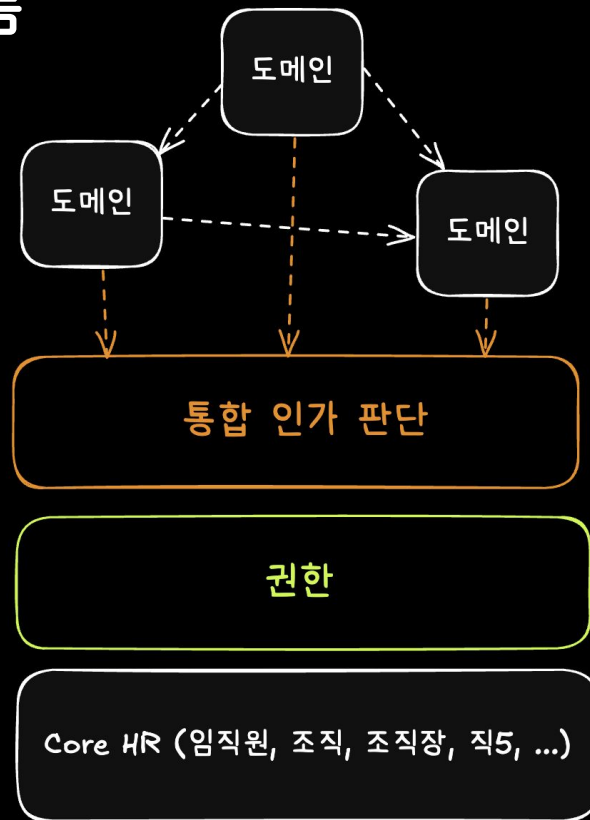


통합 인가 플랫폼

세분화된 도메인 리소스의 **관계(Relation)**
정의

Fine Grained Authorization

인가 판단의 통합



OpenFGA™

Relationship-based access control made fast, scalable, and easy to use.

OpenFGA is an open-source authorization solution that allows developers to build granular access control using an easy-to-read modeling language and friendly APIs.

 Quick start

 Join us on GitHub



OpenFGA™

Relationship-based access control made fast, scalable and easy to use.

OpenFGA is an open-source authorization solution that allows developers to build granular access control using an easy-to-read modeling language and friendly APIs.

[⚡ Quick start](#)[👤 Join us on GitHub](#)

Zanzibar is flexible, global and fast.

Zanzibar allows Google teams to specify their unique authorization models, globally replicates authorization data and responds to access checks blazing fast.

1 Basics2 Editors → Viewer3 Groups4 Folders

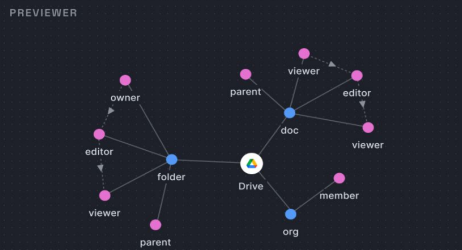
As part of defining a user's relation with an object, Zanzibar considers relations the user has with other objects. For example, hierarchical models where permissions on a child object are inherited from the permissions on the parent.

Take the tour ▶

NAMESPACE

```
01 name: "folder"
02 relation ( name: "parent" )
03 relation ( name: "owner" )
04 relation {
05   name: "editor"
06   userset_rewrite {
07     union {
08       child { _this {} }
09       child { computed_userset { relation: "owner" } }
10     }
11   }
12   relation {
13     name: "viewer"
14     userset_rewrite {
15       union {
16         child { _this {} }
17         child { computed_userset { relation: "editor" } }
18       }
19     }
20   }
21 }
```

PREVIEWER



TUPLES

folder:planning is parent of the doc:readme

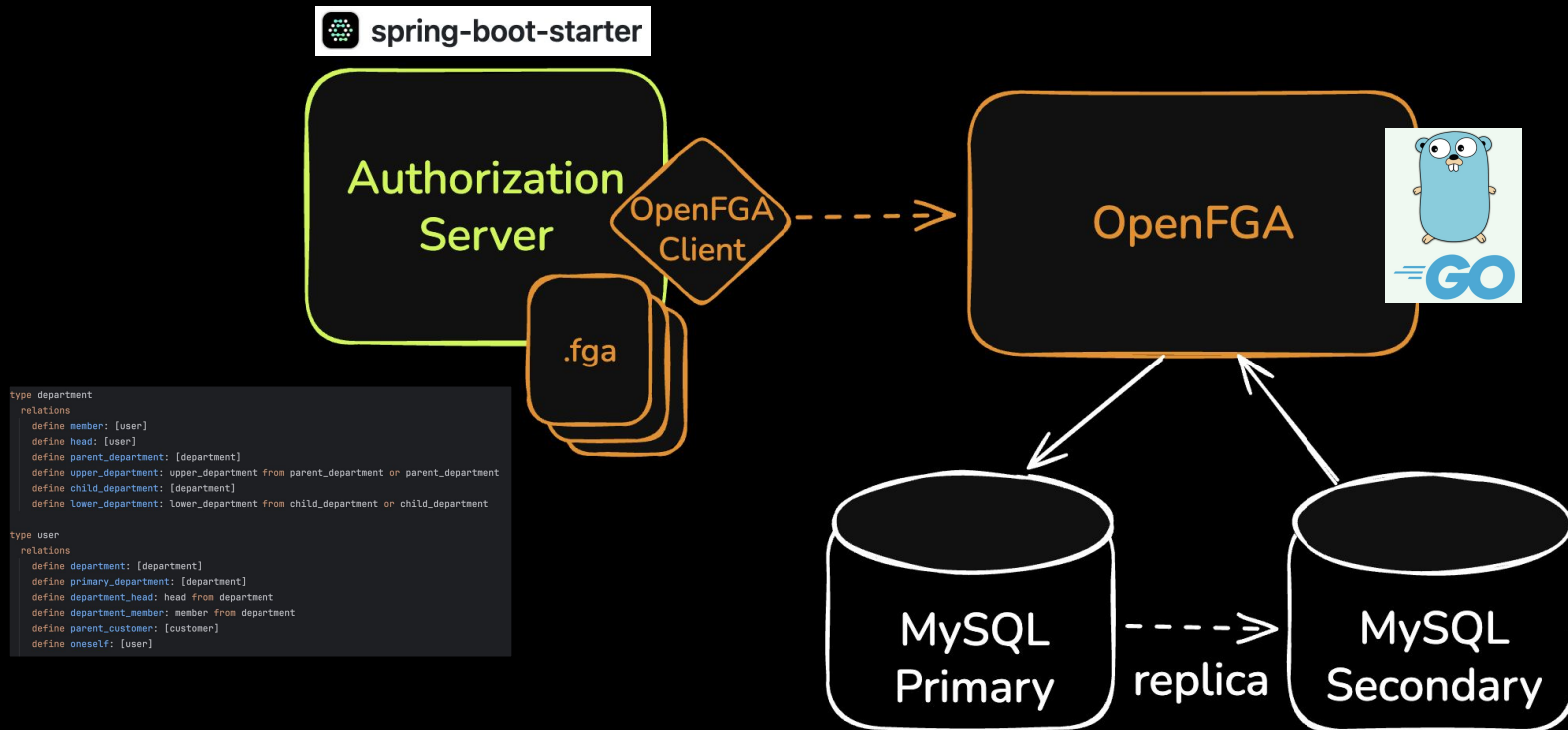
Kim is viewer of the folder:planning

QUERY

Is Kim a viewer of doc:readme ?

Yes, Kim → viewer on folder:planning, so Kim → viewer on doc:readme

flex Authorization Server w/ OpenFGA



openfga spring boot starter

```
implementation("dev.openfga:openfga-spring-boot-starter:0.2.0")
```

```
# src/main/resources/application.yaml

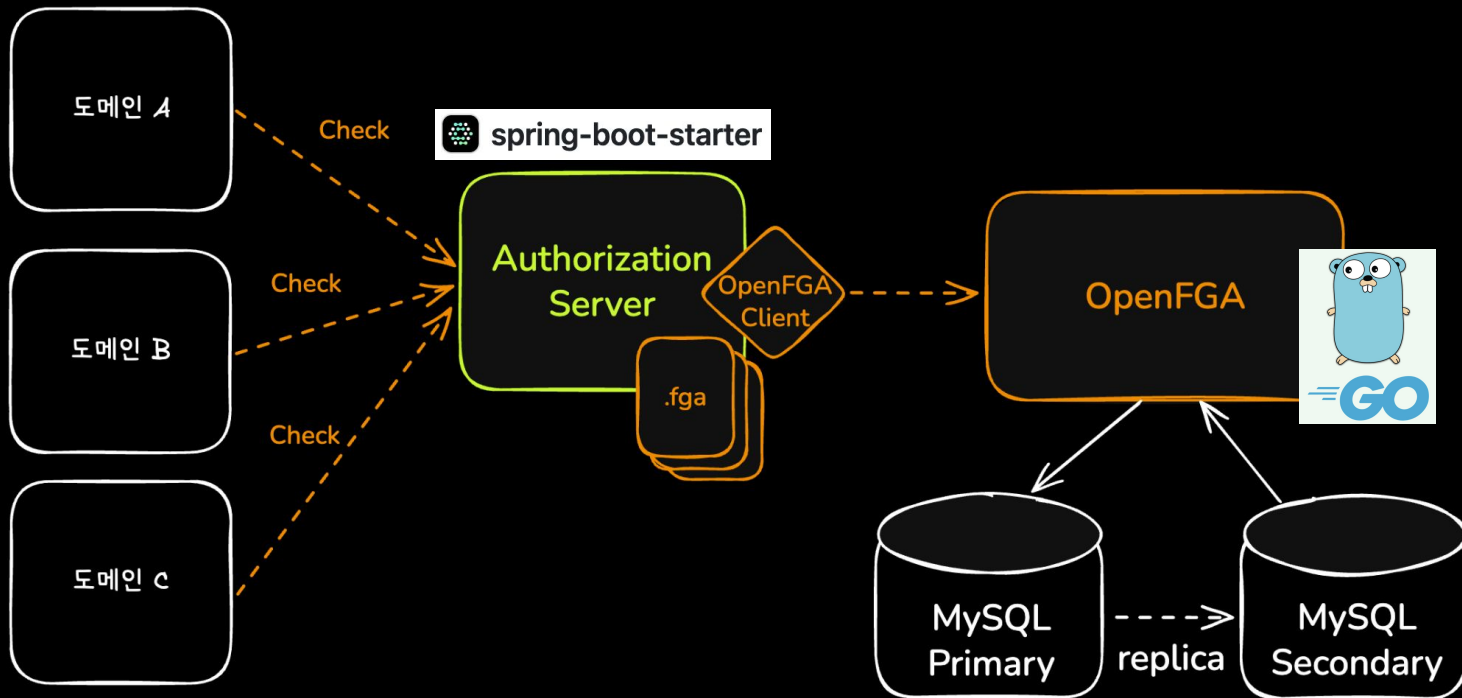
openfga:
  api-url: YOUR_FGA_API_URL
  store-id: YOUR_FGA_STORE_ID
  authorization-model-id: YOUR_FGA_AUTHORIZATION_MODEL_ID
  user-agent: YOUR_USER_AGENT # default: openfga-sdk java/ve
  read-timeout: 1m # default: 10 seconds
  connect-timeout: 15 # default: 10 seconds
  max-retries: 3 # default: no retries
  minimum-retry-delay: 1m
  http-version: HTTP_2
  default-headers:
    X-SOME-HEADER: Some Header Value
  telemetry-configuration:
    fga_client_request_model_id: YOUR_FGA_CLIENT_REQUEST_MOD
  credentials:
    method: CLIENT_CONFIGURATION # constant
  config:
    client-id: YOUR_CLIENT_ID
    client-secret: YOUR_CLIENT_SECRET
```

openfga spring boot starter

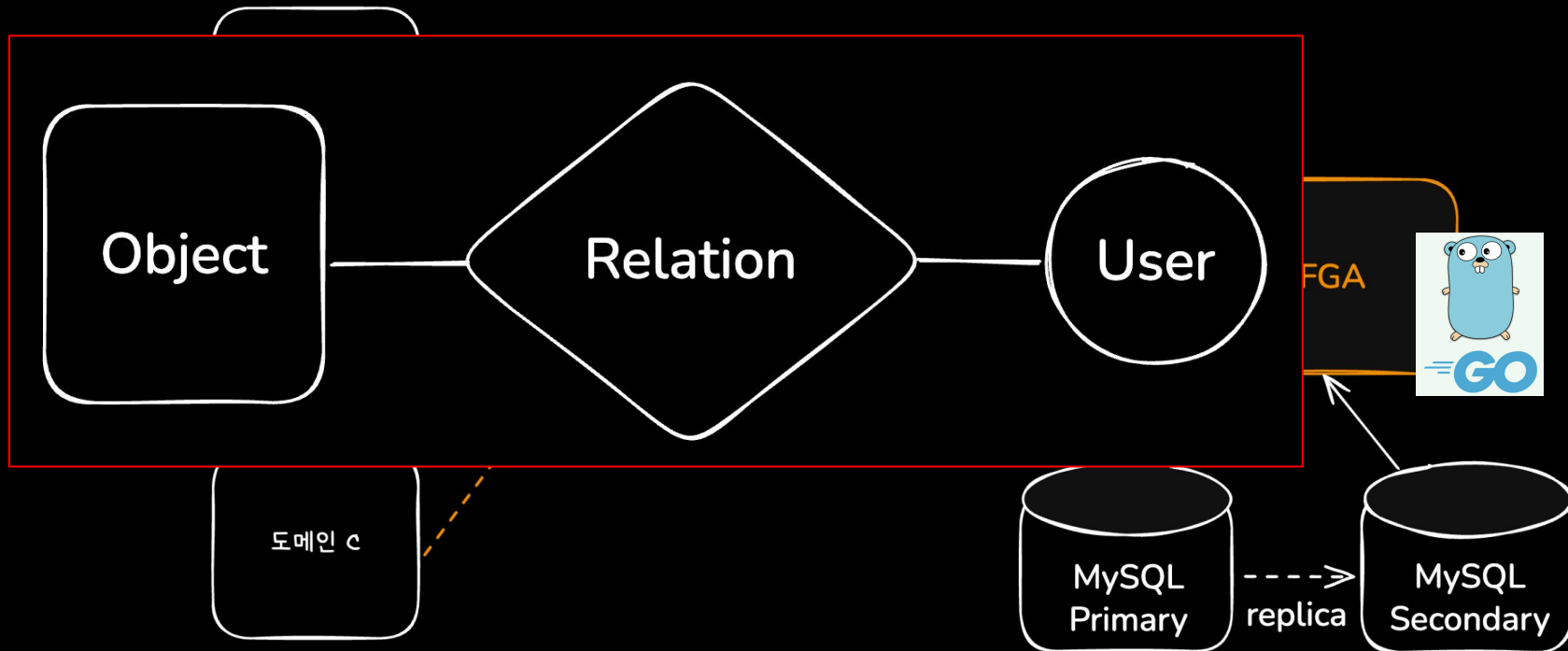
Configures an `openFgaClient` and `openFga` beans based on configuration values. The beans will only be created if the `OpenFgaClient` is present on the classpath, and the `openfga.api-url` is specified.

```
@AutoConfiguration
@ConditionalOnFgaProperties
@EnableConfigurationProperties(OpenFgaProperties.class)
public class OpenFgaAutoConfiguration {
```

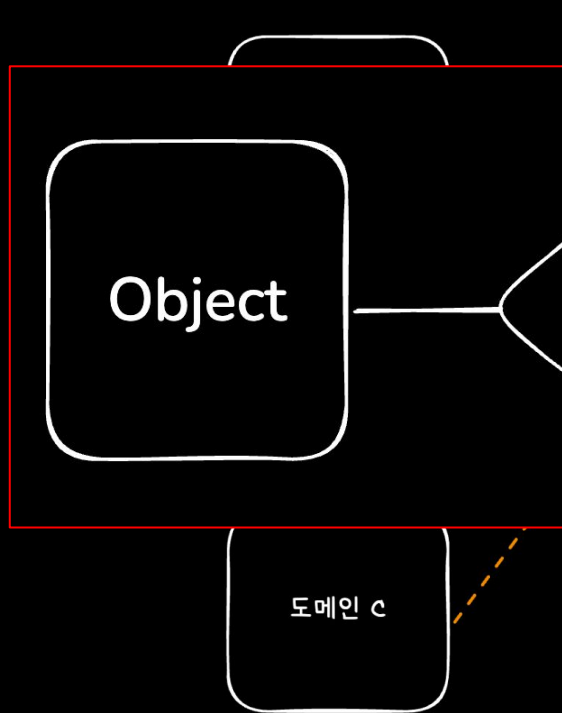
Check Authorization



Check Authorization



Check Authorization



Perform an FGA check. Returns `true` if the user has the specified relationship with the object, `false` otherwise. The user ID will be obtained from the authentication name in the `org.springframework.security.core.context.SecurityContext`

Params:

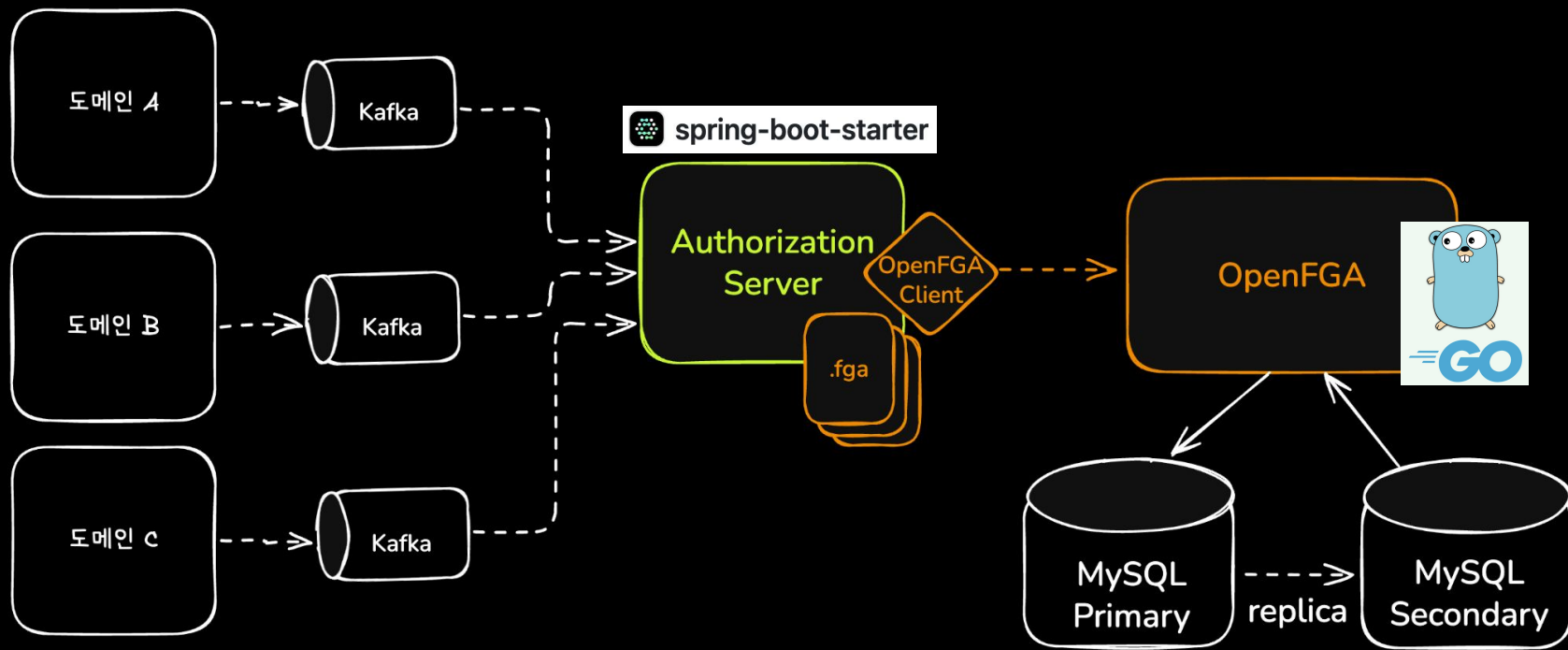
- `objectType` – The object type of the check
- `objectId` – The ID of the object to check
- `relation` – The required relation between the user and the object
- `userType` – The type of the user
- `userId` – The ID of the user

Returns: true if the user has the required relation to the object, false otherwise

See Also: [FGA Check API](#) ↗

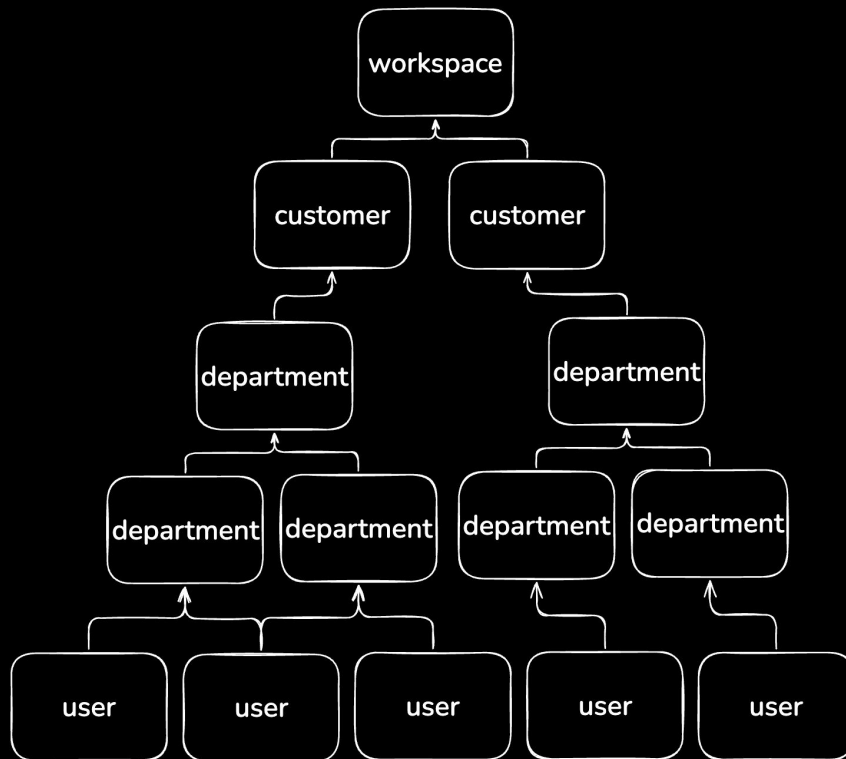
```
public boolean check(String objectType, String objectId, String relation, String userType, String userId) {  
    var body = new ClientCheckRequest()  
        .user(String.format("%s:%s", userType, userId))  
        .relation(relation)  
        ._object(String.format("%s:%s", objectType, objectId));  
  
    try {  
        return Boolean.TRUE.equals(fgaClient.check(body).get().getAllowed());  
    } catch (InterruptedException | FgaInvalidParameterException | ExecutionException e) {  
        throw new OpenFgaCheckException("Error performing FGA check", e);  
    }  
}
```

Write Relationship Tuple

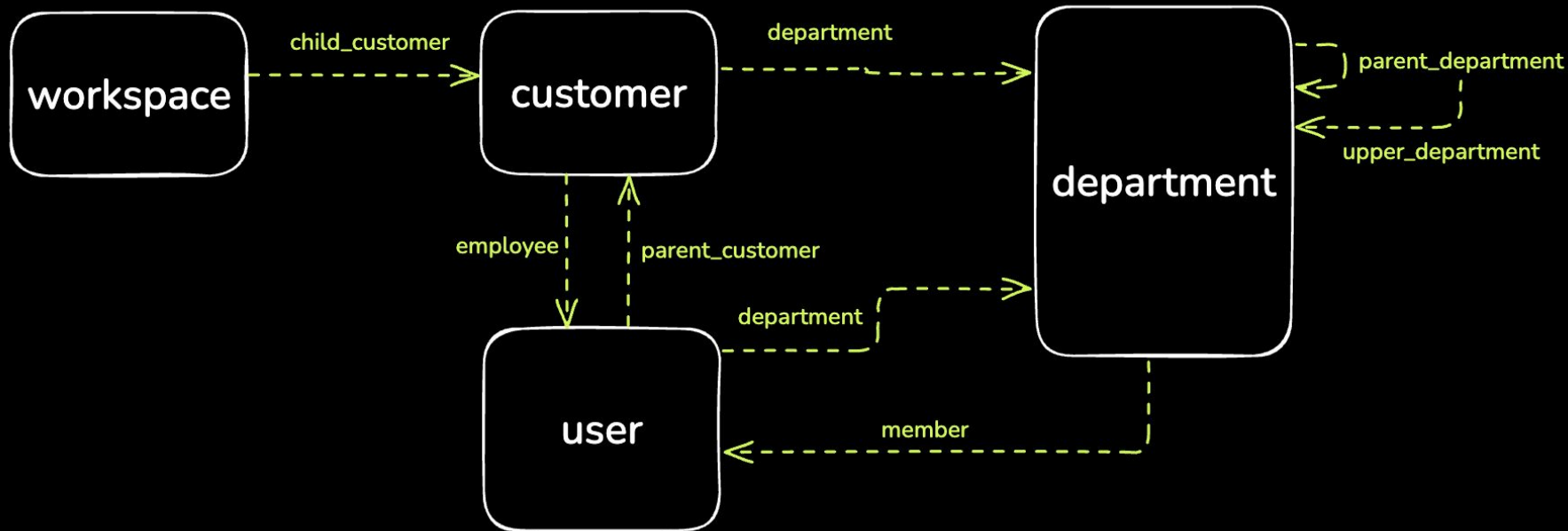


.fga
Relation 설계부터 해볼까요?

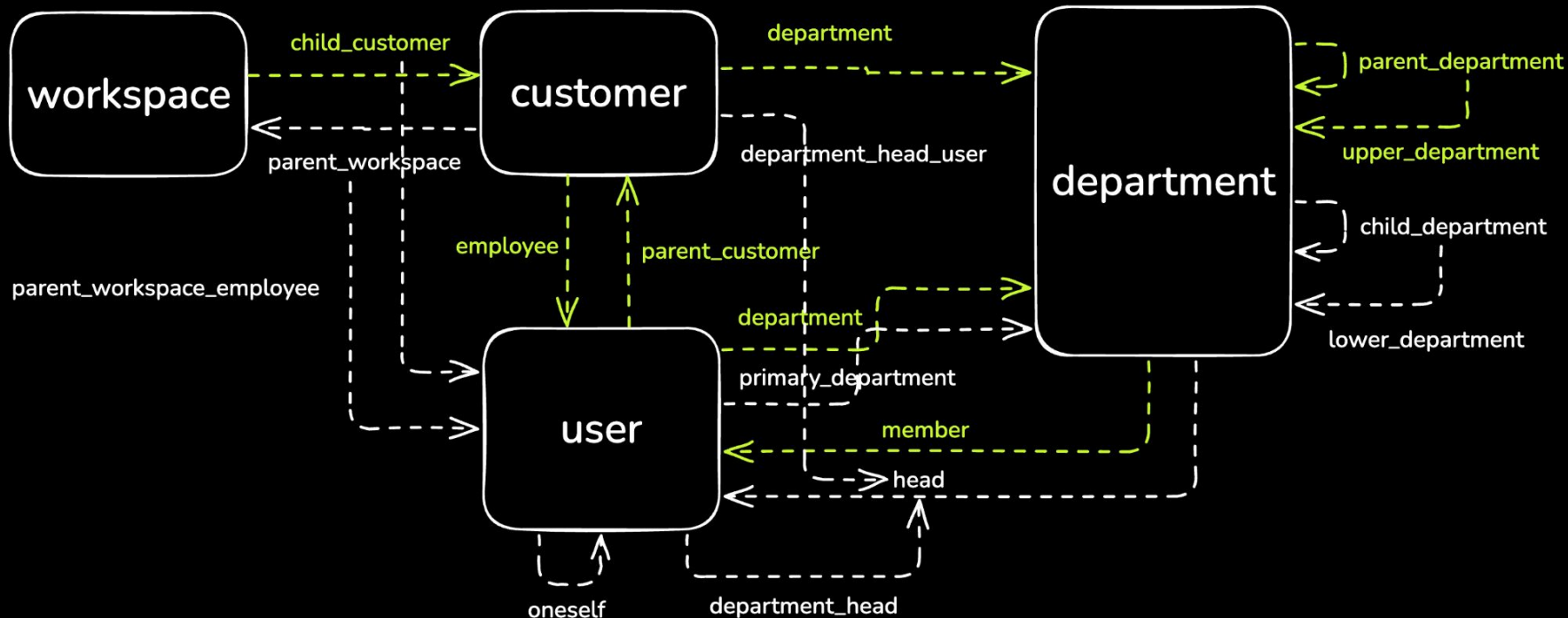
Relation : workspace - customer - department - user



Relation : workspace - customer - department - user



Relation : workspace - customer - department - user



Relation : workspace - customer - department - user

```
type department
  relations
    define member: [user]
    define head: [user]
    define parent_department: [department]
    define upper_department: upper_department from parent_department or parent_department
    define child_department: [department]
    define lower_department: lower_department from child_department or child_department

type user
  relations
    define department: [department]
    define primary_department: [department]
    define department_head: head from department
    define department_member: member from department
    define parent_customer: [customer]
    define oneself: [user]
```

t_department
department
department
department
department

Relation : workspace - customer - department - user

type department

relations

define member: [user]

define head: [user]

define parent_department: [department]

define upper_department: upper_department from parent_department or parent_department

define child_department: [department]

define lower_department: lower_department from child_department or child_department

type user

relations

define department: [department]

define primary_department: [department]

define department_head: head from department

define department_member: member from department

define parent_customer: [customer]

define oneself: [user]

t_department

department

department

department

Relation : workspace - customer - department - user

```
type department
  relations
    define member: [user]
    define head: [user]
    define parent_department: [department]
    define upper_department: upper_department from parent_department or parent_department
    define child_department: [department]
    define lower_department: lower_department from child_department or child_department

type user
  relations
    define department: [department]
    define primary_department: [department]
    define department_head: head from department
    define department_member: member from department
    define parent_customer: [customer]
    define oneself: [user]
```

t_department
department
department
department
department

Relation : workspace - customer - department - user

```
type department
  relations
    define member: [user]
    define head: [user]
    define parent_department: [department]
    define upper_department: upper_department from parent_department or parent_department
    define child_department: [department]
    define lower_department: lower_department from child_department or child_department

type user
  relations
    define department: [department]
    define primary_department: [department]
    define department_head: head from department
    define department_member: member from department
    define parent_customer: [customer]
    define oneself: [user]
```

t_department
department
department
department
department

Relation : workspace - customer - department - user

```
type department
  relations
    define member: [user]
    define head: [user]
    define parent_department: [department]
    define upper_department: upper_department from parent_department or parent_department
    define child_department: [department]
    define lower_department: lower_department from child_department or child_department

type user
  relations
    define department: [department]
    define primary_department: [department]
    define department_head: head from department
    define department_member: member from department
    define parent_customer: [customer]
    define oneself: [user]
```

t_department
department
department
department

Relation : workspace - customer - department - user

```
type department
  relations
    define member: [user]
    define head: [user]
    define parent_department: [department]
    define upper_department: upper_department from parent_department or parent_department
    define child_department: [department]
    define lower_department: lower_department from child_department or child_department

type user
  relations
    define department: [department]
    define primary_department: [department]
    define department_head: head from department
    define department_member: member from department
    define parent_customer: [customer]
    define oneseIf: [user]
```

t_department
department
department
department

Relation : workspace - customer - department - user

```
type department
  relations
    define member: [user]
    define head: [user]
    define parent_department: [department]
    define upper_department: upper_department from parent_department or parent_department
    define child_department: [department]
    define lower_department: lower_department from child_department or child_department

type user
  relations
    define department: [department]
    define primary_department: [department]
    define department_head: head from department
    define department_member: member from department
    define parent_customer: [customer]
    define oneself: [user]
```

t_department
department
department
department

Relation : workspace - customer - department - user

```
type workspace
  relations
    define child_customer: [customer]
    define employee: employee from child_customer

type customer
  relations
    define parent_workspace: [workspace]
    define parent_workspace_employee: employee from parent_workspace
    define department: [department]
    define department_head_user: head from department
    define employee: [user]
```

works

parent_work

department

department

department

department

oneself

department_head

Relation 확장(권한) 정의

```
extend type user
  relations
    define can_read_work_experience: department_head or oneself
    define can_modify_address: oneself
    define can_resign: hr_manager
```

Relation 확장(권한) 정의

```
extend type user
  relations
    define can_read_work_experience: department_head or oneself
    define can_modify_address: oneself
    define can_resign: hr_manager
```

임직원의 경력 정보 조회 relation 은 조직 또는 본인과 연결된다



임직원의 경력 정보 조회 요청은 **조직장** 또는 **본인**에게만 **허가**한다

OpenFGA Operations

OpenFGA Operations

.fga 등록 :	writeAuthorizationModel
relationship tuple 등록 / 삭제 :	write
인가 체크 :	check
Allowed 리소스 목록 조회 :	listObjects
리소스에 Allowed 유저 목록 조회 :	listUsers

Write Authorization Model

```
fun writeAuthorizationSchema(authorizationModel: AuthorizationModel): String =  
    this.openFgaClient.writeAuthorizationModel(  
        WriteAuthorizationModelRequest()  
            .schemaVersion(authorizationModel.schemaVersion)  
            .typeDefinitions(authorizationModel.typeDefinitions)  
            .conditions(authorizationModel.conditions)  
    )  
    .await()  
    .authorizationModelId
```

Write Authorization Model

```
fun writeAuthorizationSchema(authorizationModel: AuthorizationModel): String =  
    this.openFgaClient.writeAuthorizationModel(  
        WriteAuthorizationModelRequest()  
            .schemaVersion(authorizationModel.schemaVersion)  
            .typeDefinitions(authorizationModel.typeDefinitions)  
            .conditions(authorizationModel.conditions)  
    )  
    .await()  
    .authorizationModelId
```

Write Authorization Model

Table: authorization_model

	store	authorization_model_id	type	type_definition ...	schema_version	serialized_protobuf
1	01HW80GENFS8NNHGBKQ47E0TJN	01JKT8ZQXHX3DS6TQ0C813XHDS		<null>	1.2	204.8kB of 309.98kB 0x0A1A30314A4B54425A51515841
2	01HW80GENFS8NNHGBKQ47E0TJN	01JMGZZRDC3JGNT6WV1Z4596YJ		<null>	1.2	204.8kB of 309.98kB 0x0A1A30314A4D475A5A52444331
3	01HW80GENFS8NNHGBKQ47E0TJN	01JMVSDH2YAD0Q09M5KFFJKV31		<null>	1.2	204.8kB of 309.79kB 0x0A1A30314A4D56534448325941
4	01HW80GENFS8NNHGBKQ47E0TJN	01JMXE8TVG49NE12V181H3BQ1S		<null>	1.2	204.8kB of 309.79kB 0x0A1A30314A4D58453854564731
5	01HW80GENFS8NNHGBKQ47E0TJN	01JMYA32W1T16GMJGT6D4M7XR4		<null>	1.2	204.8kB of 309.79kB 0x0A1A30314A4D59413332573151
6	01HW80GENFS8NNHGBKQ47E0TJN	01JN0XEN8PKJB24T93XRPRXXQ0		<null>	1.2	204.8kB of 311.67kB 0x0A1A30314A4E3058454E385041
7	01HW80GENFS8NNHGBKQ47E0TJN	01JPKX5NH8RATWJTXPGJA889Y9		<null>	1.2	204.8kB of 313.66kB 0x0A1A30314A504B58354E483851
8	01HW80GENFS8NNHGBKQ47E0TJN	01JPPJCMNJ08883HA177KBHGNW		<null>	1.2	204.8kB of 313.49kB 0x0A1A30314A50504A434D4E4A31
9	01HW80GENFS8NNHGBKQ47E0TJN	01JPSMF03CVX4RC0N7AWBTDEHQ		<null>	1.2	204.8kB of 315.12kB 0x0A1A30314A50534D4630334351
10	01HW80GENFS8NNHGBKQ47E0TJN	01JQNQ0NY864NFBM16BK052XG7		<null>	1.2	204.8kB of 316.5kB 0x0A1A30314A514E51304E5938471
11	01HW80GENFS8NNHGBKQ47E0TJN	01JQNX5WM4QPT5RTEBQH0TV9CV		<null>	1.2	204.8kB of 316.56kB 0x0A1A30314A514E5835574D3451
12	01HW80GENFS8NNHGBKQ47E0TJN	01JR7EZDSETK99ZEBX6C5P3FP		<null>	1.2	204.8kB of 316.72kB 0x0A1A30314A5237455A44534551
13	01HW80GENFS8NNHGBKQ47E0TJN	01JS44SEQYX2WV1MTJQQFDP1CR		<null>	1.2	204.8kB of 317.05kB 0x0A1A30314A5334345345515951
14	01HW80GENFS8NNHGBKQ47E0TJN	01JTMPVVCVJP0K193WESZZD0SA		<null>	1.2	204.8kB of 315.4kB 0x0A1A30314A544D50565743564A1

Write Relationship Tuple

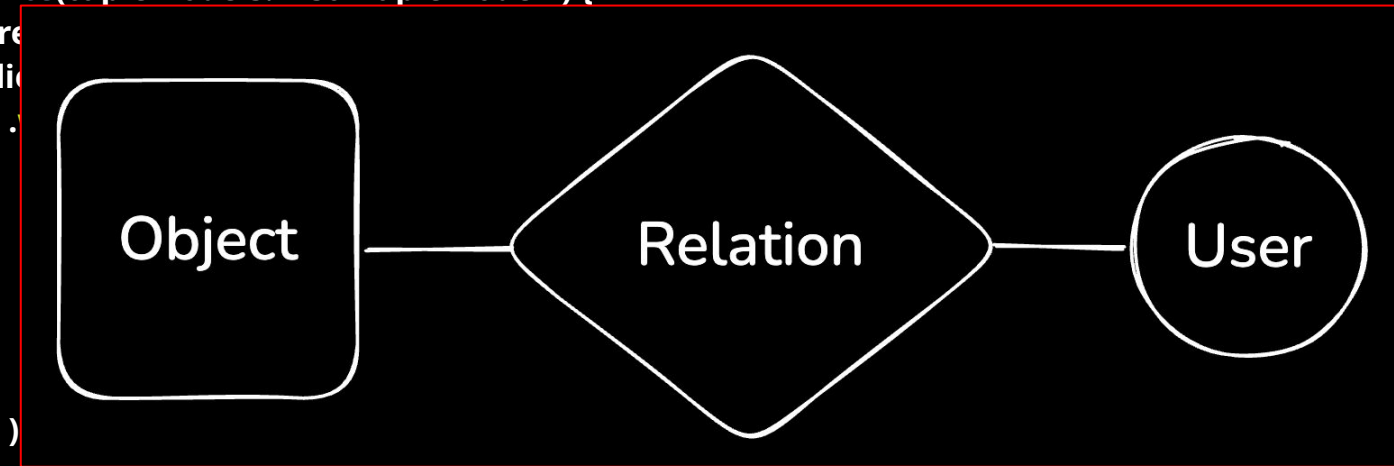
```
fun write(tupleModels: List<TupleModel>) {  
    val response = this.openFgaClient.write(  
        ClientWriteRequest()  
            .writes(  
                tupleModels.map {  
                    ClientTupleKey()  
                        ._object(it._object)    // "department:{departmentId}"  
                        .relation(it.relation)  // "member"  
                        .user(it.user)         // "user:{userId}"  
                }  
            )  
    )  
    .await()  
  
    check(response.statusCode == 200)  
}
```

Write Relationship Tuple

```
fun write(tupleModels: List<TupleModel>) {
```

```
    val re
```

```
    Cli
```



```
    ).await()
```

```
    check(response.statusCode == 200)
```

```
}
```

Write Relationship Tuple

```
fun write(tupleModels: List<TupleModel>) {  
    val response = this.openFgaClient.write(  
        ClientWriteRequest()
```

```
        .writes(  
            tupleModels.map {
```

```
                ClientTupleKey()
```

```
                ._object(it._object)
```

```
                // "department:{departmentId}"
```

```
                .relation(it.relation) // "member"
```

```
                .user(it.user)        // "user:{userId}"
```

```
            },
```

```
        )
```

```
    )
```

```
    .await()
```

```
    check(response.status)
```

```
}
```

```
type department
```

```
    relations
```

```
        define member: [user]
```

```
        define head: [user]
```

```
        define parent_department: [department]
```

```
        define upper_department: upper_department from parent_department or parent_department
```

```
        define child_department: [department]
```

```
        define lower_department: lower_department from child_department or child_department
```


Write Relationship Tuple

Table: tuple

	store	object_type	object_id	relation	_user	user_type	ulid	inserted_at
1	01HW80GENFS8NNHGBKQ47E0TJN	department	1000	member	user:2098	user	01J17RJK6NCSVKFYS9AMB14JD5	2024-06-25 22:15:16
2	01HW80GENFS8NNHGBKQ47E0TJN	department	10000	member	user:40703	user	01J19ZAS4GWG79PHKS5529NPYW	2024-06-26 18:51:49
3	01HW80GENFS8NNHGBKQ47E0TJN	department	10000	member	user:40950	user	01J19ZAWDW5T1RET4F9CCWF8NM	2024-06-26 18:51:52
4	01HW80GENFS8NNHGBKQ47E0TJN	department	10000	member	user:41100	user	01J19ZAY6Q3YC92SBH8FYDZEXE	2024-06-26 18:51:54
5	01HW80GENFS8NNHGBKQ47E0TJN	department	10000	member	user:41224	user	01J19ZAZVMJ9Q8H5TC3T37NKGJ	2024-06-26 18:51:56
6	01HW80GENFS8NNHGBKQ47E0TJN	department	10000	member	user:41329	user	01J19ZB1EN0CGT3XQEH2TVAW99	2024-06-26 18:51:57
7	01HW80GENFS8NNHGBKQ47E0TJN	department	10000	member	user:41485	user	01J19ZB3FD2J6YVS2ZAKT22JPZ	2024-06-26 18:51:59
8	01HW80GENFS8NNHGBKQ47E0TJN	department	10001	member	user:40761	user	01J19ZASREVD93N3KB2YKYFZ1J	2024-06-26 18:51:50
9	01HW80GENFS8NNHGBKQ47E0TJN	department	10001	member	user:40867	user	01J19ZAV5MG6J574YR0S24Z11V	2024-06-26 18:51:51
10	01HW80GENFS8NNHGBKQ47E0TJN	department	10001	member	user:40925	user	01J19ZAW33PV0ZNM0SRPYSE0QA	2024-06-26 18:51:52
11	01HW80GENFS8NNHGBKQ47E0TJN	department	10001	member	user:41179	user	01J19ZAZ8YJZ0ST92MNBQV7T02	2024-06-26 18:51:55
12	01HW80GENFS8NNHGBKQ47E0TJN	department	10001	member	user:41436	user	01J19ZB2XY7QM06HDPTSKKF666	2024-06-26 18:51:59
13	01HW80GENFS8NNHGBKQ47E0TJN	department	10001	member	user:41493	user	01J19ZB3N4T3A6153841D00BQ9	2024-06-26 18:52:00

Delete Relationship Tuple

```
fun delete(tupleModels: List<TupleModel>) {  
    val response = this.openFgaClient.write(  
        ClientWriteRequest()  
            .deletes(  
                tupleModels.map {  
                    ClientTupleKey()  
                        ._object(it._object)    // "department:{departmentId}"  
                        .user(it.user)          // "user:{userId}"  
                        .relation(it.relation)   // "member"  
                }  
            )  
    )  
    .await()  
  
    check(response.statusCode == 200)  
}
```

Perform a Check

```
fun check(tupleModel: TupleModel): Boolean {  
    val response = this.openFgaClient.check(  
        ClientCheckRequest()  
            .user(it.user)           // "user:{userId}"  
            .relation(it.relation)  // "can_read_work_experience"  
            ._object(it._object)    // "user:{userId}"  
    )  
    .await()  
  
    return response.allowed ? false  
}
```

Perform a Check

```
fun check(tupleModel: TupleModel): Boolean {  
    val response = this.openFgaClient.check(  
        ClientCheckRequest()  
            .user(it.user)          // "user:{userId}"  
            .relation(it.relation) // "can_read_work_experience"  
            ._object(it._object)   // "user:{userId}"  
    )  
    .await()  
  
    return response.allowed ? false  
}
```

Perform a list objects call

```
fun listObjects(tupleModel: TupleModel): List<TupleModel> {  
    val response = this.openFgaClient.listObjects(  
        ClientListObjectsRequest()  
            .user(it.user)          // "user:{userId}"  
            .relation(it.relation)  // "can_read_work_experience"  
            ._object(it._object)    // "user"  
    )  
    .await()  
  
    // 요청자가 경력 정보 접근할 수 있는 임직원 목록 : ["user:1", "user:2"]  
    return response.objects.map {  
        TupleModel(  
            user = tupleModel.user,  
            relation = tupleModel.relation,  
            _object = it,  
        )  
    }  
}
```

Perform a list users call

```
fun listUsers(objectType: String, objectId: String, relation: String): List<String> {  
    val response = this.openFgaClient.listUsers(  
        ClientListUsersRequest()  
            ._object(FgaObject().type(objectType).id(objectId)) // "user:{userId}"  
            .relation(relation) // "can_read_work_experience"  
            .userFilters(  
                listOf(  
                    UserTypeFilter().type("user")  
                )  
            )  
    )  
    .await()  
  
    // 대상 유저의 경력 정보 접근이 허용된 사람 목록 : users: [{object: { type: "user", id: 1}}, {object: { type: "user", id:  
    2}}]  
    return response.users.map { it.id }  
}
```

+ ABAC : Attribute Based Access Control

급여정산은 허용된 **IP Address** 로 접속한 경우만 접근을 허용한다

출퇴근 타각은 허용된 **GPS 좌표 범위**에서만 허용한다

특정 민감 문서는 업무 **시간**에만 열람을 허용한다

=> Contextual Tuples

 OpenFGA™

Home Docs API Blog

GitHub | 3729TwitterCNCf Slack

Search

Blocklists

Public Access

Multiple Restrictions

Custom Roles

Conditions

Token claims as Contextual Tuples

Contextual and Time-Based Authorization

Authorization Through Organization Context

Testing Models

Modular Models

> Building Blocks

▼ Advanced Use-Cases

Google Drive

GitHub

Slack

IoT

Entitlements

> Migrations

> Interacting with the API

> Best Practices

<<

> Modeling Guides > Advanced Use-Cases > Google Drive

Modeling Google Drive permissions with OpenFGA

This tutorial explains how to represent Google Drive permissions model with OpenFGA.

What you will learn

- Indicate **relationships** between a group of **users** and an **object**. See [Modeling User Groups](#) for more.
Used here to indicate that all users within a domain can access a document (sharing a document within an organization).
- Model **concentric relationship** to have a certain **relation** on an object imply another relation on the same object. See [Modeling Concepts: Concentric Relationships](#) for more.
Used here is to indicate that writers are also commenters and viewers.
- Using **the union operator** condition to indicate that a user might have a certain relation with an object if they match any of the criteria indicated.
Used here to indicate that a user can be a viewer on a document, or can have the viewer relationship implied through commenter.
- Using the **type bound public access** in a **relationship tuple's** user field to indicate that everyone has a certain relation with an object. See [Modeling Public Access](#) for more.
Used here to [share documents publicly](#).
- Model **parent-child objects** to indicate that a user having a relationship with a certain object implies having a relationship with another object in OpenFGA.
Used here is to indicate that a writer on a folder is a writer on all documents inside that folder.

Before you start

concepts

What you will be modeling

Requirements

Defined scenarios

Modeling Google Drive's permissions

01. Individual permissions

02. Organization permissions

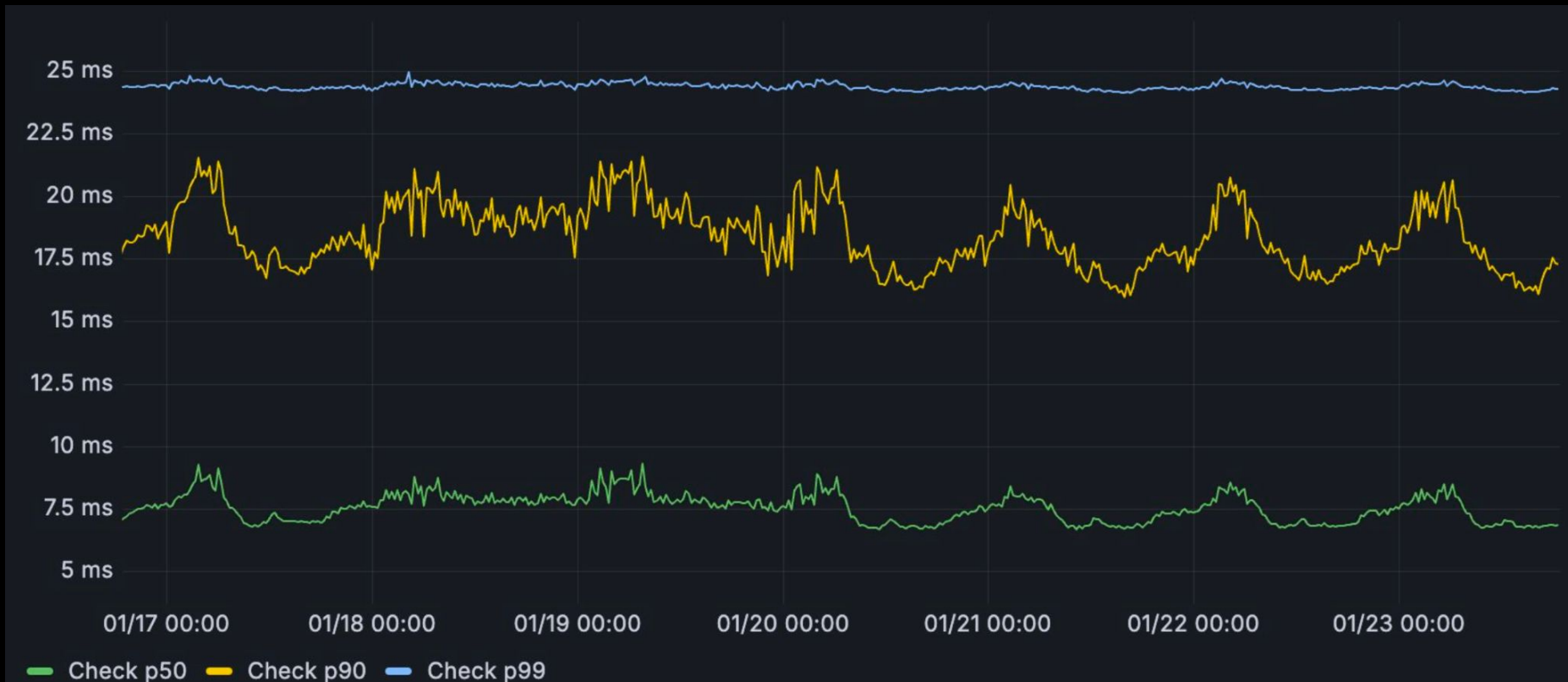
03. Folder permission propagation

04. Sharing files and folders publicly

Related Sections

통합 인가 플랫폼 분리 효과

성능 개선 : 7.5ms (p50)



비즈니스 로직(도메인)과 인가 논리(플랫폼) 독립 개발 가능

- 도메인과 무엇을 체크해야하는지 약속

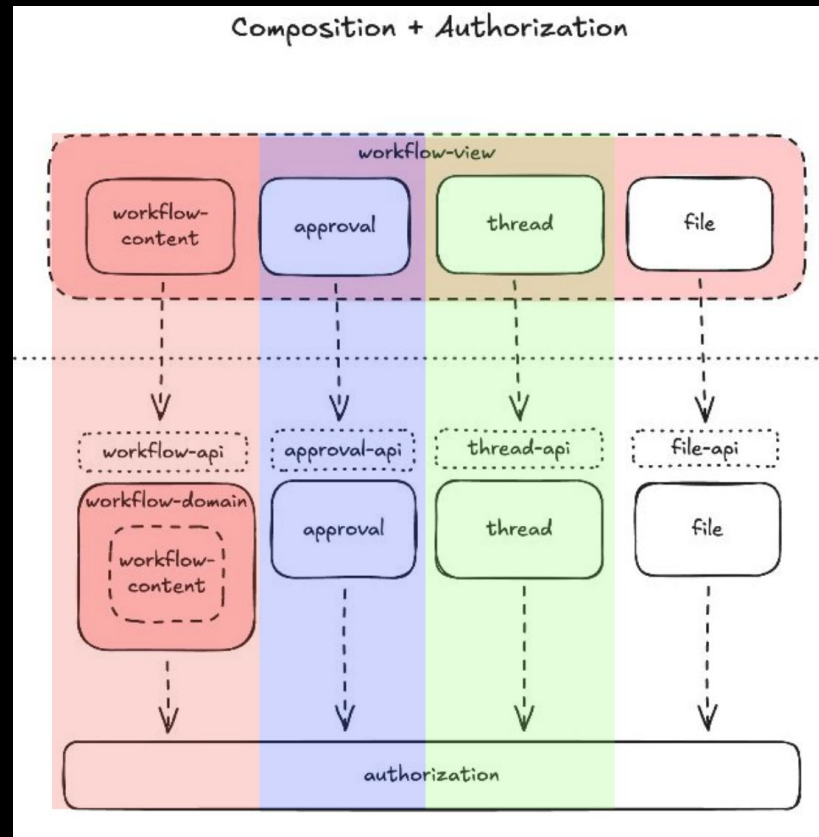
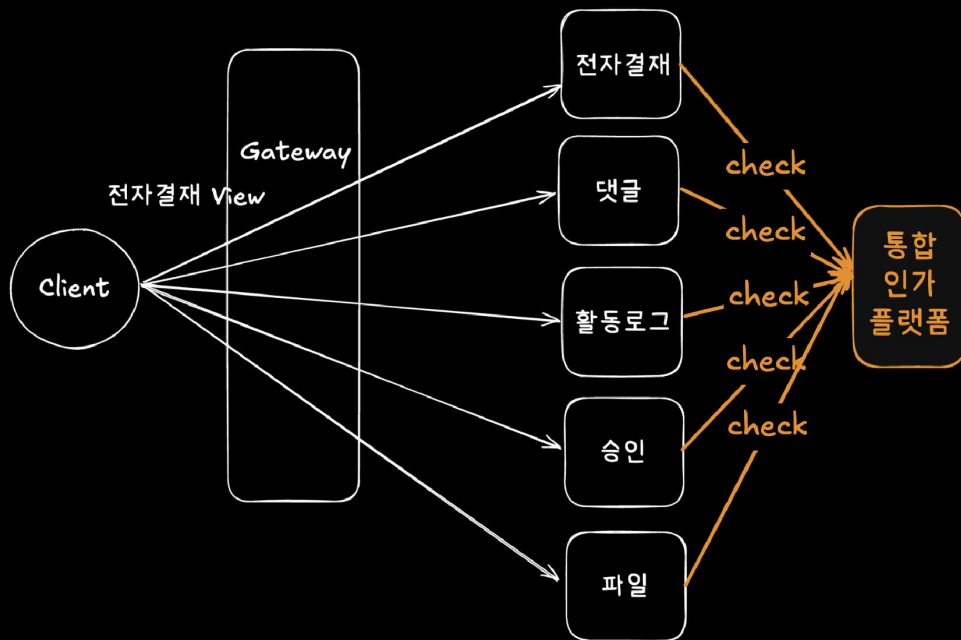
인가 구현의 난이도 감소

- 제품 전체적으로 다양한 권한 기능 제공 가능

공통 컴포넌트의 독립 서비스화

- 재사용성 증가

3. ReBAC : 통합 인가 플랫폼



통합 인가 플랫폼으로 각 도메인 서비스는 인가 판단을 위임

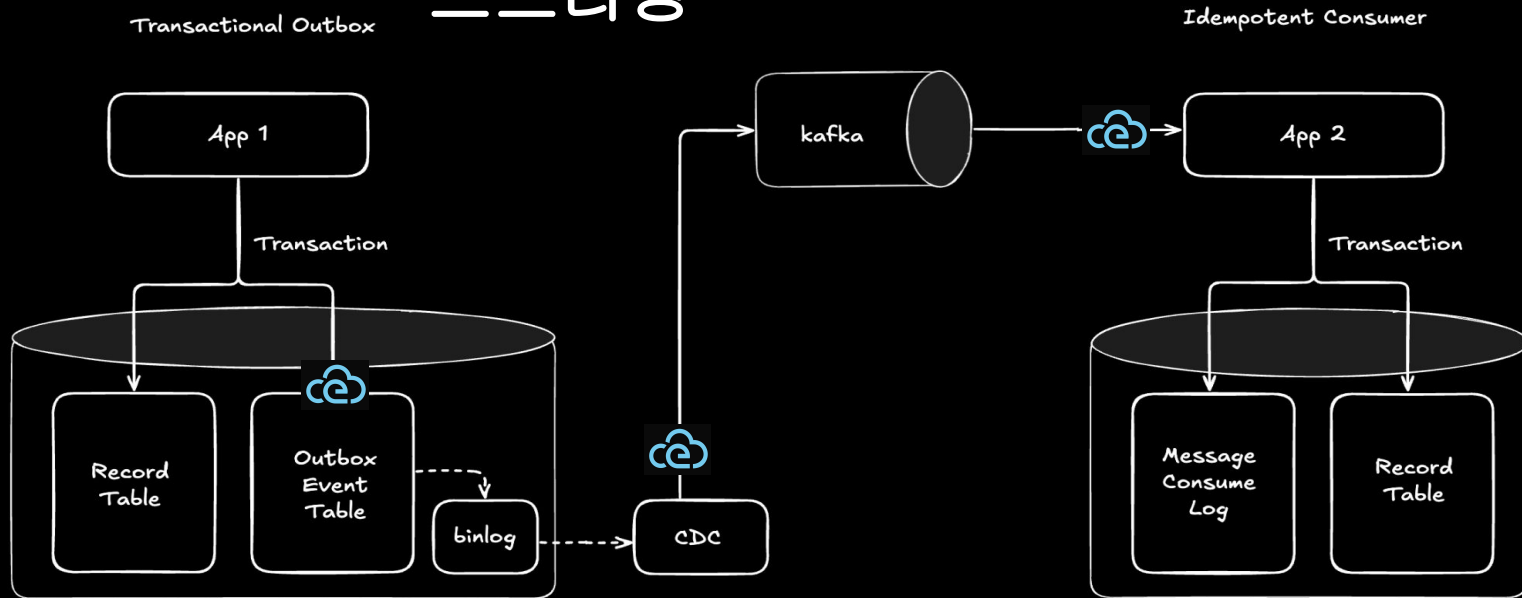
일부 도메인 논리가 분산 관리되는 것은 **통합의 가치 실현**을 위한 **Trade Off** 로 판단

나아가야할 문제:

- 분산 저장된 데이터 일관성 보장
- 인가 판단된 검색 결과
- 비동기 흐름으로 인한 클라이언트 타이밍 문제
- **Authorization <-> CoreHR(임직원)** 양방향 의존 제거

4. 통합의 완성을 위한 플랫폼 고도화

신뢰성 이벤트 스트리밍



flexible #1: 7ms로 끝내는 인가 체크, 분산환경에서 신뢰성있는 이벤트 스트리밍 환경 구축



FAQDOCUMENTATIONRELEASESCOMMUNITYBLOG

Debezium

Stream changes from

Debezium is an open source distributed system that captures changes from a wide variety of databases, including MySQL, PostgreSQL, Oracle, and Microsoft SQL Server. It can capture changes from a database and stream them to a message broker, such as Apache Kafka, or to a data warehouse, such as Amazon Redshift. Debezium can also capture changes from a database and stream them to a data lake, such as Amazon S3, or to a data stream processing engine, such as Apache Flink. Debezium can also capture changes from a database and stream them to a data analytics engine, such as Apache Spark. Debezium can also capture changes from a database and stream them to a data visualization engine, such as Tableau. Debezium can also capture changes from a database and stream them to a data integration engine, such as Informatica. Debezium can also capture changes from a database and stream them to a data migration engine, such as Talend. Debezium can also capture changes from a database and stream them to a data backup engine, such as Veeam. Debezium can also capture changes from a database and stream them to a data recovery engine, such as Veritas. Debezium can also capture changes from a database and stream them to a data archiving engine, such as Commvault. Debezium can also capture changes from a database and stream them to a data retention engine, such as EMC. Debezium can also capture changes from a database and stream them to a data deletion engine, such as IBM. Debezium can also capture changes from a database and stream them to a data purge engine, such as NetScout. Debezium can also capture changes from a database and stream them to a data scrubbing engine, such as DataSentry. Debezium can also capture changes from a database and stream them to a data deduplication engine, such as Dedupix. Debezium can also capture changes from a database and stream them to a data compression engine, such as ZFS. Debezium can also capture changes from a database and stream them to a data encryption engine, such as BitLocker. Debezium can also capture changes from a database and stream them to a data decryption engine, such as BitLocker. Debezium can also capture changes from a database and stream them to a data backup engine, such as Veeam. Debezium can also capture changes from a database and stream them to a data recovery engine, such as Veritas. Debezium can also capture changes from a database and stream them to a data archiving engine, such as Commvault. Debezium can also capture changes from a database and stream them to a data retention engine, such as EMC. Debezium can also capture changes from a database and stream them to a data deletion engine, such as IBM. Debezium can also capture changes from a database and stream them to a data purge engine, such as NetScout. Debezium can also capture changes from a database and stream them to a data scrubbing engine, such as DataSentry. Debezium can also capture changes from a database and stream them to a data deduplication engine, such as Dedupix. Debezium can also capture changes from a database and stream them to a data compression engine, such as ZFS. Debezium can also capture changes from a database and stream them to a data encryption engine, such as BitLocker. Debezium can also capture changes from a database and stream them to a data decryption engine, such as BitLocker.

Try our tutorial



Spring Integration6.5.0

Q Search3k + k

Home

Preface

What's New?

Overview

Core Messaging>

Message

Message Routing>

Message Transformation>

Messaging Endpoints>

Java DSL>

Groovy DSL

Kotlin DSL

System Management>

Reactive Streams Support

Native Images Support

Why Spring vLearn vProjects vAcademy vSolutions v

Spring Integration / Debezium Support

Debezium Support

[Debezium Engine](#), Change Data Capture (CDC) inbound channel adapter. The `DebeziumMessageProducer` allows capturing database change events, converting them into messages and streaming later to the outbound channels.

You need to include the spring integration Debezium dependency to your project:

MavenGradle

```
compile "org.springframework.integration:spring-integration-debezium:6.5.0"
```

GROOVY

You also need to include a [debezium connector](#) dependency for your input Database. For example to use Debezium with PostgreSQL you will need the postgres debezium connector:

MavenGradle

```
compile "io.debezium:debezium-connector-postgres:{debezium-version}"
```

GROOVY

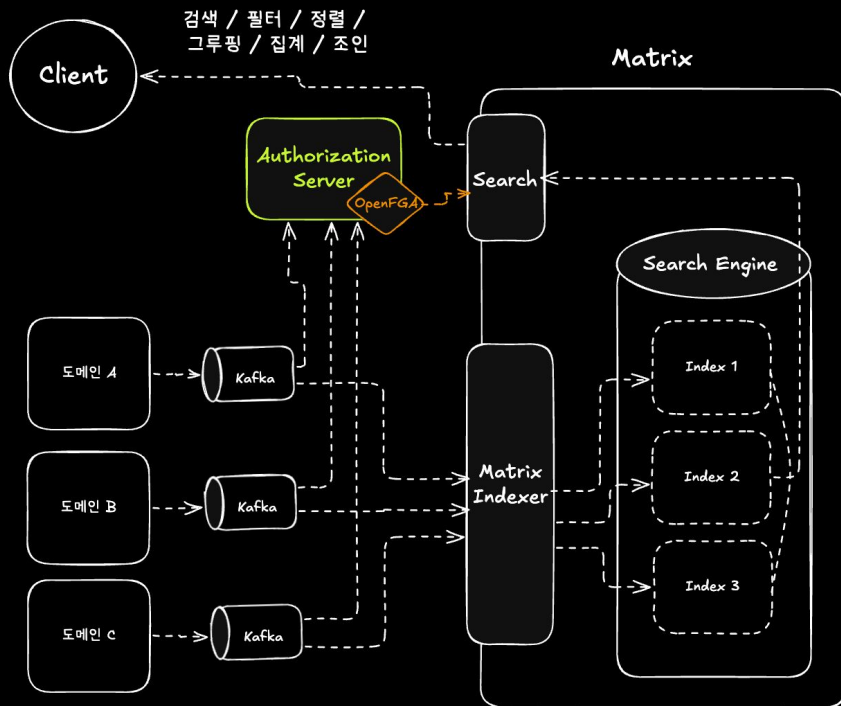
NOTE

Replace the `debezium-version` with the version compatible with the `spring-integration-debezium` version being used.

통합 검색 플랫폼 (Matrix)

MSA 로 분산된 도메인 데이터의
통합 검색 / 필터 / 정렬 / 그룹핑
지원

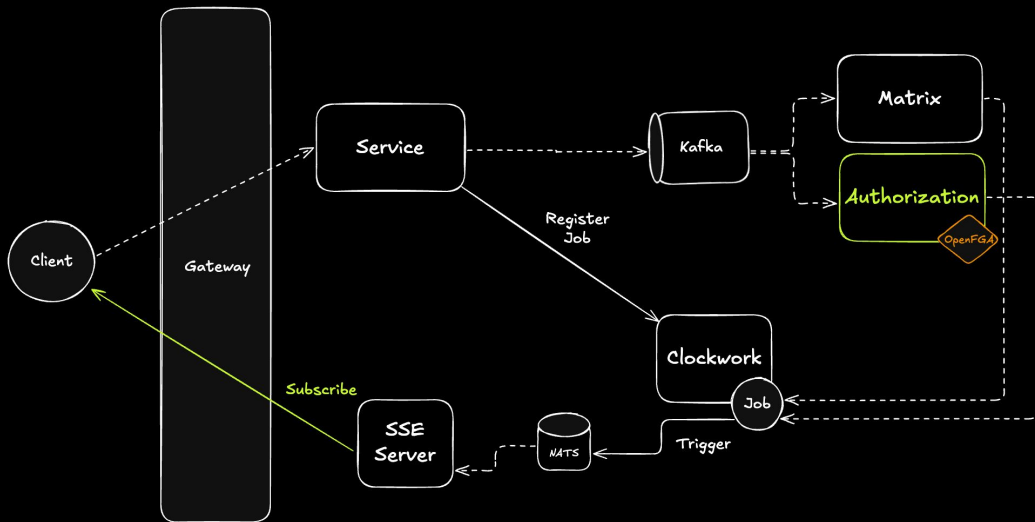
통합 인가 플랫폼과의 연계
인가 판단이 적용된 검색 플랫폼



Server Sent Event Process

비동기로 완결되는
비즈니스 변경 프로세스에서

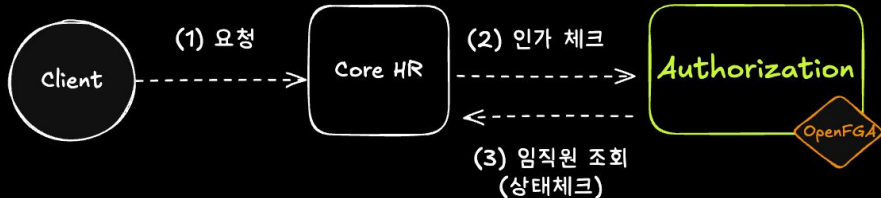
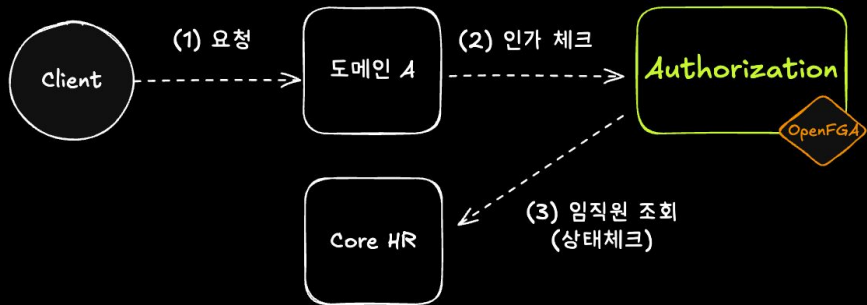
Client 는 SSE 를 구독하여
결과를 통지 받고
사용자에게 후속 경험을 제공한다



Core HR(임직원) 서비스 양방향 의존 문제

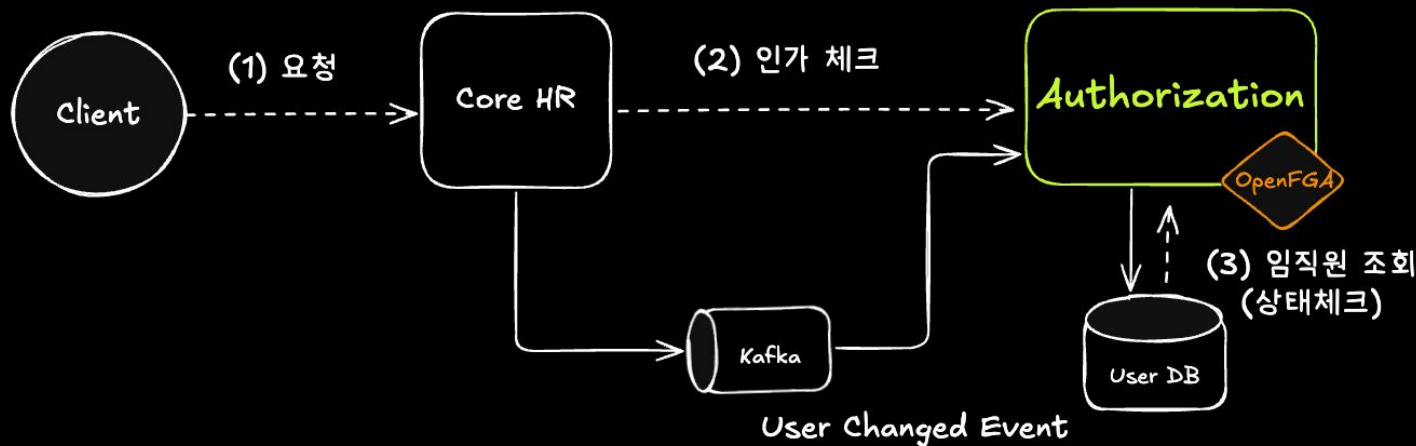
인가 체크는 요청자 상태 체크가 포함

Core HR 요청과 인가 체크에는 양방향 의존과 호출 문제 발생



Core HR(임직원) 서비스 양방향 의존 문제

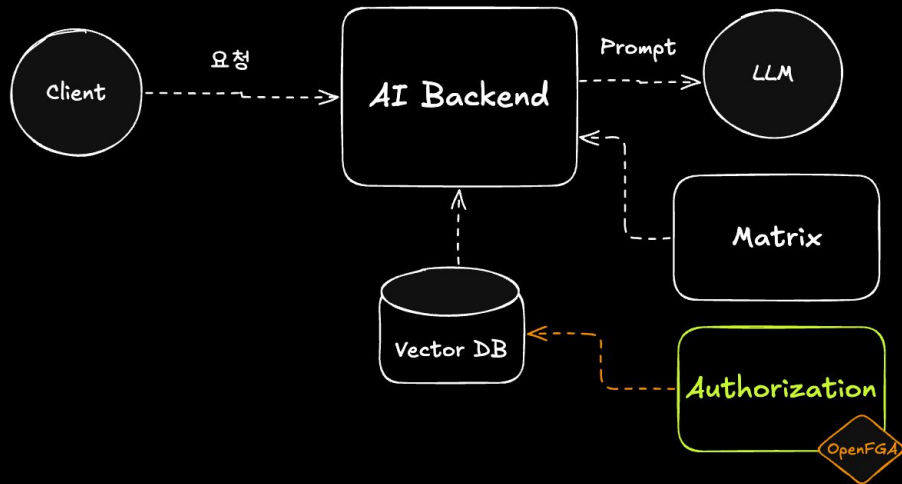
인가 체크에 필요한 유저 상태 정보를 복제 관리하여 양방향 의존 제거



AI Vector DB

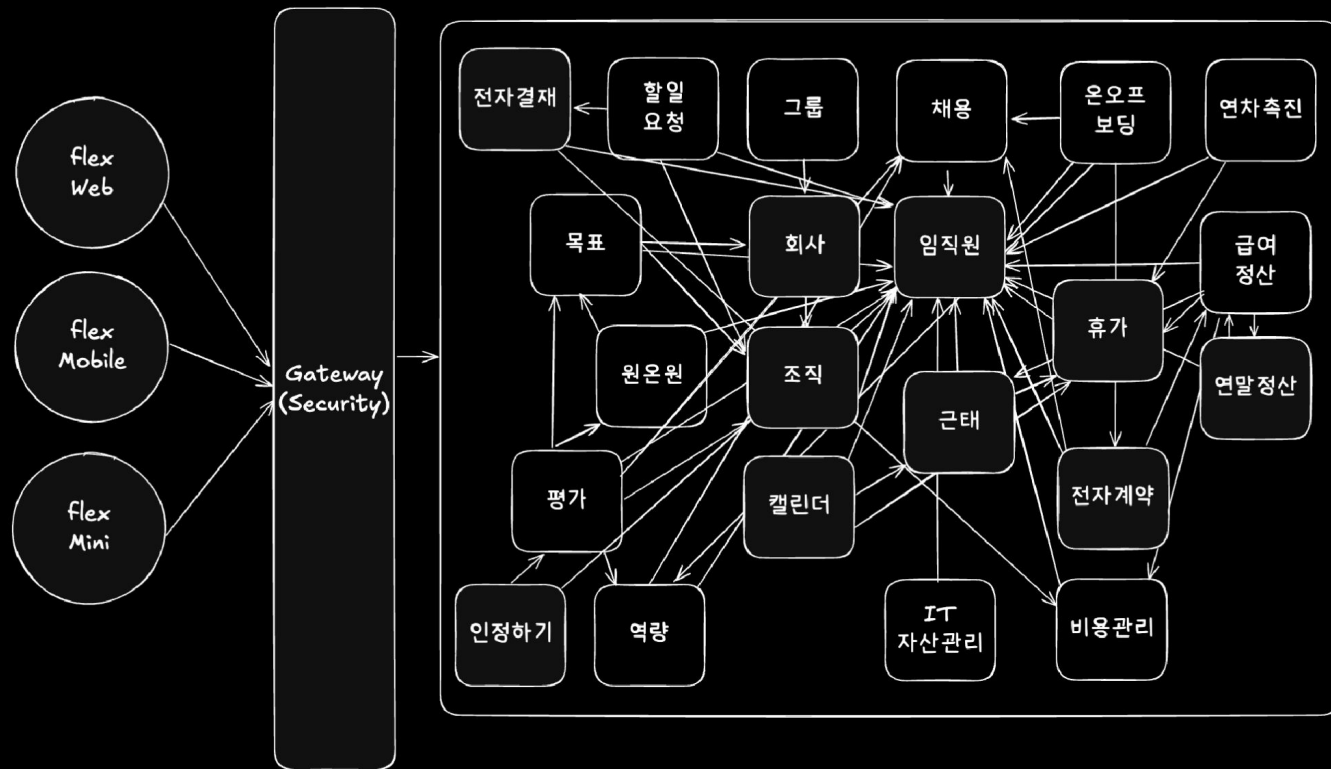
인가 판단을 적용한

RAG Vector Store

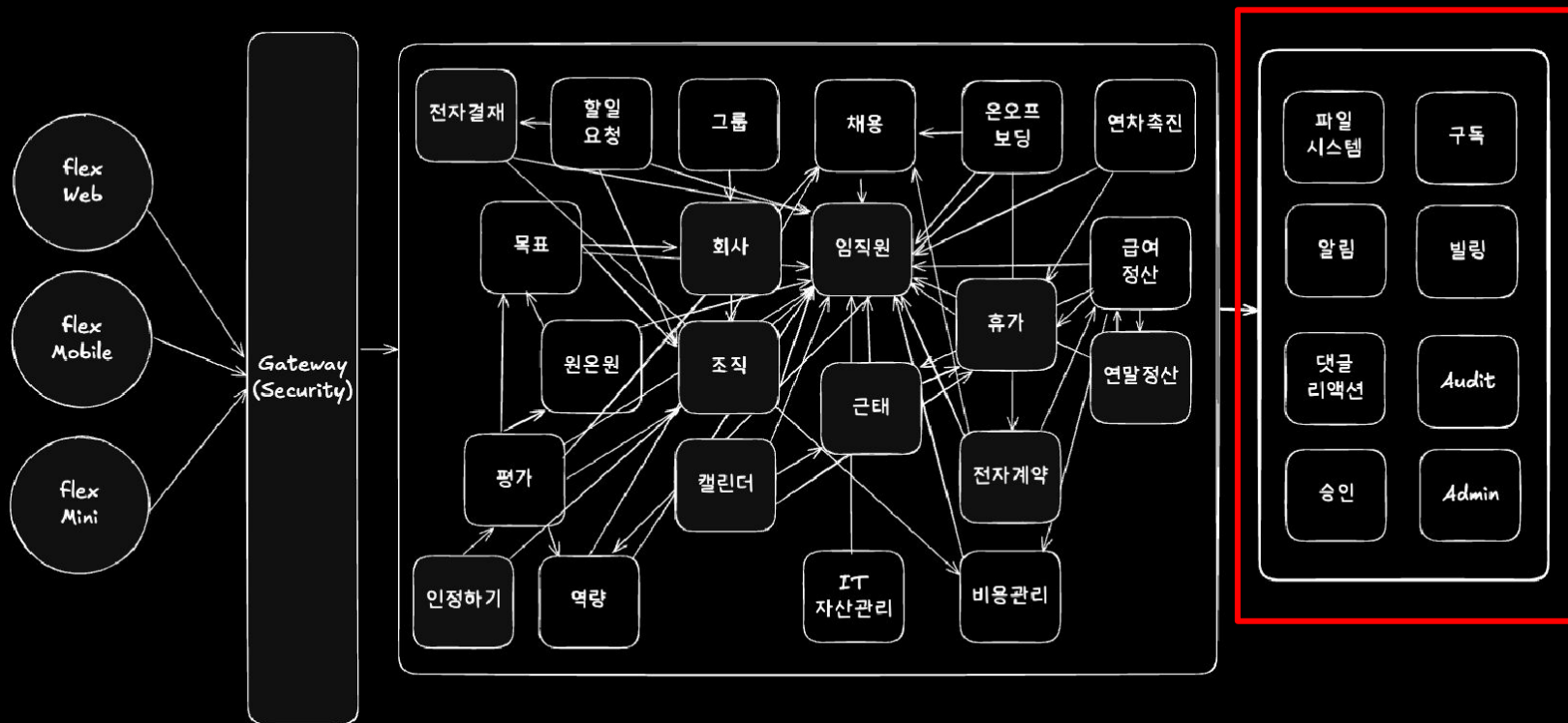


5. 단단한 플랫폼 위에 쌓아 올리는 통합의 가치 실현

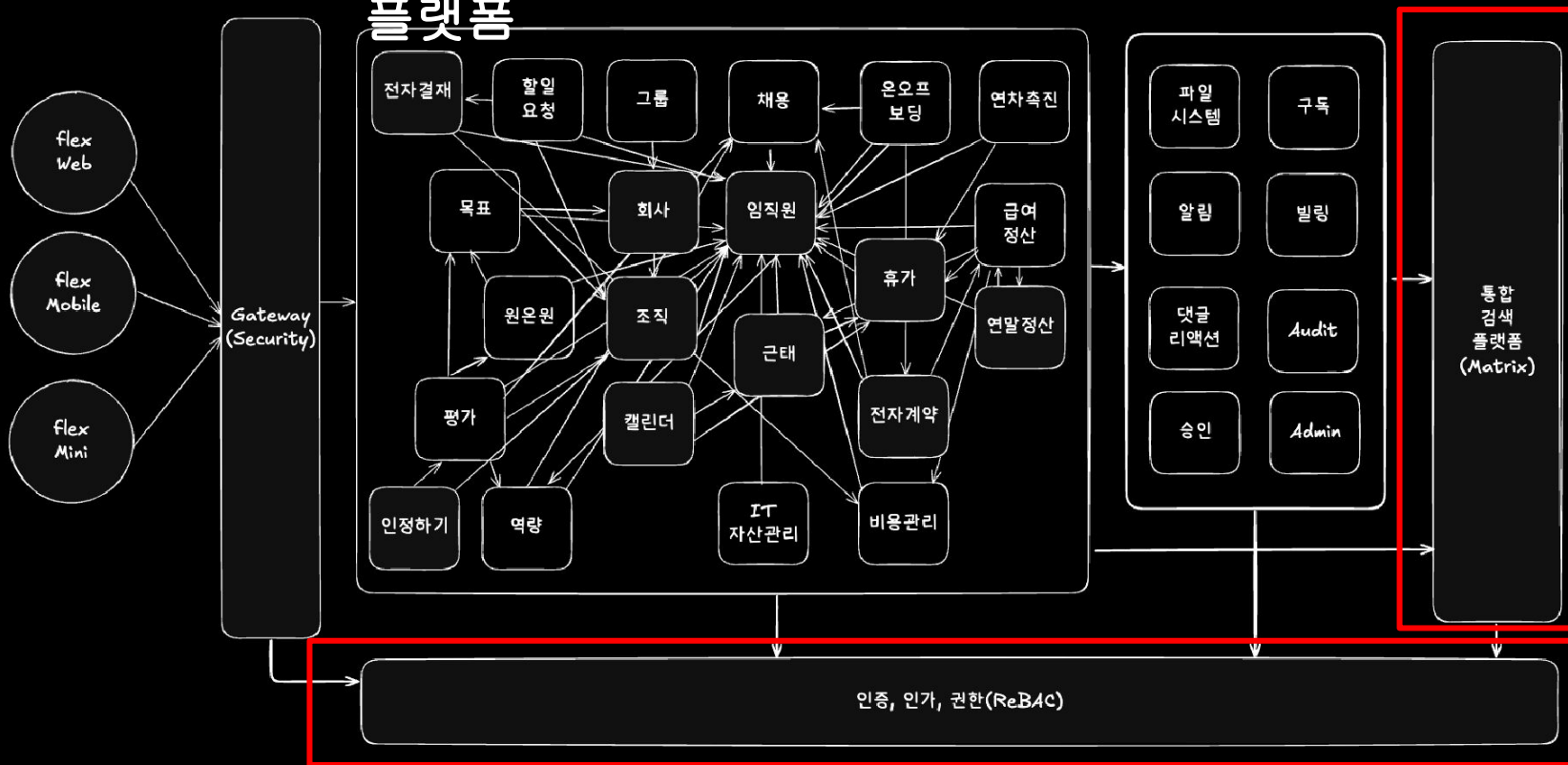
도메인 확장과 함께 진화해온 아키텍처



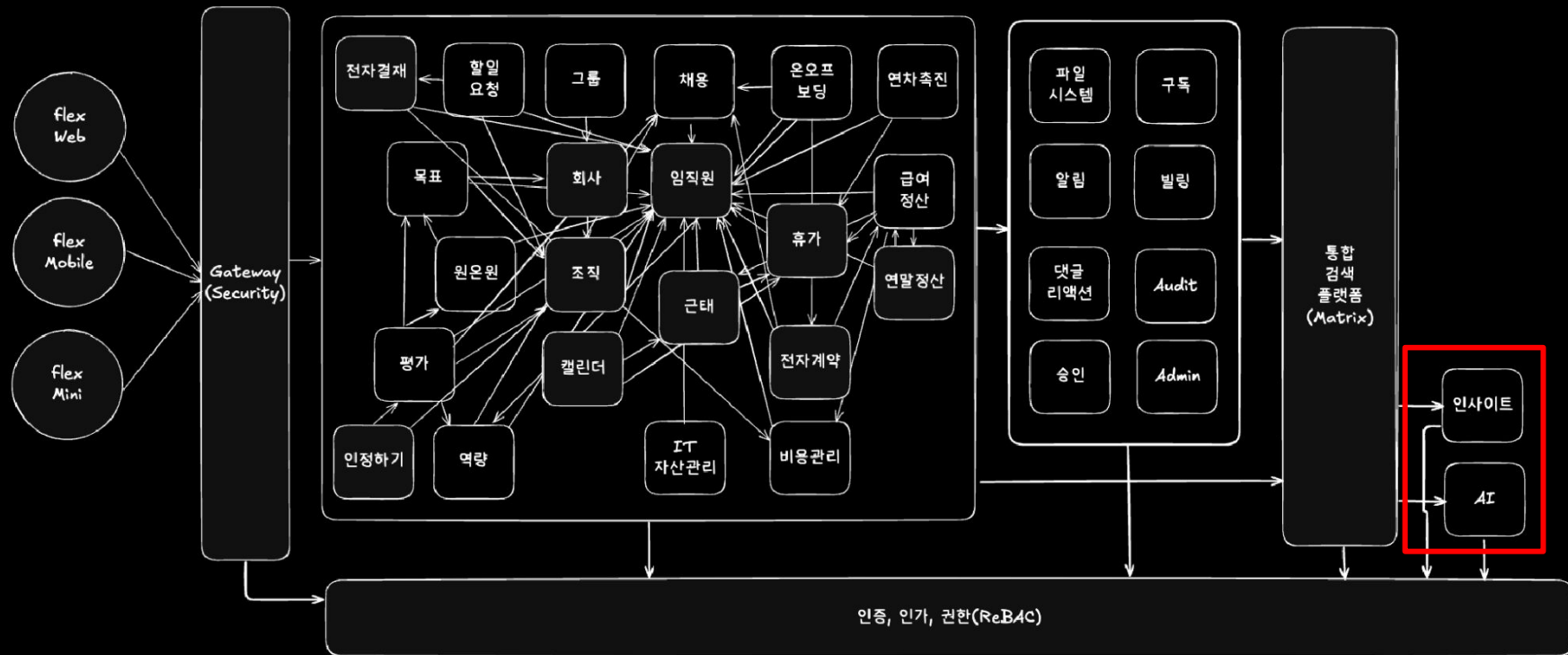
공통 서비스 플랫폼 그초



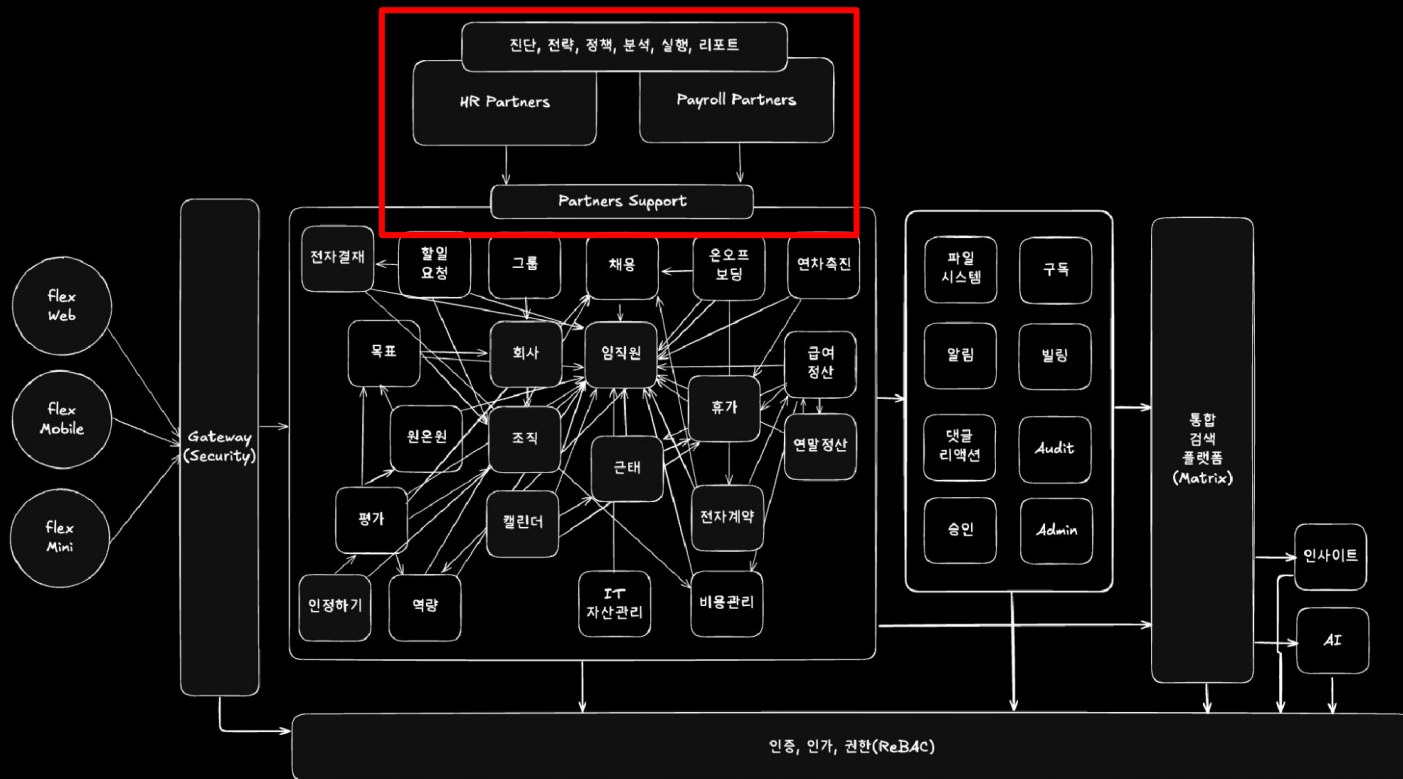
통합 인가 플랫폼 + 검색 플랫폼 = 백본 플랫폼



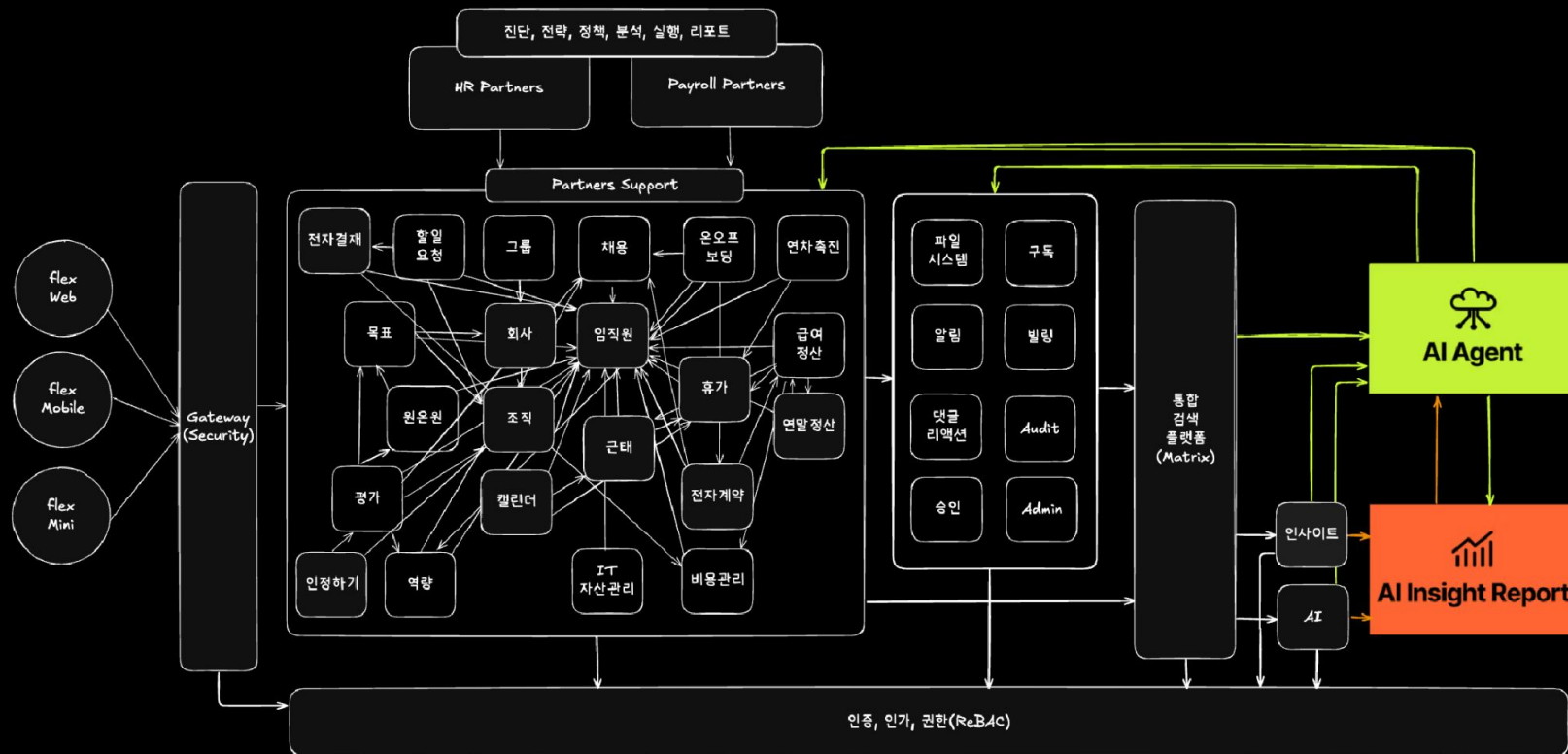
인사이드, AI



파트너스 : 진단, 전략, 정책, 분석, 실행,



Next flex : AI SaaS



Closing

복잡한 이해관계자가 통합되는 B2B Enterprise 확장에서
권한/인가는 해결해야 되는 문제입니다

통합의 가치가 중요한 서비스를 고민하고 있다면,
플렉스팀에서 고민하고 해결했던 경험에서
다음 레벨로 도약할 수 있는 실마리를 찾을 수 있으면 좋겠습니다

flexible 인가 이야기 시리즈

flexible #1: 7ms로 끝내는 인가 체크, 분산환경에서 신뢰성있는 이벤트 스트리밍
환경 구축

Spring Camp 2025: 통합의 근본! 인가 플랫폼

[7/16 예정] flexible #6: 당신의 서비스가 확장하지 못하는 이유 - 통합 인가와
재사용성

flexible 인가 이야기 시리즈

flexible #1: 7ms로 끝내는 인가 체크, 분산환경에서 신뢰성있는 이벤트 스트리밍 환경 구축

Spring Camp 2025: 통합의 근본! 인가 플랫폼

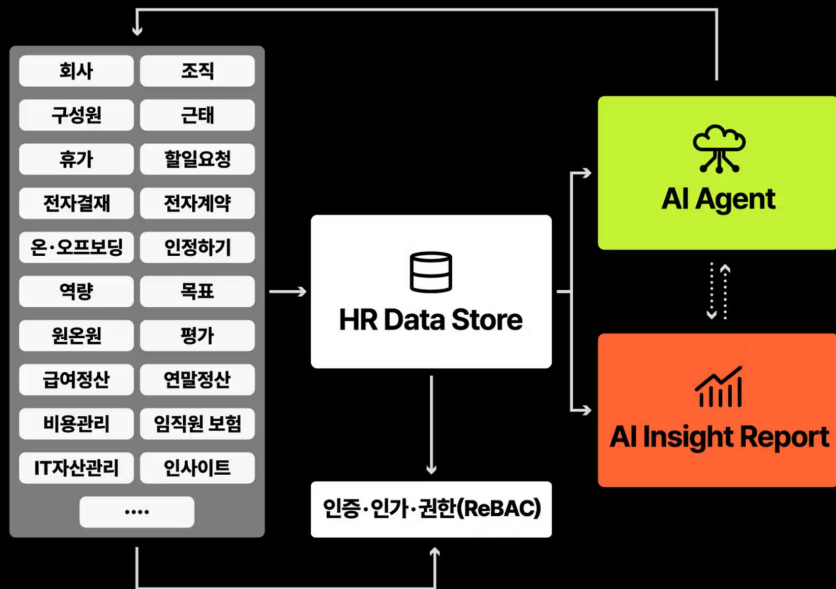
[7/16 예정] flexible #6: 당신의 서비스가 확장하지 못하는 이유 - 재사용성

Discord 채널 + flexible #6 신청



복잡성

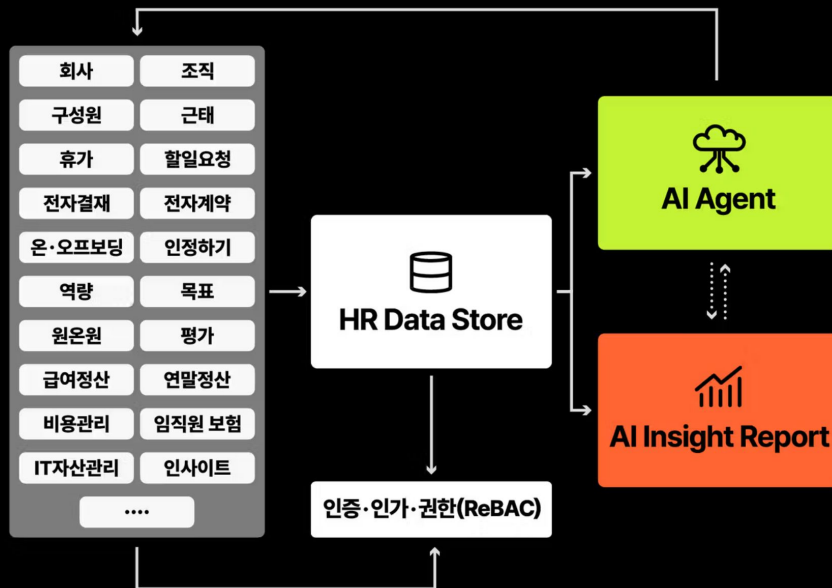
flex AI team
tech architecture



복잡성

같이 풀어 보실 분?

flex AI team
tech architecture



Product Engineer (Backend)



flex AI team
tech architecture

